

Lemma Supplement: A Lean-Verified Single-Transition Theorem for CMV Ball Density

Proof built, Lean-verified, and semantically accepted by
Agentic Lean Math Assistant · github.com/RosaRojacr/agentic-lean-math-assistant

Technical supplement and declaration guide

The entries give the mathematical content of each declaration in the publication closure. Declarations classified as main support the conventional proof; supplement entries record the elementary trigonometric identities, algebra, continuity facts, and domain-transport details used to implement it. Each entry states its mathematical hypotheses and conclusion separately from the source metadata and Lean code supplied with it. There are no configured generated families and no generated declarations in this inventory, so no family representatives or computational certificate ledger are required.

Use the main proof's notation: $S(t) = t - \text{sintcost}$, $a = \arccos \lambda$, $s = \sqrt{1 - \lambda^2}$, $\beta_\lambda(R) = \pi/2 - \arctan((R - \lambda)/s)$, and $b(R) = \arctan(1/R)$. Angular area and perimeter functions are denoted by A, P ; their radius versions by A, P ; and the equal-area profiles by I . Write $Q_B(R) = \pi - S(\beta_\lambda(R))$ and $Q_C(R) = \pi - (1 - \lambda)S(b(R))$. The functions H, D, L, N , common-perimeter inverses r_B, r_C , and area difference G are those defined in the main proof. Density dependence is suppressed in this notation except when density is the differentiation variable.

The formulas are total real-valued expressions in the source, but their geometric interpretation is restricted to the stated admissible domains. In particular, no geometric meaning is assigned here to a totalized division at an excluded endpoint. A derivative described as strict is the two-point derivative limit used by the inverse function theorem; it is not an assertion that the derivative is strictly positive. Common-perimeter inverses are defined only for $p > 2\lambda\pi$. The auxiliary clamping operation merely realizes their area difference as an ordinary real function on a fixed upper interval. It does not change a candidate family or extend an inverse to an inadmissible perimeter. The supplement supplies analytic details, not the unformalized geometric classification discussed at the close of the main proof.

Conceptual declaration ledger

CMVBallCase.AdmissibleDensity

Classification: main

Source: CMVBallDefinitions.lean:8-10

Declaration SHA-256: 4e2cd6052fa41c3d2768e49b680c9466bd3d9e7beae3a567c7ebfa8e5256a410

Mathematical statement. An admissible density parameter is a real λ satisfying $0 < \lambda$ and $\lambda < 1$.

The predicate abbreviates exactly

$$0 < \lambda < 1.$$

It excludes both constant-density endpoints and adds no condition about candidate optimality or minimizer classification.

```
def AdmissibleDensity (lam : ℝ) : Prop := 0 < lam ∧ lam < 1

/-- The source angle  $\alpha = \beta + \arccos \lambda$  for a type-(B) candidate. -/
```

CMVBallCase.BAdmissible

Classification: main

Source: CMVBallCase.lean:18-22

Declaration SHA-256: e044429fd7ffa85e0d7057933c7cab810b11565444bcff740b4688861a6b8c0d

Mathematical statement. A type-B angle is admissible precisely when $0 < \beta \leq \pi - \arccos \lambda$.

The predicate is

$$0 < \beta \quad \text{and} \quad \beta \leq \pi - \arccos \lambda.$$

For $0 < \lambda < 1$ it retains the zero-radius endpoint and excludes the infinite-radius limit $\beta = 0$. The predicate itself does not impose the density hypothesis.

```
def BAdmissible (lam beta : ℝ) : Prop :=
  0 < beta ∧ beta ≤ Real.pi - Real.arccos lam

/-- Finite type-(C) orthogonal-ball candidates. Both angular endpoints are
excluded degenerations. -/
```

CMVBallCase.CAdmissible

Classification: main

Source: CMVBallCase.lean:23-27

Declaration SHA-256: 0caecffaa369b9ef07b21d623e4e6c06f6a3df2ac3a26609e00159b4e2dad1f5

Mathematical statement. A type-C angle is admissible precisely when $0 < \beta < \pi/2$.

The domain is the open interval

$$0 < \beta < \frac{\pi}{2}.$$

Its endpoints correspond to infinite and zero radius, respectively, and neither is a finite positive-radius candidate.

```
def CAdmissible (beta : ℝ) : Prop :=
  0 < beta ∧ beta < Real.pi / 2

/-- On the audited type-(C) angle domain, the orthogonal-ball radius is
strictly positive. -/
```

CMVBallCase.CommonPerimeter

Classification: main

Source: CMVBallCase.lean:1832-1840

Declaration SHA-256: e2043ced3f49efd166f38b27622f70ea021fb2c91e44ee48e02ae0fb32781041

Mathematical statement. For a real λ , the common-perimeter domain consists of the real p with $p > 2\lambda\pi$.

The domain is

$$J_\lambda = (2\lambda\pi, \infty).$$

Packaging the inequality with p ensures that inverse radii are defined only where both positive-radius perimeter equations are proved solvable.

```
def CommonPerimeter (lam : ℝ) : Type :=
  {p : ℝ // 2 * lam * Real.pi < p}

instance commonPerimeterLinearOrder (lam : ℝ) :
  LinearOrder (CommonPerimeter lam) :=
```

```
inferInstanceAs (LinearOrder {p : ℝ // 2 * lam * Real.pi < p})

/-- Every positive type-(C) radius has perimeter strictly larger than `π R`.
This supplies the unbounded endpoint needed for the common-perimeter inverse. -/
```

CMVBallCase.arcSegment

Classification: main

Source: CMVBallCase.lean:488-489

Declaration SHA-256: a0f37e78a16fab57786c18fa0ca15e0aec499e6edf77b3e0d77a98c1a7c7cdec

Mathematical statement. $S(x)$ is defined as $x - \sin x \cos x$.

The definition is

$$S(x) = x - \sin x \cos x.$$

For $0 \leq x \leq \pi$, this is the unit-radius segment area with central angle $2x$: sector area x minus signed triangle area $\sin x \cos x$. It is not twice that segment area.

```
def arcSegment (x : ℝ) : ℝ := x - Real.sin x * Real.cos x
```

CMVBallCase.arcSegment_hasDerivAt

Classification: main

Source: CMVBallCase.lean:490-512

Declaration SHA-256: 32a91ac001b703b525fd39e1c6017868329abe912be28a6cd8b30291e39dfc4d

Mathematical statement. If f has derivative f' at x , then $S \circ f$ has derivative $2\sin^2(f(x))f'$ there.

Differentiate $S(t) = t - \sin t \cos t$ and apply the chain rule to obtain

$$S'(t) = 2\sin^2 t, \quad (S \circ f)'(x) = 2\sin^2(f(x))f'.$$

No angular-domain restriction is required.

```
lemma arcSegment_hasDerivAt {f : ℝ → ℝ} {f' x : ℝ}
  (hf : HasDerivAt f f' x) :
  HasDerivAt (fun y => arcSegment (f y))
    (2 * Real.sin (f x) ^ 2 * f') x := by
  have hraw := hf.sub (hf.sin.mul hf.cos)
  have hfun : HasDerivAt
    (fun y => arcSegment (f y))
    (f' - (Real.cos (f x) * f' * Real.cos (f x) +
      Real.sin (f x) * (-Real.sin (f x) * f')) x := by
    convert hraw using 1 <>
    first | rfl | exact Subsingleton.elim _ _ | (funext y; rfl)
  have htrig := Real.sin_sq_add_cos_sq (f x)
  have hcos : Real.cos (f x) ^ 2 = 1 - Real.sin (f x) ^ 2 := by
    linarith
  have hcoef : 2 * Real.sin (f x) ^ 2 * f' =
    f' - (Real.cos (f x) * f' * Real.cos (f x) +
      Real.sin (f x) * (-Real.sin (f x) * f')) := by
    rw [show Real.cos (f x) * f' * Real.cos (f x) =
      Real.cos (f x) ^ 2 * f' by ring, hcos]
    ring
  rw [hcoef]
  exact hfun
```

CMVBallCase.arcSegment_lt_self

Classification: main

Source: CMVBallCase.lean:767-775

Declaration SHA-256: 7f8e1ec7190e9bd71f5255f6b427e61d471c4bb1d11fb61e4053360c07533a98

Mathematical statement. If $0 < x < \pi/2$, then $S(x) < x$.

Both sine and cosine are positive on this interval. Thus subtracting $\sin x \cos x$ from x gives a strictly smaller value, as required in the type-C estimates.

```
lemma arcSegment_lt_self {x : ℝ} (hx0 : 0 < x) (hx2 : x < Real.pi / 2) :  
  arcSegment x < x := by  
  unfold arcSegment  
  have hs := Real.sin_pos_of_pos_of_lt_pi hx0  
    (by linarith [hx2, Real.pi_pos])  
  have hc := Real.cos_pos_of_mem_Ioo  
    (by linarith [hx0, Real.pi_pos], hx2)  
  nlinarith
```

CMVBallCase.arcSegment_pos

Classification: main

Source: CMVBallCase.lean:610-622

Declaration SHA-256: 4edb95bb9a7ef1f46aba008febfd569d56f56863cb28cc1fdf213082055ae376

Mathematical statement. $S(x) > 0$ whenever $0 < x < \pi$.

If $\cos x \leq 0$, then $S(x) \geq x > 0$. Otherwise $\sin x \cos x \leq \sin x < x$. These cases cover the full interval, not only acute angles.

```
lemma arcSegment_pos {x : ℝ} (hx0 : 0 < x) (hxpi : x < Real.pi) :  
  0 < arcSegment x := by  
  unfold arcSegment  
  have hs : 0 < Real.sin x := Real.sin_pos_of_pos_of_lt_pi hx0 hxpi  
  by_cases hc : Real.cos x ≤ 0  
  · have hp : Real.sin x * Real.cos x ≤ 0 :=  
    mul_nonpos_of_nonneg_of_nonpos hs.le hc  
    linarith  
  · have hc1 : Real.cos x ≤ 1 := Real.cos_le_one x  
    have hp : Real.sin x * Real.cos x ≤ Real.sin x :=  
      mul_le_of_le_one_right hs.le hc1  
    linarith [Real.sin_lt hx0]
```

CMVBallCase.bAtRadius

Classification: main

Source: CMVBallCase.lean:131-132

Declaration SHA-256: 4e3efb2ab3dd0b9f28df9260c5b5c743c4b28ba5b46ff3289c5cef1736b7d246

Mathematical statement. The type-C inverse chart is $b(R) = \arctan(1/R)$.

The defining formula is

$$b(R) = \arctan \frac{1}{R}.$$

For $R > 0$, this lies in $(0, \pi/2)$ and solves $\cot b = R$. The source defines a total expression, but only positive radii are used geometrically.

```
def bAtRadius (R : ℝ) : ℝ := Real.arctan (1 / R)
```

CMVBallCase.bAtRadius_admissible

Classification: main

Source: CMVBallCase.lean:284-289

Declaration SHA-256: f75ea7649633f8c47ad6d1362804bfff9e3318e36e14147c0e5b256b30829c2cc

Mathematical statement. Every positive radius R gives an admissible type-C angle $b(R)$.

For $R > 0$, $1/R > 0$, so

$$0 < b(R) = \arctan(1/R) < \frac{\pi}{2}.$$

This supplies the angular representative of every positive radius.

```
lemma bAtRadius_admissible {R : ℝ} (hR : 0 < R) :  
  CAdmissible (bAtRadius R) := by  
  constructor  
  · exact Real.arctan_pos.2 (one_div_pos.mpr hR)  
  · exact Real.arctan_lt_pi_div_two _
```

CMVBallCase.bAtRadius_typeCRadius

Classification: main

Source: CMVBallCase.lean:296-303

Declaration SHA-256: 79019d0015f2e75012a056920022f8e8538d02c8a466b649dea10f1677f1855a

Mathematical statement. For every admissible type-C angle β , applying b to its radius recovers β .

The radius is positive, and the reconstructed angle is admissible with the same radius. Injectivity of cotangent on $(0, \pi/2)$ proves

$$b(1/\tan\beta) = \beta.$$

```
lemma bAtRadius_typeCRadius {beta : ℝ} (hbeta : CAdmissible beta) :  
  bAtRadius (typeCRadius beta) = beta := by  
  apply typeCRadius_injective_on_admissible  
  (bAtRadius_admissible (typeCRadius_pos hbeta)) hbeta  
  rw [typeCRadius_bAtRadius (typeCRadius_pos hbeta)]  
  
/-- The complete positive type-(C) radius range. -/
```

CMVBallCase.betaAtRadius

Classification: main

Source: CMVBallCase.lean:127-130

Declaration SHA-256: a3124126d7f948e4397701451937e291db6bb2593b860fca2145e77ec6602177

Mathematical statement. The type-B inverse chart is $\beta_\lambda(R) = \pi/2 - \arctan((R - \lambda)/\sqrt{1 - \lambda^2})$.

The defining formula is

$$\beta_\lambda(R) = \frac{\pi}{2} - \arctan \frac{R - \lambda}{\sqrt{1 - \lambda^2}}.$$

It is defined for all real inputs as a total real expression. Its inverse-chart interpretation uses $0 < \lambda < 1$ and $R \geq 0$; it covers angles on both sides of $\pi/2$.

```
def betaAtRadius (lam R : ℝ) : ℝ :=  
  Real.pi / 2 - Real.arctan ((R - lam) / Real.sqrt (1 - lam ^ 2))
```

```
/-- The branch-safe type-(C) inverse radius chart. -/
```

CMVBallCase.betaAtRadius_admissible

Classification: main

Source: CMVBallCase.lean:183-199

Declaration SHA-256: 24ebcc1a37c0cdea1e90d38794d7abdbb4d018ce1729c36cafdfb499f7fb0029

Mathematical statement. For $0 < \lambda < 1$ and $R \geq 0$, $\beta_\lambda(R)$ is an admissible type-B angle.

The chart satisfies

$$0 < \beta_\lambda(R) \leq \pi - a.$$

The upper bound follows by comparing its arctangent argument with the argument at $R = 0$.

```
lemma betaAtRadius_admissible {lam R : ℝ}
  (hlam0 : 0 < lam) (hlam1 : lam < 1) (hR : 0 ≤ R) :
  BAdmissible lam (betaAtRadius lam R) := by
  constructor
  · exact betaAtRadius_pos hlam0 hlam1
  · have hsqrt : 0 < Real.sqrt (1 - lam ^ 2) := sqrt_one_sub_sq_pos hlam0 hlam1
    have hquot : -lam / Real.sqrt (1 - lam ^ 2) ≤
      (R - lam) / Real.sqrt (1 - lam ^ 2) := by
      apply (div_le_div_iff_of_pos_right hsqrt).2
      linarith
    have hatan : Real.arctan (-lam / Real.sqrt (1 - lam ^ 2)) ≤
      Real.arctan ((R - lam) / Real.sqrt (1 - lam ^ 2)) :=
      Real.arctan_strictMono.monotone hquot
    rw [arctan_neg_lam_div_sqrt_eq_arccos_sub_pi_div_two hlam0 hlam1] at hatan
    unfold betaAtRadius
    linarith
```

CMVBallCase.betaAtRadius_eq_arctan_reciprocal

Classification: main

Source: CMVBallCase.lean:1508-1514

Declaration SHA-256: 1b10b001ba78b0255df00a8213bf477e9519c5b01dc94215f4dab6c80ce5263f

Mathematical statement. If $0 < \lambda < 1$ and $\lambda < R$, then $\beta_\lambda(R) = \arctan(\sqrt{1 - \lambda^2} / (R - \lambda))$.

The argument $(R - \lambda)/s$ is positive, so the complementary arctangent identity gives

$$\beta_\lambda(R) = \arctan \frac{\sqrt{1 - \lambda^2}}{R - \lambda}.$$

The restriction $R > \lambda$ prevents using this acute-angle chart in the other quadrant.

```
lemma betaAtRadius_eq_arctan_reciprocal {lam R : ℝ}
  (hlam0 : 0 < lam) (hlam1 : lam < 1) (hR : lam < R) :
  betaAtRadius lam R = Real.arctan (Real.sqrt (1 - lam ^ 2) / (R - lam)) := by
  rw [betaAtRadius, ← Real.arctan_inv_of_pos
    (div_pos (sub_pos.mpr hR) (sqrt_one_sub_sq_pos hlam0 hlam1)), inv_div]

/-- The zero-density endpoint is nonsingular for every positive radius. -/
```

CMVBallCase.betaAtRadius_le_pi_div_two

Classification: main

Source: CMVBallCase.lean:148-156

Declaration SHA-256: 6d70e241c44c4fc39707fea591fb93e2c57b4830b8617af8a63dd5d2033c3256

Mathematical statement. For $0 < \lambda < 1$ and $\lambda \leq R$, $\beta_\lambda(R) \leq \pi/2$.

The quotient $(R - \lambda)/s$ is nonnegative, hence so is its arctangent. This bound is used in the large-radius estimates and at the corner $R = 1$ of the density limit.

```
lemma betaAtRadius_le_pi_div_two {lam R : ℝ}
  (hlam0 : 0 < lam) (hlam1 : lam < 1) (hR : lam ≤ R) :
  betaAtRadius lam R ≤ Real.pi / 2 := by
  unfold betaAtRadius
  have hsqrt : 0 < Real.sqrt (1 - lam ^ 2) := sqrt_one_sub_sq_pos hlam0 hlam1
  have hquot : 0 ≤ (R - lam) / Real.sqrt (1 - lam ^ 2) :=
    div_nonneg (by linarith) hsqrt.le
  linarith [Real.arctan_nonneg.2 hquot]
```

CMVBallCase.betaAtRadius_typeBRadius

Classification: main

Source: CMVBallCase.lean:249-259

Declaration SHA-256: 2b33f238d62a297b2ebc523e21829901b4443f725ae563fae3e91de57dabbcb4

Mathematical statement. For $0 < \lambda < 1$ and admissible β , applying β_λ to the type-B radius of β recovers β .

The original radius is nonnegative, so the inverse chart produces an admissible angle with the same radius. Injectivity of the angular radius map gives

$$\beta_\lambda\left(\frac{\sin(\beta + a)}{\sin\beta}\right) = \beta.$$

```
lemma betaAtRadius_typeBRadius {lam beta : ℝ}
  (hlam0 : 0 < lam) (hlam1 : lam < 1) (hbeta : BAdmissible lam beta) :
  betaAtRadius lam (typeBRadius lam beta) = beta := by
  apply typeBRadius_injective_on_admissible hlam0 hlam1
  (betaAtRadius_admissible hlam0 hlam1 (typeBRadius_nonneg hlam0 hlam1 hbeta))
  hbeta
  rw [typeBRadius_betaAtRadius hlam0 hlam1
    (typeBRadius_nonneg hlam0 hlam1 hbeta)]

/-- The B chart has exactly the audited nonnegative radius range. -/
```

CMVBallCase.betaAtRadius_zero

Classification: main

Source: CMVBallCase.lean:272-277

Declaration SHA-256: 2b9379d0da8b78ab25eb062b33cc303712c06916a5c942063bd420dc473f31ab

Mathematical statement. For $0 < \lambda < 1$, $\beta_\lambda(0) = \pi - \arccos\lambda$.

The zero-radius chart value is

$$\beta_\lambda(0) = \pi - \arccos\lambda.$$

This is the actual type-B angular endpoint, not an arbitrary value chosen to define an inverse outside its range.

```

lemma betaAtRadius_zero {lam : ℝ} (hlam0 : 0 < lam) (hlam1 : lam < 1) :
  betaAtRadius lam 0 = Real.pi - Real.arccos lam := by
  rw [betaAtRadius, zero_sub,
    arctan_neg_lam_div_sqrt_eq_arccos_sub_pi_div_two hlam0 hlam1]
  ring

```

CMVBallCase.commonPerimeterAreaGap

Classification: main

Source: CMVBallCase.lean:2108-2116

Declaration SHA-256: 71b55c7b076991056eaa30f6760e5f80aa24222bb5abfcede16ad3020d3377f0

Mathematical statement. For $0 < \lambda < 1$ and $p > 2\lambda\pi$, $G(p)$ is $A_B(r_B(p)) - A_C(r_C(p))$.

The definition is

$$G(p) = A_B(r_B(p)) - A_C(r_C(p)).$$

Both radii realize the same genuine perimeter p . The gap is defined on this open perimeter domain, not on all real perimeters and not as a difference of equal-area profiles.

```

def commonPerimeterAreaGap {lam : ℝ} (hlam : AdmissibleDensity lam)
  (p : CommonPerimeter lam) : ℝ :=
  typeBAreaAtRadius lam (typeBRadiusAtPerimeter hlam p) -
  typeCAreaAtRadius lam (typeCRadiusAtPerimeter hlam p)

/-- A local real representative of the genuine common-perimeter gap exists at
every point. It agrees locally with the subtype-valued inverse construction
and has derivative `r_B - r_C`; no value of either inverse is assigned outside
the genuine perimeter domain. -/

```

CMVBallCase.commonPerimeterAreaGap_at_typeBProfile_nonpos

Classification: main

Source: CMVBallCase.lean:2334-2377

Declaration SHA-256: 455cce8a30fd3fea9935e7a342738957b15a2a9f4b562858dc267332ef60912f

Mathematical statement. If $0 < \lambda < 1$, $v > \lambda\pi$, and $I_C(v) \leq I_B(v)$, then $G(I_B(v)) \leq 0$.

At $p = I_B(v)$, the type-B area is v . The type-C area radius has perimeter at most p , hence is at most $r_C(p)$. Area monotonicity gives

$$A_C(r_C(p)) \geq v, \quad G(p) = v - A_C(r_C(p)) \leq 0.$$

```

lemma commonPerimeterAreaGap_at_typeBProfile_nonpos {lam v : ℝ}
  (hlam : AdmissibleDensity lam) (hv : lam * Real.pi < v)
  (hprofile :
    typeCPerimeterAtArea lam v ≤ typeBPerimeterAtArea lam v) :
  commonPerimeterAreaGap hlam
    (typeBPerimeterAtArea lam v,
      typeBPerimeterAtArea_gt_commonEndpoint hlam.1 hlam.2 hv) ≤ 0 := by
  rcases exists_typeBRadius_at_area hlam.1 hlam.2 hv with
    ⟨RB, hRB, hareaB⟩
  rcases exists_typeCRadius_at_area hlam.1 hlam.2 hv with
    ⟨SC, hSC, hareaC⟩
  change 0 < RB at hRB
  change 0 < SC at hSC
  let p : CommonPerimeter lam :=
    (typeBPerimeterAtArea lam v,
      typeBPerimeterAtArea_gt_commonEndpoint hlam.1 hlam.2 hv)
  have hprofileB :
    typeBPerimeterAtArea lam v = typeBPerimeterAtRadius lam RB :=
    typeBPerimeterAtArea_eq_radius hlam.1 hlam.2 hRB hareaB

```

```

have hprofileC :
  typeCPerimeterAtArea lam v = typeCPerimeterAtRadius lam SC :=
  typeCPerimeterAtArea_eq_radius hlam.1 hlam.2 hSC hareaC
have heqB : (typeBRadiusAtPerimeter hlam p).1 = RB :=
  typeBRadiusAtPerimeter_unique hlam p hRB hprofileB.symm
have hSCle : SC ≤ (typeCRadiusAtPerimeter hlam p).1 := by
  by_contra hnot
  have hlt : (typeCRadiusAtPerimeter hlam p).1 < SC := lt_of_not_ge hnot
  have hperimlt := typeCPerimeterAtRadius_strictMonoOn hlam.1 hlam.2
    (typeCRadiusAtPerimeter_pos hlam p) hSC hlt
  rw [typeCPerimeterAt_typeCRadiusAtPerimeter hlam p,
    ← hprofileC] at hperimlt
  exact (not_lt_of_ge hprofile) hperimlt
have harea_le :=
  (typeCAreaAtRadius_strictMonoOn hlam.1 hlam.2).monotoneOn
  hSC (typeCRadiusAtPerimeter_pos hlam p) hSCle
rw [hareaC] at harea_le
change commonPerimeterAreaGap hlam p ≤ 0
unfold commonPerimeterAreaGap
rw [heqB, hareaB]
linarith

```

-- A negative common-perimeter gap at the B profile value converts back to strict C-profile dominance at the same weighted area. -/

CMVBallCase.commonPerimeterAreaGap_local_hasDerivAt

Classification: main

Source: CMVBallCase.lean:2117-2222

Declaration SHA-256: 8ac1ee8e9903b31ca5fa31187a2831b990514e5786275449b934fb9522a630a5

Mathematical statement. For an admissible density and each $p > 2\lambda\pi$, there is a real function agreeing locally with G on its domain and having derivative $r_B(p) - r_C(p)$ at p .

The statement supplies $g: \mathbb{R} \rightarrow \mathbb{R}$ and a neighborhood U of p such that

$$g'(p) = r_B(p) - r_C(p), \quad g(q) = G(q) \quad (q \in U, q > 2\lambda\pi).$$

Local inverse derivatives and $A' = RP'$ give the formula; no global off-domain inverse is introduced.

```

lemma commonPerimeterAreaGap_local_hasDerivAt {lam : ℝ}
  (hlam : AdmissibleDensity lam) (p : CommonPerimeter lam) :
  ∃ gap : ℝ → ℝ,
    HasDerivAt gap
      ((typeBRadiusAtPerimeter hlam p).1 -
        (typeCRadiusAtPerimeter hlam p).1) p.1 ^
      ∀ f q in nhds p.1, ∀ hq : 2 * lam * Real.pi < q,
        gap q = commonPerimeterAreaGap hlam ⟨q, hq⟩ := by
  let RB : ℝ := typeBRadiusAtPerimeter hlam p
  let RC : ℝ := typeCRadiusAtPerimeter hlam p
  let dB : ℝ := 2 * (Real.pi - betaAtRadius lam RB +
    Real.sin (betaAtRadius lam RB) * Real.cos (betaAtRadius lam RB))
  let dC : ℝ := 2 * (Real.pi - (1 - lam) *
    (bAtRadius RC - Real.sin (bAtRadius RC) * Real.cos (bAtRadius RC)))
  have hPB : HasStrictDerivAt (typeBPerimeterAtRadius lam) dB RB := by
    dsimp [dB, RB]
    exact typeBPerimeterAtRadius_hasStrictDerivAt hlam.1 hlam.2
  have hPC : HasStrictDerivAt (typeCPerimeterAtRadius lam) dC RC := by
    dsimp [dC, RC]
    exact typeCPerimeterAtRadius_hasStrictDerivAt hlam.1 hlam.2
    (typeCRadiusAtPerimeter_pos hlam p)
  have hdB : dB ≠ 0 := by
    have hpos := typeBPerimeterAtRadius_deriv_pos
      (R := RB) hlam.1 hlam.2 (typeBRadiusAtPerimeter_pos hlam p)
    rw [hPB.hasDerivAt.deriv] at hpos
    exact ne_of_gt hpos
  have hdC : dC ≠ 0 := by

```

```

have hpos := typeCPerimeterAtRadius_deriv_pos
  (R := RC) hlam.1 hlam.2 (typeCRadiusAtPerimeter_pos hlam p)
rw [hPC.hasDerivAt.deriv] at hpos
exact ne_of_gt hpos
let gB :  $\mathbb{R} \rightarrow \mathbb{R}$  := hPB.localInverse (typeBPerimeterAtRadius lam) dB RB hdB
let gC :  $\mathbb{R} \rightarrow \mathbb{R}$  := hPC.localInverse (typeCPerimeterAtRadius lam) dC RC hDC
have hpB : typeBPerimeterAtRadius lam RB = p.1 := by
  exact typeBPerimeterAt_typeBRadiusAtPerimeter hlam p
have hpC : typeCPerimeterAtRadius lam RC = p.1 := by
  exact typeCPerimeterAt_typeCRadiusAtPerimeter hlam p
have hgB : HasDerivAt gB  $dB^{-1}$  p.1 := by
  rw [← hpB]
  exact hPB.to_localInverse hdB |>.hasDerivAt
have hgC : HasDerivAt gC  $dC^{-1}$  p.1 := by
  rw [← hpC]
  exact hPC.to_localInverse hDC |>.hasDerivAt
have hgBval : gB p.1 = RB := by
  rw [← hpB]
  exact (hPB.eventually_left_inverse hdB).self_of_nhds
have hgCval : gC p.1 = RC := by
  rw [← hpC]
  exact (hPC.eventually_left_inverse hDC).self_of_nhds
have hAB : HasDerivAt (typeBAreaAtRadius lam) (RB * dB) RB := by
  dsimp [RB, dB]
  exact typeBAreaAtRadius_hasDerivAt hlam.1 hlam.2
    (typeBRadiusAtPerimeter_pos hlam p)
have hAC : HasDerivAt (typeCAreaAtRadius lam) (RC * dC) RC := by
  dsimp [RC, dC]
  exact typeCAreaAtRadius_hasDerivAt (lam := lam)
    (typeCRadiusAtPerimeter_pos hlam p)
have hABcomp : HasDerivAt (typeBAreaAtRadius lam ∘ gB) RB p.1 := by
  have hAB' : HasDerivAt (typeBAreaAtRadius lam) (RB * dB) (gB p.1) := by
    rw [hgBval]
    exact hAB
  have h := hAB'.comp p.1 hgB
  have hcoef : RB * dB *  $dB^{-1}$  = RB := by
    field_simp [hdB]
  rwa [hcoef] at h
have hACcomp : HasDerivAt (typeCAreaAtRadius lam ∘ gC) RC p.1 := by
  have hAC' : HasDerivAt (typeCAreaAtRadius lam) (RC * dC) (gC p.1) := by
    rw [hgCval]
    exact hAC
  have h := hAC'.comp p.1 hgC
  have hcoef : RC * dC *  $dC^{-1}$  = RC := by
    field_simp [hDC]
  rwa [hcoef] at h
refine ⟨(typeBAreaAtRadius lam ∘ gB) -
  (typeCAreaAtRadius lam ∘ gC), ?_, ?_⟩
· change HasDerivAt
  ((typeBAreaAtRadius lam ∘ gB) -
  (typeCAreaAtRadius lam ∘ gC)) (RB - RC) p.1
  exact hABcomp.sub hACcomp
have hrightB := hPB.eventually_right_inverse hdB
have hrightC := hPC.eventually_right_inverse hDC
rw [hpB] at hrightB
rw [hpC] at hrightC
have hposB :  $\forall^f q$  in nhds p.1,  $0 < gB q$  :=
  hgB.continuousAt.eventually (Ioi_mem_nhds (by
    rw [hgBval]
    exact typeBRadiusAtPerimeter_pos hlam p))
have hposC :  $\forall^f q$  in nhds p.1,  $0 < gC q$  :=
  hgC.continuousAt.eventually (Ioi_mem_nhds (by
    rw [hgCval]
    exact typeCRadiusAtPerimeter_pos hlam p))
filter_upwards [hrightB, hrightC, hposB, hposC] with q hqB hqC hqBpos hqCpos
intro hq
let qs : CommonPerimeter lam := ⟨q, hq⟩
have heqB : (typeBRadiusAtPerimeter hlam qs).1 = gB q :=
  typeBRadiusAtPerimeter_unique hlam qs hqBpos hqB
have heqC : (typeCRadiusAtPerimeter hlam qs).1 = gC q :=
  typeCRadiusAtPerimeter_unique hlam qs hqCpos hqC
change typeBAreaAtRadius lam (gB q) -

```

```

typeCAreaAtRadius lam (gC q) = _
unfold commonPerimeterAreaGap
rw [heqB, heqC]

```

```

/-- Clamp a real perimeter to an interval's genuine lower endpoint. This is
used only to transport the subtype gap across a fixed compact interval. -/

```

CMVBallCase.commonPerimeterAreaGap_strictAnti

Classification: main

Source: CMVBallCase.lean:2267-2320

Declaration SHA-256: 91f23a2fb97afb962a3fafd5eb5a6db4289103e7ba32d905aa7120164cb1c340

Mathematical statement. For every $0 < \lambda < 1$, G is strictly decreasing on the common-perimeter domain $(2\lambda\pi, \infty)$.

For $2\lambda\pi < p < q$, use the continuous clamped representative on $[p, q]$. Its interior derivative is $r_B - r_C < 0$, so the mean value theorem gives

$$G(q) < G(p).$$

```

theorem commonPerimeterAreaGap_strictAnti {lam : ℝ}
  (hlam : AdmissibleDensity lam) :
  StrictAnti (commonPerimeterAreaGap hlam) := by
intro p q hpq
let F : ℝ → ℝ := commonPerimeterAreaGapAbove hlam p
have hcont : ContinuousOn F (Set.Icc p.1 q.1) := by
  intro x hx
  have hder := commonPerimeterAreaGapAbove_hasDerivWithinAt hlam p hx.1
  exact (hder.mono (Set.Icc_subset_Ici_self)).continuousWithinAt
have hinterior : interior (Set.Icc p.1 q.1) = Set.Ioo p.1 q.1 :=
  interior_Icc
let radiusGap : ℝ → ℝ := fun x =>
  (typeBRadiusAtPerimeter hlam (commonPerimeterClamp p x)).1 -
  (typeCRadiusAtPerimeter hlam (commonPerimeterClamp p x)).1
have hderiv : ∀ x ∈ interior (Set.Icc p.1 q.1),
  HasDerivWithinAt F (radiusGap x)
  (interior (Set.Icc p.1 q.1)) x := by
intro x hx
have hx' : x ∈ Set.Ioo p.1 q.1 := by rwa [hinterior] at hx
have hlocal0 :=
  commonPerimeterAreaGapAbove_hasDerivWithinAt hlam p hx'.1.le
have hlocal : HasDerivWithinAt (commonPerimeterAreaGapAbove hlam p)
  ((typeBRadiusAtPerimeter hlam
    ⟨x, p.2.trans_le hx'.1.le⟩).1 -
    (typeCRadiusAtPerimeter hlam
    ⟨x, p.2.trans_le hx'.1.le⟩).1)
  (interior (Set.Icc p.1 q.1)) x :=
  hlocal0.mono (by
    intro y hy
    rw [hinterior] at hy
    exact hy.1.le)
change HasDerivWithinAt (commonPerimeterAreaGapAbove hlam p)
  ((typeBRadiusAtPerimeter hlam (commonPerimeterClamp p x)).1 -
  (typeCRadiusAtPerimeter hlam (commonPerimeterClamp p x)).1)
  (interior (Set.Icc p.1 q.1)) x
rw [commonPerimeterClamp_eq p hx'.1.le]
exact hlocal
have hneg : ∀ x ∈ interior (Set.Icc p.1 q.1), radiusGap x < 0 := by
  intro x _
  exact sub_neg.mpr
  (typeBRadiusAtPerimeter_lt_typeCRadiusAtPerimeter hlam
    (commonPerimeterClamp p x))
have hanti : StrictAntiOn F (Set.Icc p.1 q.1) :=
  strictAntiOn_of_hasDerivWithinAt_neg (convex_Icc p.1 q.1)
  hcont hderiv hneg
have hresult := hanti
  (show p.1 ∈ Set.Icc p.1 q.1 from ⟨le_refl, hpq.le⟩)

```

```

(show q.1 ∈ Set.Icc p.1 q.1 from ⟨hpq.le, le_refl⟩) hpq
dsimp [F, commonPerimeterAreaGapAbove] at hresult
rw [commonPerimeterClamp_eq p (le_refl p.1),
  commonPerimeterClamp_eq p hpq.le] at hresult
exact hresult

```

CMVBallCase.existsUnique_typeBRadius_at_perimeter

Classification: main

Source: CMVBallCase.lean:1853-1884

Declaration SHA-256: b6f0fa2f243c3e91136c9bd09e3537e769591131b17de77172ce5ad85508eb6b

Mathematical statement. For $0 < \lambda < 1$ and $p > 2\lambda\pi$, there is exactly one R with $R > 0$ and $P_B(R) = p$.

Start with $P_B(0) = 2\lambda\pi < p$, and use

$$P_B(R) > P_C(R) > \pi R$$

at a large radius. Continuity gives existence and strict perimeter increase gives uniqueness among positive radii.

```

lemma existsUnique_typeBRadius_at_perimeter {lam p : ℝ}
  (hlam0 : 0 < lam) (hlam1 : lam < 1) (hp : 2 * lam * Real.pi < p) :
  ∃! R, 0 < R ∧ typeBPerimeterAtRadius lam R = p := by
  have hp0 : 0 < p := by nlinarith [Real.pi_pos]
  let M := p / Real.pi
  have hM0 : 0 < M := div_pos hp0 Real.pi_pos
  have hBM : p < typeBPerimeterAtRadius lam M := by
    have hC := typeCPerimeterAtRadius_gt_pi_mul_radius hlam0 hlam1 hM0
    have hBC := sameRadius_typeB_gt_typeC hlam0 hlam1 hM0
    dsimp [M] at hC hBC
    have hpi : Real.pi * (p / Real.pi) = p := by
      field_simp [ne_of_gt Real.pi_pos]
    rw [hpi] at hC
    exact hC.trans hBC
  have hB0 : typeBPerimeterAtRadius lam 0 < p := by
    rw [typeBPerimeterAtRadius_zero hlam0 hlam1]
    exact hp
  have hvalue : p ∈ Set.Icc (typeBPerimeterAtRadius lam 0)
    (typeBPerimeterAtRadius lam M) := ⟨hB0.le, hBM.le⟩
  rcases intermediate_value_Icc hM0.le
    typeBPerimeterAtRadius_continuous.continuousOn hvalue with ⟨R, hR, hperim⟩
  have hR0 : 0 < R := by
    rcases hR.1.eq_or_lt with hzero | hpos
    · rw [← hzero, typeBPerimeterAtRadius_zero hlam0 hlam1] at hperim
      linarith
    · exact hpos
  refine ⟨R, ⟨hR0, hperim⟩, ?_⟩
  intro S hS
  apply (typeBPerimeterAtRadius_strictMonoOn hlam0 hlam1).injOn hS.1.le hR0.le
  rw [hS.2, hperim]

/-- A type-(C) radius exists uniquely at every genuine common perimeter. -/

```

CMVBallCase.existsUnique_typeCRadius_at_perimeter

Classification: main

Source: CMVBallCase.lean:1885-1922

Declaration SHA-256: bd2558a1451eb56264500c5c5fcf17b65bf2502c53fb9be537238e05e61eadd9

Mathematical statement. For $0 < \lambda < 1$ and $p > 2\lambda\pi$, there is exactly one R with $R > 0$ and $P_C(R) = p$.

The radius $R = \lambda$ has perimeter less than $2\lambda\pi$; sufficiently large radii have perimeter greater than p . The intermediate value theorem and strict increase give existence and uniqueness without evaluating at radius zero.

```

lemma existsUnique_typeCRadius_at_perimeter {lam p : ℝ}
  (hlam0 : 0 < lam) (hlam1 : lam < 1) (hp : 2 * lam * Real.pi < p) :
  ∃! R, 0 < R ∧ typeCPerimeterAtRadius lam R = p := by
  have hp0 : 0 < p := by nlinarith [Real.pi_pos]
  let M := p / Real.pi
  have hM0 : 0 < M := div_pos hp0 Real.pi_pos
  have hCM : p < typeCPerimeterAtRadius lam M := by
    have hC := typeCPerimeterAtRadius_gt_pi_mul_radius hlam0 hlam1 hM0
    dsimp [M] at hC ⊢
    have hpi : Real.pi * (p / Real.pi) = p := by
      field_simp [ne_of_gt Real.pi_pos]
    rw [hpi] at hC
    exact hC
  have hC1 : typeCPerimeterAtRadius lam lam < p := by
    rw [typeCPerimeterAtRadius]
    have hb := bAtRadius_admissible hlam0
    have hfactor : Real.pi - (1 - lam) * bAtRadius lam < Real.pi := by
      have hpositive : 0 < (1 - lam) * bAtRadius lam :=
        mul_pos (by linarith) hb.1
      linarith
    nlinarith
  have hM1 : lam < M := by
    dsimp [M]
    apply (lt_div_iff0 Real.pi_pos).2
    nlinarith [Real.pi_pos]
  have hvalue : p ∈ Set.Icc (typeCPerimeterAtRadius lam lam)
    (typeCPerimeterAtRadius lam M) := ⟨hC1.le, hCM.le⟩
  have hcont : ContinuousOn (typeCPerimeterAtRadius lam) (Set.Icc lam M) :=
    typeCPerimeterAtRadius_continuousOn.mono fun x hx => hlam0.trans_le hx.1
  rcases intermediate_value_Icc hM1.le hcont hvalue with ⟨R, hR, hperim⟩
  have hR0 : 0 < R := hlam0.trans_le hR.1
  refine ⟨R, ⟨hR0, hperim⟩, ?_⟩
  intro S hS
  apply (typeCPerimeterAtRadius_strictMonoOn hlam0 hlam1).injOn hS.1 hR0
  rw [hS.2, hperim]

/-- The type-(B) common-perimeter inverse carries both positivity and its
defining equation in its codomain; there is no value outside the open domain. -/

```

CMVBallCase.exists_typeBRadius_at_area

Classification: main

Source: CMVBallCase.lean:993-1019

Declaration SHA-256: 2eb4d55fd6e17591ebc46f3f94a245862a68631d4afc013b3f0eb27bc15ab111

Mathematical statement. If $0 < \lambda < 1$ and $v > \lambda\pi$, some $R > 0$ satisfies $A_B(R) = v$.

The area starts at $\lambda\pi$, is continuous, and exceeds every prescribed value at sufficiently large radius. The intermediate value theorem gives a radius, and the strict area inequality excludes zero.

```

lemma exists_typeBRadius_at_area {lam v : ℝ}
  (hlam0 : 0 < lam) (hlam1 : lam < 1) (hv : lam * Real.pi < v) :
  ∃ R ∈ Set.Ioi 0, typeBAreaAtRadius lam R = v := by
  let M := v + Real.pi + 1
  have hv0 : 0 < v := lt_trans (mul_pos hlam0 Real.pi_pos) hv
  have hM0 : 0 < M := by dsimp [M]; positivity
  have hMlam : lam ≤ M := by dsimp [M]; linarith [Real.pi_pos]
  have hlower := typeBAreaAtRadius_large_lower hlam0 hlam1 hMlam
  have hpi2 : 1 < Real.pi / 2 := by linarith [Real.pi_gt_three]
  have hM1 : 1 < M := by dsimp [M]; linarith [Real.pi_pos]
  have hsq : M < M ^ 2 := by nlinarith
  have hupper : v < typeBAreaAtRadius lam M := by nlinarith
  have hzero : typeBAreaAtRadius lam 0 = lam * Real.pi :=
    typeBAreaAtRadius_zero hlam0 hlam1
  have hvI : v ∈ Set.Icc (typeBAreaAtRadius lam 0)
    (typeBAreaAtRadius lam M) := by
    rw [hzero]
    exact ⟨hv.le, hupper.le⟩

```

```

have him := intermediate_value_Icc hM0.le
  typeBAreaAtRadius_continuous.continuousOn hvI
rcases him with ⟨R, hRI, harea⟩
have hRpos : 0 < R := lt_of_le_of_ne hRI.1 fun hR0 => by
  subst R
  rw [hzero] at harea
  linarith
exact ⟨R, hRpos, harea⟩

```

CMVBallCase.exists_typeCRadius_at_area

Classification: main

Source: CMVBallCase.lean:1020-1067

Declaration SHA-256: 63e6ea9666dcdbe0cc4e7fdc659230e14fd24f25f1cdeb2756c507b3e9eb7e2f

Mathematical statement. If $0 < \lambda < 1$ and $v > \lambda\pi$, some $R > 0$ satisfies $A_C(R) = v$.

Use the lower comparison

$$A_C(\lambda) < \pi\lambda^2 < \lambda\pi < v$$

and the quadratic lower bound at a large positive radius. Continuity between these radii gives the required candidate without evaluating the formula at zero.

```

lemma exists_typeCRadius_at_area {lam v : ℝ}
  (hlam0 : 0 < lam) (hlam1 : lam < 1) (hv : lam * Real.pi < v) :
  ∃ R ∈ Set.Ioi 0, typeCAreaAtRadius lam R = v := by
  let M := (v + Real.pi + 1) / lam
  have hv0 : 0 < v := lt_trans (mul_pos hlam0 Real.pi_pos) hv
  have hnum0 : 0 < v + Real.pi + 1 := by positivity
  have hM0 : 0 < M := div_pos hnum0 hlam0
  have hlamSqOne : lam ^ 2 < 1 := by nlinarith
  have hlamM : lam < M := by
    dsimp [M]
    apply (lt_div_iff0 hlam0).2
    nlinarith [Real.pi_pos]
  have hlower := typeCAreaAtRadius_large_lower hlam0 hlam1 hM0
  have hM1 : 1 < M := by
    dsimp [M]
    apply (lt_div_iff0 hlam0).2
    linarith [Real.pi_pos]
  have hlamMval : lam * M = v + Real.pi + 1 := by
    dsimp [M]
    field_simp [hM0.ne']
  have hpi1 : 1 < Real.pi := by linarith [Real.pi_gt_three]
  have hpiM : 1 < Real.pi * M := by
    nlinarith [mul_pos (sub_pos.mpr hpi1) (sub_pos.mpr hM1)]
  have hprod : v + Real.pi + 1 <
    (v + Real.pi + 1) * (Real.pi * M) := by
    simpa using mul_lt_mul_of_pos_left hpiM hnum0
  have hupper : v < typeCAreaAtRadius lam M := by
    have hrearr : lam * Real.pi * M ^ 2 =
      (lam * M) * (Real.pi * M) := by ring
    rw [hrearr, hlamMval] at hlower
    linarith
  have hlow := typeCAreaAtRadius_lt_full hlam0 hlam1 hlam0
  have hlamSq : Real.pi * lam ^ 2 < lam * Real.pi := by
    have hll : lam ^ 2 < lam := by nlinarith
    nlinarith [mul_lt_mul_of_pos_left hll Real.pi_pos]
  have hareaLam : typeCAreaAtRadius lam lam < v :=
    lt_trans hlow (lt_trans hlamSq hv)
  have hvI : v ∈ Set.Icc (typeCAreaAtRadius lam lam)
    (typeCAreaAtRadius lam M) := ⟨hareaLam.le, hupper.le⟩
  have hcont : ContinuousOn (typeCAreaAtRadius lam) (Set.Icc lam M) :=
    typeCAreaAtRadius_continuousOn.mono fun x hx =>
      lt_of_lt_of_le hlam0 hx.1
  have him := intermediate_value_Icc hlamM.le hcont hvI
  rcases him with ⟨R, hRI, harea⟩

```

```

exact ⟨R, lt_of_lt_of_le hlam0 hRI.1, harea⟩

/-- Every type-(B) perimeter arising from an admissible candidate of exactly
weighted area `v`. -/

```

CMVBallCase.fixedRadiusD

Classification: main

Source: CMVBallCase.lean:1245-1248

Declaration SHA-256: 4e15dd87d6e310dd5beb90b522db2cdea3c545a1ce486ef9aaa593aa1c201858

Mathematical statement. $D(\lambda, R)$ is $(R - \lambda)^2 + (1 - \lambda^2)$.

The defining and expanded expressions are

$$D(\lambda, R) = (R - \lambda)^2 + 1 - \lambda^2 = R^2 - 2R\lambda + 1.$$

The sum-of-squares form shows it is positive for $0 < \lambda < 1$, and the expanded form makes its density derivative simple.

```

def fixedRadiusD (lam R : ℝ) : ℝ :=
  (R - lam) ^ 2 + (1 - lam ^ 2)

/-- The derivative kernel used only on the nontrivial region `R > lam`. -/

```

CMVBallCase.fixedRadiusD_pos

Classification: main

Source: CMVBallCase.lean:1364-1369

Declaration SHA-256: ab40250bb6c14fc69fd59cdf3f0cdb3baf9f85a08aed8a6aea9a5ea02fc5cf9a

Mathematical statement. For $0 < \lambda < 1$ and every real R , $D(\lambda, R) > 0$.

The term $(R - \lambda)^2$ is nonnegative and $1 - \lambda^2$ is positive. Thus the denominators in the density calculation do not vanish anywhere in the admissible density interval.

```

lemma fixedRadiusD_pos {lam R : ℝ}
  (hlam0 : 0 < lam) (hlam1 : lam < 1) :
  0 < fixedRadiusD lam R := by
  unfold fixedRadiusD
  nlinarith [sq_nonneg (R - lam)]

```

CMVBallCase.fixedRadiusHalfGap

Classification: main

Source: CMVBallCase.lean:1240-1244

Declaration SHA-256: 7109dcf6ef8b06b1460afe81b7d4dc3cd06804fef3b67f56231578b2334248e2

Mathematical statement. $H(\lambda, R)$ is $(\lambda - R)\beta_\lambda(R) + \lambda \arccos \lambda + (1 - \lambda)Rb(R)$.

The defining expression is

$$H(\lambda, R) = (\lambda - R)\beta_\lambda(R) + \lambda \arccos \lambda + (1 - \lambda)Rb(R).$$

This total expression is half the difference of the two radius-coordinate perimeters. Its sign is proved by a direct argument for $R \leq \lambda$ and by density differentiation for $R > \lambda$.

```

def fixedRadiusHalfGap (lam R : ℝ) : ℝ :=
  (lam - R) * betaAtRadius lam R + lam * Real.arccos lam +
  (1 - lam) * R * bAtRadius R

```

```
-- The positive denominator in the density-ratio calculation. -/
```

CMVBallCase.fixedRadiusHalfGap_density_ratio_continuousAt

Classification: main

Source: CMVBallCase.lean:1660-1664

Declaration SHA-256: 63a18ea3889cd159ace5ef02e45f92fdd7d2a7fad2bca6f2d3d6f49921156755

Mathematical statement. For any real R and $0 < c < 1$, the function $x \mapsto H(x, R)/x$ is continuous at c .

Its established density derivative implies continuity. In particular, for $0 < R < 1$, this allows comparison with the finite endpoint $c = R$ of the decreasing interval.

```
lemma fixedRadiusHalfGap_density_ratio_continuousAt {c R : ℝ}
  (hc0 : 0 < c) (hc1 : c < 1) :
  ContinuousAt (fun x => fixedRadiusHalfGap x R / x) c :=
  (fixedRadiusHalfGap_density_ratio_hasDerivAt hc0 hc1).continuousAt
```

CMVBallCase.fixedRadiusHalfGap_density_ratio_hasDerivAt

Classification: main

Source: CMVBallCase.lean:1480-1500

Declaration SHA-256: 8201756840d20049cb38f7556cdc3b5c84839cfb60b6a731c3c4da0399a6d421

Mathematical statement. For $0 < \lambda < 1$ and every real R , the derivative of $c \mapsto H(c, R)/c$ at λ is $RL(\lambda, R)/\lambda^2$.

Apply product and quotient rules to the explicit expression for H . The resulting identity is

$$\left. \frac{d}{dc} \frac{H(c, R)}{c} \right|_{c=\lambda} = \frac{R}{\lambda^2} L(\lambda, R).$$

Its strict negative sign additionally requires $R > \lambda$.

```
lemma fixedRadiusHalfGap_density_ratio_hasDerivAt {lam R : ℝ}
  (hlam0 : 0 < lam) (hlam1 : lam < 1) :
  HasDerivAt (fun c => fixedRadiusHalfGap c R / c)
    (R / lam ^ 2 * fixedRadiusL lam R) lam := by
  have hs := sqrt_one_sub_sq_pos hlam0 hlam1
  have hs2 := sq_sqrt_one_sub_sq hlam0 hlam1
  have hd := fixedRadiusD_pos (R := R) hlam0 hlam1
  have hb := betaAtRadius_density_hasDerivAt (R := R) hlam0 hlam1
  have ha := Real.hasDerivAt_arccos (x := lam) (by linarith) (by linarith)
  have h := (((hasDerivAt_id lam).sub_const R).mul hb).add
    ((hasDerivAt_id lam).mul ha)).add
    (((hasDerivAt_const lam 1).sub (hasDerivAt_id lam)).mul_const R).mul_const
    (bAtRadius R))
  have hq := h.div (hasDerivAt_id lam) hlam0.ne'
  convert hq using 1 <=> try rfl
  simp only [Pi.sub_apply, Pi.mul_apply, Pi.add_apply, id_eq]
  unfold fixedRadiusL
  field_simp [hs.ne', hd.ne', hlam0.ne']
  unfold fixedRadiusD
  linear_combination -R * lam * hs2
```

CMVBallCase.fixedRadiusHalfGap_density_ratio_strictAntiOn

Classification: main

Source: CMVBallCase.lean:1640-1654

Declaration SHA-256: 2f0edd3e2968d32be31b213f6dbe635d46d10fd277b8e4e89a8b9f3fd3a0d39e

Mathematical statement. For $R > 0$, $c \mapsto H(c, R)/c$ is strictly decreasing on $(0, \min(1, R))$.

On that interval, $L(c, R) < 0$ and $R/c^2 > 0$, so

$$\frac{d}{dc} \frac{H(c, R)}{c} = \frac{R}{c^2} L(c, R) < 0.$$

The mean value theorem yields strict decrease.

```
lemma fixedRadiusHalfGap_density_ratio_strictAntiOn {R : ℝ} (hR : 0 < R) :
  StrictAntiOn (fun c => fixedRadiusHalfGap c R / c) (Set.Ioo 0 (min 1 R)) := by
  apply strictAntiOn_of_deriv_neg (convex_Ioo _ _)
  · intro c hc
    exact (fixedRadiusHalfGap_density_ratio_hasDerivAt hc.1
      (hc.2.trans_le (min_le_left _ _))).continuousAt.continuousWithinAt
  · intro c hc
    rw [interior_Ioo] at hc
    have hc1 := hc.2.trans_le (min_le_left 1 R)
    have hcR := hc.2.trans_le (min_le_right 1 R)
    rw [(fixedRadiusHalfGap_density_ratio_hasDerivAt hc.1 hc1).deriv]
    exact mul_neg_of_pos_of_neg (div_pos hR (sq_pos_of_pos hc.1))
      (fixedRadiusL_neg hc.1 hc1 hcR)

/-- The finite endpoint `c = R < 1` has a strictly positive gap. -/
```

CMVBallCase.fixedRadiusHalfGap_density_ratio_tendsto_one

Classification: main

Source: CMVBallCase.lean:1726-1733

Declaration SHA-256: 38b39009304f9c27c735560f1daa9451bc247788bdacf9b12a22ee47421536b1

Mathematical statement. For every $R \geq 1$, $H(c, R)/c$ tends to zero as c tends to one from below.

The proof separates $R = 1$, handled by a squeeze estimate, from $R > 1$, handled by the reciprocal chart. In both cases,

$$\lim_{c \uparrow 1} \frac{H(c, R)}{c} = 0.$$

This is a pointwise limit for each radius, with no uniform-radius assertion.

```
lemma fixedRadiusHalfGap_density_ratio_tendsto_one {R : ℝ} (hR : 1 ≤ R) :
  Filter.Tendsto (fun c => fixedRadiusHalfGap c R / c)
    (nhdsWithin 1 (Set.Iio 1)) (nhds 0) := by
  rcases eq_or_lt_of_le hR with h | h
  · subst R
    exact fixedRadiusHalfGap_density_ratio_tendsto_one_at_one
  · exact fixedRadiusHalfGap_density_ratio_tendsto_one_of_gt h
```

CMVBallCase.fixedRadiusHalfGap_density_ratio_tendsto_one_at_one

Classification: main

Source: CMVBallCase.lean:1695-1725

Declaration SHA-256: 72d28fc306e34fd07f9a091bc1f04b99fa14efc650ed4c7075d01cf65333b270

Mathematical statement. $H(c, 1)/c$ tends to zero as c tends to one from below.

At this corner, use the bound

$$0 \leq (1 - c)\beta_c(1) \leq (1 - c)\frac{\pi}{2} \quad (0 < c < 1).$$

This product tends to zero without assigning an endpoint angle to the singular chart; the arccosine and remaining weighted term also vanish. Consequently

$$\lim_{c \uparrow 1} \frac{H(c, 1)}{c} = 0.$$

```
lemma fixedRadiusHalfGap_density_ratio_tendsto_one_at_one :
  Filter.Tendsto (fun c => fixedRadiusHalfGap c 1 / c)
    (nhdsWithin 1 (Set.Iio 1)) (nhds 0) := by
  have hi : Filter.Tendsto (fun c : ℝ => c)
    (nhdsWithin 1 (Set.Iio 1)) (nhds 1) := nhdsWithin_le_nhds
  have hp : Filter.Tendsto (fun c => (1 - c) * betaAtRadius c 1)
    (nhdsWithin 1 (Set.Iio 1)) (nhds 0) := by
    apply squeeze_zero' (g := fun c : ℝ => (1 - c) * (Real.pi / 2))
    · filter_upwards [self_mem_nhdsWithin, mem_nhdsWithin_of_mem_nhds
      (Ioi_mem_nhds (show (0 : ℝ) < 1 by norm_num))] with c hc hc0
      exact mul_nonneg (sub_nonneg.mpr hc.le) (betaAtRadius_pos hc0 hc).le
    · filter_upwards [self_mem_nhdsWithin, mem_nhdsWithin_of_mem_nhds
      (Ioi_mem_nhds (show (0 : ℝ) < 1 by norm_num))] with c hc hc0
      exact mul_le_mul_of_nonneg_left (betaAtRadius_le_pi_div_two hc0 hc hc.le)
        (sub_nonneg.mpr hc.le)
    · convert (tendsto_const_nhds.sub hi).mul_const (Real.pi / 2) using 1 <=>
        first | rfl | norm_num
  have ha : Filter.Tendsto (fun c : ℝ => Real.arccos c)
    (nhdsWithin 1 (Set.Iio 1)) (nhds 0) := by
    convert Real.continuous_arccos.continuousAt.tendsto.comp hi using 1 <=>
      first | rfl | simp
  have ht := ((hp.neg.add (hi.mul ha)).add
    (((tendsto_const_nhds (x := (1 : ℝ))).sub hi).mul_const (bAtRadius 1))).div hi
    (by norm_num)
  convert ht using 1
  · funext c
  · unfold fixedRadiusHalfGap
  · simp only [Pi.div_apply]
  · ring
  · norm_num
```

CMVBallCase.fixedRadiusHalfGap_density_ratio_tendsto_one_of_gt

Classification: main

Source: CMVBallCase.lean:1665-1694

Declaration SHA-256: ec5c31b64505c09d08c58f3f1b7efe55ef00bdd5aad235b9e99482e73cfe7914

Mathematical statement. For $R > 1$, $H(c, R)/c$ tends to zero as c tends to one from below.

Use the reciprocal chart: $\beta_c(R) \rightarrow 0$, $\arccosc \rightarrow 0$, and $1 - c \rightarrow 0$, while $c \rightarrow 1$. Every term in the quotient tends to zero, giving

$$\lim_{c \uparrow 1} \frac{H(c, R)}{c} = 0.$$

```
lemma fixedRadiusHalfGap_density_ratio_tendsto_one_of_gt {R : ℝ} (hR : 1 < R) :
  Filter.Tendsto (fun c => fixedRadiusHalfGap c R / c)
    (nhdsWithin 1 (Set.Iio 1)) (nhds 0) := by
  have hb : ContinuousAt
    (fun c : ℝ => Real.arctan (Real.sqrt (1 - c ^ 2) / (R - c))) 1 := by
    apply Real.continuous_arctan.continuousAt.comp
    apply ContinuousAt.div
    · fun_prop
    · fun_prop
    · exact sub_ne_zero.mpr (ne_of_gt hR)
```

```

have ht : ContinuousAt (fun c : ℝ =>
  ((c - R) * Real.arctan (Real.sqrt (1 - c ^ 2) / (R - c)) +
   c * Real.arccos c + (1 - c) * R * bAtRadius R) / c) 1 := by
  apply ContinuousAt.div
  · exact (((continuousAt_id.sub continuousAt_const).mul hb).add
    (continuousAt_id.mul Real.continuous_arccos.continuousAt)).add
    (((continuousAt_const.sub continuousAt_id).mul continuousAt_const).mul continuousAt_const)
  · exact continuousAt_id
  · norm_num
have he : (fun c => fixedRadiusHalfGap c R / c) =f[nhdsWithin 1 (Set.Iio 1)]
  (fun c => ((c - R) * Real.arctan (Real.sqrt (1 - c ^ 2) / (R - c)) +
   c * Real.arccos c + (1 - c) * R * bAtRadius R) / c) := by
  filter_upwards [self_mem_nhdsWithin, mem_nhdsWithin_of_mem_nhds
    (Ioi_mem_nhds (show (0 : ℝ) < 1 by norm_num))] with c hc hc0
  rw [fixedRadiusHalfGap, betaAtRadius_eq_arctan_reciprocal hc0 hc (hc.trans hR)]
have ht' := ht.tendsto.mono_left (nhdsWithin_le_nhds (s := Set.Iio 1))
simp using ht'.congr' he.symm

/-- At the singular corner `R = 1`, the bounded angle is multiplied by
`1 - c`. Squeezing this product avoids assigning a spurious endpoint angle. -/

```

CMVBallCase.fixedRadiusHalfGap_pos_of_gt

Classification: main

Source: CMVBallCase.lean:1773-1782

Declaration SHA-256: fe2e7d9f4b2c9b099c9f8b560428cb8beb3d38b8a8d99f10c67f26ccbe96aa81

Mathematical statement. If $0 < \lambda < 1$ and $R > \lambda$, then $H(\lambda, R) > 0$.

Split into $R < 1$ and $R \geq 1$. The finite diagonal comparison and the unit-density limit, respectively, cover all radii in this region, including radii arbitrarily close to λ .

```

lemma fixedRadiusHalfGap_pos_of_gt {lam R : ℝ}
  (hlam0 : 0 < lam) (hlam1 : lam < 1) (hR : lam < R) :
  0 < fixedRadiusHalfGap lam R := by
  rcases lt_or_ge R 1 with h | h
  · exact fixedRadiusHalfGap_pos_of_lt_radius_lt_one hlam0 hR h
  · exact fixedRadiusHalfGap_pos_of_one_le_radius hlam0 hlam1 h

/-- Global comparison of the faithful radius-coordinate candidates.
This is an analytic perimeter theorem; no finite-perimeter classification
or equal-area profile-transfer premise is used. -/

```

CMVBallCase.fixedRadiusHalfGap_pos_of_le

Classification: main

Source: CMVBallCase.lean:1386-1399

Declaration SHA-256: 70db5ffdb68d856e4b264b6a6571be92f1d5430fbe1f89edab30d115fd2ca0a8

Mathematical statement. If $0 < \lambda < 1$ and $0 < R \leq \lambda$, then $H(\lambda, R) > 0$.

The factor $\lambda - R$ and angle $\beta_\lambda(R)$ make the first term nonnegative. Both $\lambda \arccos \lambda$ and $(1 - \lambda)Rb(R)$ are positive, proving a strictly positive sum.

```

lemma fixedRadiusHalfGap_pos_of_le {lam R : ℝ}
  (hlam0 : 0 < lam) (hlam1 : lam < 1) (hR0 : 0 < R) (hR : R ≤ lam) :
  0 < fixedRadiusHalfGap lam R := by
  have hb := betaAtRadius_pos (R := R) hlam0 hlam1
  have hc := (bAtRadius_admissible hR0).1
  have ha : 0 < Real.arccos lam := Real.arccos_pos.2 hlam1
  have h1 : 0 ≤ (lam - R) * betaAtRadius lam R :=
    mul_nonneg (sub_nonneg.mpr hR) hb.le
  have h2 : 0 < lam * Real.arccos lam := mul_pos hlam0 ha
  have h3 : 0 < (1 - lam) * R * bAtRadius R :=

```

```
mul_pos (mul_pos (sub_pos.mpr hlam1) hR0) hc
unfold fixedRadiusHalfGap
linarith
```

CMVBallCase.fixedRadiusHalfGap_pos_of_lt_radius_lt_one

Classification: main

Source: CMVBallCase.lean:1734-1753

Declaration SHA-256: fbe9df98337f6aceb7cf9c5bfef2a39837777085850f68cc2b9e8dfc133753b5

Mathematical statement. If $0 < \lambda < R < 1$, then $H(\lambda, R) > 0$.

The ratio extends continuously to $c = R$ and is strictly decreasing on $(0, R]$. Hence

$$\frac{H(\lambda, R)}{\lambda} > \frac{H(R, R)}{R} > 0,$$

where the endpoint positivity comes from the direct $R \leq c$ case.

```
lemma fixedRadiusHalfGap_pos_of_lt_radius_lt_one {lam R : ℝ}
  (hlam0 : 0 < lam) (hR : lam < R) (hR1 : R < 1) :
  0 < fixedRadiusHalfGap lam R := by
  have hR0 := hlam0.trans hR
  have hm : StrictAntiOn (fun c => fixedRadiusHalfGap c R / c) (Set.Ioc 0 R) := by
    apply strictAntiOn_of_deriv_neg (convex_Ioc _)
  · intro c hc
    exact (fixedRadiusHalfGap_density_ratio_continuousAt hc.1
      (hc.2.trans_lt hR1)).continuousWithinAt
  · intro c hc
    rw [interior_Ioc] at hc
    have hc1 := hc.2.trans hR1
    rw [(fixedRadiusHalfGap_density_ratio_hasDerivAt hc.1 hc1).deriv]
    exact mul_neg_of_pos_of_neg (div_pos hR0 (sq_pos_of_pos hc.1))
      (fixedRadiusL_neg hc.1 hc1 hc.2)
  have hpos : 0 < fixedRadiusHalfGap R R / R :=
    div_pos (fixedRadiusHalfGap_pos_of_le hR0 hR1 hR0 le_rfl) hR0
  have hlt := hm ⟨hlam0, hR.le⟩ ⟨hR0, le_rfl⟩ hR
  exact (div_pos_iff_of_pos_right hlam0).mp (hpos.trans hlt)
```

CMVBallCase.fixedRadiusHalfGap_pos_of_one_le_radius

Classification: main

Source: CMVBallCase.lean:1754-1772

Declaration SHA-256: e873740cbb342f72d438af00f9b3a7382b645a81e5e013505d7f0ed284853138

Mathematical statement. If $0 < \lambda < 1$ and $R \geq 1$, then $H(\lambda, R) > 0$.

The ratio strictly decreases on $(0, 1)$ and tends to zero at one. Comparing λ with any larger interior density, whose ratio is nonnegative by the endpoint limit, yields strict positivity at λ .

```
lemma fixedRadiusHalfGap_pos_of_one_le_radius {lam R : ℝ}
  (hlam0 : 0 < lam) (hlam1 : lam < 1) (hR : 1 ≤ R) :
  0 < fixedRadiusHalfGap lam R := by
  have hR0 : 0 < R := lt_of_lt_of_le (by norm_num) hR
  have hm : StrictAntiOn (fun c => fixedRadiusHalfGap c R / c) (Set.Ioo 0 1) := by
    simpa only [min_eq_left hR] using fixedRadiusHalfGap_density_ratio_strictAntiOn hR0
  obtain ⟨c, hlc, hc1⟩ := exists_between hlam1
  have hc0 := hlam0.trans hlc
  have hnonneg : 0 ≤ fixedRadiusHalfGap c R / c := by
    apply le_of_tendsto (fixedRadiusHalfGap_density_ratio_tendsto_one hR)
    filter_upwards [self_mem_nhdsWithin, mem_nhdsWithin_of_mem_nhds
      (Ioi_mem_nhds hc1)] with x hx hcx
    exact (hm ⟨hc0, hc1⟩ ⟨hc0.trans hcx, hx⟩ hcx).le
  have hlt := hm ⟨hlam0, hlam1⟩ ⟨hc0, hc1⟩ hlc
```

```
exact (div_pos_iff_of_pos_right hlam0).mp (hnonneg.trans_lt hlt)
```

```
-- The hard fixed-radius inequality, proved through the density ratio.
The endpoint split covers radii arbitrarily close to `lam`, `R = 1`,
and all `R > 1`, without a uniform margin assumption. -/
```

CMVBallCase.fixedRadiusL

Classification: main

Source: CMVBallCase.lean:1249-1253

Declaration SHA-256: f9ee2e168349ec1a77b121c311033b4f50ae517f2839c175c84616e6784fbea2

Mathematical statement. $L(\lambda, R)$ is $\beta_\lambda(R) - b(R) - \lambda\sqrt{1 - \lambda^2} / D(\lambda, R)$.

The definition is

$$L(\lambda, R) = \beta_\lambda(R) - b(R) - \frac{\lambda\sqrt{1 - \lambda^2}}{D(\lambda, R)}.$$

This is the derivative factor in $\partial_\lambda(H/\lambda) = RL/\lambda^2$. The sign needed in the proof is established only for $0 < \lambda < 1$ and $R > \lambda$, not for all positive radii.

```
def fixedRadiusL (lam R : ℝ) : ℝ :=
  betaAtRadius lam R - bAtRadius R -
    lam * Real.sqrt (1 - lam ^ 2) / fixedRadiusD lam R
-- An exact counterexample to the false unrestricted sign assertion for
`fixedRadiusL`: its derivative kernel is positive in the `R < λ` branch. -/
```

CMVBallCase.fixedRadiusL_continuousAt_zero

Classification: main

Source: CMVBallCase.lean:1515-1535

Declaration SHA-256: d2266e6ab3843cad1067d095c347fbf2258071549d62313f18f286686d040702

Mathematical statement. For $R > 0$, the function $c \mapsto L(c, R)$ is continuous at $c = 0$.

At zero density the square-root denominator is one and $D(0, R) = R^2 + 1 > 0$. Ordinary continuity rules therefore apply, justifying comparison with $L(0, R) = 0$.

```
lemma fixedRadiusL_continuousAt_zero {R : ℝ} (hR : 0 < R) :
  ContinuousAt (fun c => fixedRadiusL c R) 0 := by
  have hs : Real.sqrt (1 - (0 : ℝ) ^ 2) ≠ 0 := by norm_num
  have hd : fixedRadiusD 0 R ≠ 0 := by
    unfold fixedRadiusD
    positivity
  unfold fixedRadiusL betaAtRadius
  apply ContinuousAt.sub
  · exact ((continuousAt_const.sub
    (Real.continuous_arctan.continuousAt.comp
      ((continuousAt_const.sub continuousAt_id).div
        (Real.continuous_sqrt.continuousAt.comp
          (continuousAt_const.sub (continuousAt_id.pow 2)))) hs))))).sub
    continuousAt_const
  · apply ContinuousAt.div
    · fun_prop
    · unfold fixedRadiusD
      fun_prop
    · exact hd

-- For `R > 1` the reciprocal chart supplies the exact left endpoint at density one. -/
```

CMVBallCase.fixedRadiusL_density_hasDerivAt

Classification: main

Source: CMVBallCase.lean:1462-1479

Declaration SHA-256: db1661fbfdaeefec8e9d4f2e678214b5a54356b68cb7f3953f8533e10e578e7d

Mathematical statement. For $0 < \lambda < 1$ and every real R , the density derivative of L is $-\lambda N(\lambda, R)/(\sqrt{1 - \lambda^2} D(\lambda, R)^2)$.

Differentiate $\beta_\lambda(R) - b(R) - \lambda s/D$, using the chart, square-root, and denominator derivatives. Exact algebra gives

$$\left. \frac{d}{dc} L(c, R) \right|_{c=\lambda} = -\frac{\lambda N(\lambda, R)}{\sqrt{1 - \lambda^2} D(\lambda, R)^2}.$$

The sign depends only on $-N$ when $\lambda > 0$.

```
lemma fixedRadiusL_density_hasDerivAt {lam R : ℝ}
  (hlam0 : 0 < lam) (hlam1 : lam < 1) :
  HasDerivAt (fun c => fixedRadiusL c R)
    (-lam * fixedRadiusN lam R /
      (Real.sqrt (1 - lam ^ 2) * fixedRadiusD lam R ^ 2)) lam := by
  have hs := sqrt_one_sub_sq_pos hlam0 hlam1
  have hs2 := sq_sqrt_one_sub_sq hlam0 hlam1
  have hd := fixedRadiusD_pos (R := R) hlam0 hlam1
  have h := ((betaAtRadius_density_hasDerivAt (R := R) hlam0 hlam1).sub_const
    (bAtRadius R)).sub
    (((hasDerivAt_id lam).mul (sqrt_one_sub_sq_hasDerivAt hlam0 hlam1)).div
      (fixedRadiusD_density_hasDerivAt lam R) hd.ne')
  convert h using 1 <=> try rfl
  simp only [Pi.mul_apply, id_eq]
  field_simp [hs.ne', hd.ne']
  unfold fixedRadiusD fixedRadiusN
  nlinarith [hs2]
```

CMVBallCase.fixedRadiusL_neg

Classification: main

Source: CMVBallCase.lean:1630-1639

Declaration SHA-256: 7412414d7e4f71205db23e4c7a9ba2c16e8de6f3ee9f630908e2c9dd59858948

Mathematical statement. If $0 < \lambda < 1$ and $\lambda < R$, then $L(\lambda, R) < 0$.

For $R \leq 1$, positivity of N applies. For $R > 1$, split according to $N \geq 0$ or $N < 0$, using the zero-density or unit-density endpoint respectively. No sign is claimed outside $R > \lambda$.

```
lemma fixedRadiusL_neg {lam R : ℝ}
  (hlam0 : 0 < lam) (hlam1 : lam < 1) (hR : lam < R) :
  fixedRadiusL lam R < 0 := by
  by_cases hR1 : R ≤ 1
  · exact fixedRadiusL_neg_of_N_nonneg hlam0 hlam1 hR
    (fixedRadiusN_pos_of_radius_le_one hlam0 hR hR1).le
  · rcases le_or_gt 0 (fixedRadiusN lam R) with hN | hN
    · exact fixedRadiusL_neg_of_N_nonneg hlam0 hlam1 hR hN
    · exact fixedRadiusL_neg_of_N_neg hlam0 hlam1 (lt_of_not_ge hR1) hN
```

CMVBallCase.fixedRadiusL_neg_of_N_neg

Classification: main

Source: CMVBallCase.lean:1604-1629

Declaration SHA-256: 7e62014feed7939abad45447e00b04f292d440a21f48f2f7d10485082ddd4696

Mathematical statement. If $0 < \lambda < 1 < R$ and $N(\lambda, R) < 0$, then $L(\lambda, R) < 0$.

For $\lambda < c < 1$, the polynomial remains negative, so L increases on $[\lambda, 1)$. Its left limit is $-b(R) < 0$; therefore

$$L(\lambda, R) \leq -b(R) < 0.$$

```
lemma fixedRadiusL_neg_of_N_neg {lam R : ℝ}
  (hlam0 : 0 < lam) (hlam1 : lam < 1) (hR : 1 < R)
  (hN : fixedRadiusN lam R < 0) :
  fixedRadiusL lam R < 0 := by
  have hm : StrictMonoOn (fun c => fixedRadiusL c R) (Set.Ico lam 1) := by
    apply strictMonoOn_of_deriv_pos (convex_Ico _ _)
    · intro c hc
      exact (fixedRadiusL_density_hasDerivAt (hlam0.trans_le hc.1) hc.2).continuousAt.continuousWithinAt
    · intro c hc
      rw [interior_Ico] at hc
      have hc0 := hlam0.trans hc.1
      have hNc : fixedRadiusN c R < 0 :=
        (fixedRadiusN_strictAntiOn (by linarith) hlam1.le hc.2.le hc.1).trans hN
      rw [(fixedRadiusL_density_hasDerivAt hc0 hc.2).deriv]
      exact div_pos (mul_pos_of_neg_of_neg (neg_neg_of_pos hc0) hNc)
        (mul_pos (sqrt_one_sub_sq_pos hc0 hc.2)
          (sq_pos_of_pos (fixedRadiusD_pos hc0 hc.2)))
  have hle : fixedRadiusL lam R ≤ -bAtRadius R := by
    apply ge_of_tendsto (fixedRadiusL_tendsto_one hR)
    filter_upwards [self_mem_nhdsWithin, mem_nhdsWithin_of_mem_nhds
      (Ioi_mem_nhds hlam1)] with c hc hcl
    exact (hm ⟨le_rfl, hlam1⟩ ⟨hc.le, hc⟩ hcl).le
    exact hle.trans_lt (neg_neg_of_pos (bAtRadius_admissible (by linarith)).1)

/-- The sign is deliberately restricted to `R > lam`; the retained exact
counterexample rules out removing this hypothesis. -/
```

CMVBallCase.fixedRadiusL_neg_of_N_nonneg

Classification: main

Source: CMVBallCase.lean:1571-1594

Declaration SHA-256: e7586227d8c6bfcd2ebb1b0a33219042c5b328676e83c2a41deef131c44f22f

Mathematical statement. If $0 < \lambda < 1$, $\lambda < R$, and $N(\lambda, R) \geq 0$, then $L(\lambda, R) < 0$.

Strict decrease of $N(\cdot, R)$ gives $N(c, R) > 0$ for $0 < c < \lambda$, even when $N(\lambda, R) = 0$. Hence L strictly decreases from its continuous zero value on $[0, \lambda]$.

```
lemma fixedRadiusL_neg_of_N_nonneg {lam R : ℝ}
  (hlam0 : 0 < lam) (hlam1 : lam < 1) (hR : lam < R)
  (hN : 0 ≤ fixedRadiusN lam R) :
  fixedRadiusL lam R < 0 := by
  have hR0 := hlam0.trans hR
  have hm : StrictAntiOn (fun c => fixedRadiusL c R) (Set.Icc 0 lam) := by
    apply strictAntiOn_of_deriv_neg (convex_Icc _ _)
    · intro c hc
      rcases eq_or_lt_of_le hc.1 with h | h
      · subst c
        exact (fixedRadiusL_continuousAt_zero hR0).continuousWithinAt
      · exact (fixedRadiusL_density_hasDerivAt h (hc.2.trans_lt hlam1)).continuousAt.continuousWithinAt
    · intro c hc
      rw [interior_Icc] at hc
      have hN' : 0 < fixedRadiusN c R :=
        hN.trans_lt (fixedRadiusN_strictAntiOn hR0
          (hc.2.le.trans hlam1.le) hlam1.le hc.2)
      rw [(fixedRadiusL_density_hasDerivAt hc.1 (hc.2.trans hlam1)).deriv]
      exact div_neg_of_neg_of_pos (mul_neg_of_neg_of_pos (neg_neg_of_pos hc.1) hN')
        (mul_pos (sqrt_one_sub_sq_pos hc.1 (hc.2.trans hlam1))
          (sq_pos_of_pos (fixedRadiusD_pos hc.1 (hc.2.trans hlam1))))
  have h := hm ⟨le_rfl, hlam0.le⟩ ⟨hlam0.le, le_rfl⟩ hlam0
  simpa only [fixedRadiusL_zero hR0] using h
```

CMVBallCase.fixedRadiusL_tendsto_one

Classification: main

Source: CMVBallCase.lean:1536-1570

Declaration SHA-256: 13ea5fd875867d8d70d1cab8b12f6ea2839f979405c944fc74ef550ef86c0035

Mathematical statement. For $R > 1$, $L(c, R)$ tends to $-b(R)$ as c tends to one from below.

The reciprocal chart gives $\beta_c(R) \rightarrow 0$, while $c\sqrt{1-c^2}/D(c, R) \rightarrow 0$ because $D(1, R) = (R-1)^2 > 0$. Therefore

$$\lim_{c \uparrow 1} L(c, R) = -b(R).$$

The claim deliberately excludes $R = 1$.

```
lemma fixedRadiusL_tendsto_one {R : ℝ} (hR : 1 < R) :
  Filter.Tendsto (fun c => fixedRadiusL c R) (nhdsWithin 1 (Set.Iio 1))
    (nhds (-bAtRadius R)) := by
  have hb : ContinuousAt
    (fun c : ℝ => Real.arctan (Real.sqrt (1 - c ^ 2) / (R - c))) 1 := by
    apply Real.continuous_arctan.continuousAt.comp
    apply ContinuousAt.div
    · fun_prop
    · fun_prop
    · exact sub_ne_zero.mpr (ne_of_gt hR)
  have hd : fixedRadiusD 1 R ≠ 0 := by
    simp only [fixedRadiusD, one_pow, sub_self, add_zero]
    exact pow_ne_zero 2 (sub_ne_zero.mpr (ne_of_gt hR))
  have hc : ContinuousAt (fun c => c * Real.sqrt (1 - c ^ 2) / fixedRadiusD c R) 1 := by
    apply ContinuousAt.div
    · fun_prop
    · unfold fixedRadiusD
      fun_prop
    · exact hd
  have ht : Filter.Tendsto
    (fun c => Real.arctan (Real.sqrt (1 - c ^ 2) / (R - c)) -
      bAtRadius R - c * Real.sqrt (1 - c ^ 2) / fixedRadiusD c R)
    (nhdsWithin 1 (Set.Iio 1)) (nhds (-bAtRadius R)) := by
    convert ((hb.sub (continuousAt_const (y := bAtRadius R))).sub hc).tendsto.mono_left
    (nhdsWithin_le_nhds (s := Set.Iio 1)) using 1 <=> first | rfl | simp
  have he : (fun c => fixedRadiusL c R) =[nhdsWithin 1 (Set.Iio 1)]
    (fun c => Real.arctan (Real.sqrt (1 - c ^ 2) / (R - c)) -
      bAtRadius R - c * Real.sqrt (1 - c ^ 2) / fixedRadiusD c R) := by
    filter_upwards [self_mem_nhdsWithin, mem_nhdsWithin_of_mem_nhds
      (Ioi_mem_nhds (show (0 : ℝ) < 1 by norm_num))] with c hc hc0
    rw [fixedRadiusL, betaAtRadius_eq_arctan_reciprocal hc0 hc (hc.trans hR)]
    simpa using ht.congr' he.symm

/-- This includes the stationary locus `N = 0`: `N` is strictly positive
strictly to its left, so `L` has already decreased from its zero endpoint. -/
```

CMVBallCase.fixedRadiusL_zero

Classification: main

Source: CMVBallCase.lean:1501-1507

Declaration SHA-256: 29c562fbf4f67e14b5c469f6f6f6dcfe57423601de3464b20c18a1c168e35d8e

Mathematical statement. For every $R > 0$, $L(0, R) = 0$.

The correction term vanishes and $\pi/2 - \arctan R = \arctan(1/R) = b(R)$. Hence

$$L(0, R) = 0.$$

This zero-density endpoint anchors the decreasing part of the sign argument.

```

lemma fixedRadiusL_zero {R : ℝ} (hR : 0 < R) :
  fixedRadiusL 0 R = 0 := by
  simp only [fixedRadiusL, betaAtRadius, zero_pow (by decide : 2 ≠ 0),
    sub_zero, Real.sqrt_one, div_one, zero_mul, zero_div]
  rw [bAtRadius, one_div, Real.arctan_inv_of_pos hR]
  ring

```

CMVBallCase.fixedRadiusN

Classification: main

Source: CMVBallCase.lean:1361-1363

Declaration SHA-256: c72277ca17b4cfcd6ad9cc4b941bef23e2a9af37be01cd916422a0b4b63e81c1

Mathematical statement. $N(\lambda, R)$ is $R^3 - 4R^2\lambda + 2R\lambda^2 + 3R - 2\lambda$.

The polynomial is defined by

$$N(\lambda, R) = R^3 - 4R^2\lambda + 2R\lambda^2 + 3R - 2\lambda.$$

It controls the density derivative of L through $\partial_\lambda L = -\lambda N/(\sqrt{1-\lambda^2} D^2)$. Its strict decrease in density makes the sign analysis global rather than numerical.

```

def fixedRadiusN (lam R : ℝ) : ℝ :=
  R ^ 3 - 4 * R ^ 2 * lam + 2 * R * lam ^ 2 + 3 * R - 2 * lam

```

CMVBallCase.fixedRadiusN_density_derivative_identity

Classification: main

Source: CMVBallCase.lean:1370-1379

Declaration SHA-256: 80b7c6ee3bbac239cfec59bac30eb2868a417e75a97013642551d21b2c7f23ee

Mathematical statement. For all real λ and R , the density derivative of N at λ is $-(4(R - 1/2)^2 + 1 + 4R(1 - \lambda))$.

Differentiation and completing the square give

$$\left. \frac{d}{dc} N(c, R) \right|_{c=\lambda} = -4R^2 + 4R\lambda - 2 = -\left(4\left(R - \frac{1}{2}\right)^2 + 1 + 4R(1 - \lambda)\right).$$

The sign is transparent for $R > 0, \lambda < 1$.

```

lemma fixedRadiusN_density_derivative_identity (lam R : ℝ) :
  HasDerivAt (fun c => fixedRadiusN c R)
    (-(4 * (R - 1 / 2) ^ 2 + 1 + 4 * R * (1 - lam))) lam := by
  unfold fixedRadiusN
  convert (((hasDerivAt_const lam (R ^ 3)).sub
    ((hasDerivAt_id lam).const_mul (4 * R ^ 2))).add
    (((hasDerivAt_id lam).pow 2).const_mul (2 * R))).add_const
    (3 * R)).sub ((hasDerivAt_id lam).const_mul 2) using 1 <=>
  first | rfl | (simp only [id_eq] <=> ring)

```

CMVBallCase.fixedRadiusN_density_derivative_neg

Classification: main

Source: CMVBallCase.lean:1407-1412

Declaration SHA-256: 0a45acd25d263bfc0ab66ae247903b4ac172086632c9ab50764b1204d06b8751

Mathematical statement. If $\lambda < 1$ and $R > 0$, then $-(4(R - 1/2)^2 + 1 + 4R(1 - \lambda)) < 0$.

The square term is nonnegative, the constant is positive, and $4R(1 - \lambda) > 0$. Thus

$$-\left(4\left(R - \frac{1}{2}\right)^2 + 1 + 4R(1 - \lambda)\right) < 0.$$

No lower bound on λ is needed for this derivative sign.

```
lemma fixedRadiusN_density_derivative_neg {lam R : ℝ}
  (hlam1 : lam < 1) (hR : 0 < R) :
  -(4 * (R - 1 / 2) ^ 2 + 1 + 4 * R * (1 - lam)) < 0 := by
  have : 0 < R * (1 - lam) := mul_pos hR (sub_pos.mpr hlam1)
  nlinarith [sq_nonneg (R - 1 / 2)]
```

CMVBallCase.fixedRadiusN_diagonal

Classification: main

Source: CMVBallCase.lean:1413-1417

Declaration SHA-256: 1e28106ded58409dc99d7ec87ff622e35ba249710e63bf8dc7c272196b448794

Mathematical statement. For every real R , $N(R, R) = R(1 - R^2)$.

Substitute the radius as the density variable in the polynomial and collect terms to obtain

$$N(R, R) = R(1 - R^2).$$

For $0 < R \leq 1$, this supplies a nonnegative comparison value at the upper end of the density interval $c < R$.

```
lemma fixedRadiusN_diagonal (R : ℝ) :
  fixedRadiusN R R = R * (1 - R ^ 2) := by
  unfold fixedRadiusN
  ring
```

CMVBallCase.fixedRadiusN_pos_of_radius_le_one

Classification: main

Source: CMVBallCase.lean:1595-1603

Declaration SHA-256: 02c15b97515cc50c1a2b220426d81b6aedc3b955c8b35761b70257e60ada615f

Mathematical statement. If $0 < \lambda < R \leq 1$, then $N(\lambda, R) > 0$.

Strict density decrease and the diagonal identity give

$$N(\lambda, R) > N(R, R) = R(1 - R^2) \geq 0,$$

including $R = 1$.

```
lemma fixedRadiusN_pos_of_radius_le_one {lam R : ℝ}
  (hlam0 : 0 < lam) (hR : lam < R) (hR1 : R ≤ 1) :
  0 < fixedRadiusN lam R := by
  have hR0 := hlam0.trans hR
  have hd : 0 ≤ fixedRadiusN R R := by
    rw [fixedRadiusN_diagonal]
    exact mul_nonneg hR0.le (by nlinarith)
  exact hd.trans_lt (fixedRadiusN_strictAntiOn hR0 (hR.le.trans hR1) hR1 hR)
```

CMVBallCase.fixedRadiusN_strictAntiOn

Classification: main

Source: CMVBallCase.lean:1418-1427

Declaration SHA-256: 25b0b9b9ea43f5e32fc07e9659bfab1ecf97131d024c9759e860808c59663501

Mathematical statement. For each $R > 0$, the function $c \mapsto N(c, R)$ is strictly decreasing on $(-\infty, 1]$.

The polynomial is continuous on the closed half-line and has negative derivative at every interior point $c < 1$. The mean value theorem therefore proves strict decrease up to and including $c = 1$.

```
lemma fixedRadiusN_strictAntiOn {R : ℝ} (hR : 0 < R) :
  StrictAntiOn (fun c => fixedRadiusN c R) (Set.Iic 1) := by
  apply strictAntiOn_of_deriv_neg (convex_Iic _)
  · unfold fixedRadiusN
  · fun_prop
  · intro c hc
    rw [interior_Iic] at hc
    rw [(fixedRadiusN_density_derivative_identity c R).deriv]
    exact fixedRadiusN_density_derivative_neg hc hR
```

CMVBallCase.perimeter_gap_eq_two_mul_halfGap

Classification: main

Source: CMVBallCase.lean:1380-1385

Declaration SHA-256: 8b62e7839212847fab2541e10be82a3d8159e4fde430a8284b2a3ed1383c3a75

Mathematical statement. For all real λ and R , $P_B(R) - P_C(R) = 2H(\lambda, R)$.

Expand both perimeter definitions and cancel the common term $2\pi R$. The remaining terms give the exact identity

$$P_B(R) - P_C(R) = 2H(\lambda, R).$$

No inequalities or domain assumptions enter this identity.

```
lemma perimeter_gap_eq_two_mul_halfGap (lam R : ℝ) :
  typeBPerimeterAtRadius lam R - typeCPerimeterAtRadius lam R =
    2 * fixedRadiusHalfGap lam R := by
  unfold typeBPerimeterAtRadius typeCPerimeterAtRadius fixedRadiusHalfGap
  ring
```

CMVBallCase.sameRadius_typeB_gt_typeC

Classification: main

Source: CMVBallCase.lean:1783-1799

Declaration SHA-256: 9c6c3244ad6f03f01ef9a5860148ac78a1967d0c98c4165b638dc559db07fdf5

Mathematical statement. For every $0 < \lambda < 1$ and $R > 0$, $P_C(R) < P_B(R)$.

The gap identity reduces the conclusion to positivity of H :

$$P_B(R) - P_C(R) = 2H(\lambda, R) > 0.$$

The cases $R \leq \lambda$ and $R > \lambda$ prove it over the complete positive radius domain.

```
theorem sameRadius_typeB_gt_typeC {lam R : ℝ}
  (hlam0 : 0 < lam) (hlam1 : lam < 1) (hR : 0 < R) :
  typeCPerimeterAtRadius lam R < typeBPerimeterAtRadius lam R := by
  rcases le_or_gt R lam with h | h
  · exact sameRadius_typeB_gt_typeC_of_le hlam0 hlam1 hR h
  · rw [← sub_pos, perimeter_gap_eq_two_mul_halfGap]
    exact mul_pos (by norm_num) (fixedRadiusHalfGap_pos_of_gt hlam0 hlam1 h)

/-- The exact common-perimeter differential mechanism. If `radiusB p` and
`radiusC p` are the radii realizing the same perimeter `p`, the first
variation identities make the derivative of the area gap equal to their
```

difference. A strict radius order therefore makes that gap strictly decreasing, by the kernel-checked mean-value theorem. -/

CMVBallCase.sameRadius_typeB_gt_typeC_of_le

Classification: main

Source: CMVBallCase.lean:1400-1406

Declaration SHA-256: 5dd1341e2c387a274bfb506d761e3f1b5f27cc70f720d3f55540418d8785332e

Mathematical statement. If $0 < \lambda < 1$ and $0 < R \leq \lambda$, then $P_C(R) < P_B(R)$.

In the low-radius region the gap identity and positivity of H give

$$P_B(R) - P_C(R) = 2H(\lambda, R) > 0.$$

This treats that entire region without invoking the density-derivative sign lemma.

```
lemma sameRadius_typeB_gt_typeC_of_le {lam R : ℝ}
  (hlam0 : 0 < lam) (hlam1 : lam < 1) (hR0 : 0 < R) (hR : R ≤ lam) :
  typeCPerimeterAtRadius lam R < typeBPerimeterAtRadius lam R := by
  have hgap := fixedRadiusHalfGap_pos_of_le hlam0 hlam1 hR0 hR
  rw [← sub_pos, perimeter_gap_eq_two_mul_halfGap]
  positivity
```

CMVBallCase.typeBAlpha

Classification: main

Source: CMVBallDefinitions.lean:11-13

Declaration SHA-256: b162ed041fbf8301c033d26810d7ba715023e6c12a98223c2a0238ca585853e2

Mathematical statement. The type-B auxiliary angle is $\alpha = \beta + \arccos \lambda$.

The definition is

$$\alpha = \beta + \arccos \lambda.$$

This is the half-angle of the unit-circle boundary arc in the source parametrization. On the admissible density and angle domain, $0 < \alpha \leq \pi$.

```
def typeBAlpha (lam beta : ℝ) : ℝ := beta + Real.arccos lam
/-- The exterior-circle radius for a type-(B) candidate. -/
```

CMVBallCase.typeBArea

Classification: main

Source: CMVBallDefinitions.lean:22-29

Declaration SHA-256: c81a1639e7c0ba21669318475c1f246edae8a287721cbc51038eb28187feccf8

Mathematical statement. The angular type-B weighted area is $r^2(\pi - \beta + \sin \beta \cos \beta) + \alpha - \sin \alpha \cos \alpha - (1 - \lambda)\pi$.

The two chord segments have areas $r^2 S(\pi - \beta)$ and $S(\alpha)$. Their sum is the Euclidean region area; subtract $(1 - \lambda)\pi$ because the region contains the entire unit disk. Thus, with $\alpha = \beta + \arccos \lambda$ and $r = \sin \alpha / \sin \beta$,

$$A_B(\beta) = r^2(\pi - \beta + \sin \beta \cos \beta) + \alpha - \sin \alpha \cos \alpha - (1 - \lambda)\pi.$$

```
def typeBArea (lam beta : ℝ) : ℝ :=
  (typeBRadius lam beta) ^ 2 *
  (Real.pi - beta + Real.sin beta * Real.cos beta) +
  (typeBAlpha lam beta -
```

```

    Real.sin (typeBAlpha lam beta) * Real.cos (typeBAlpha lam beta)) -
    (1 - lam) * Real.pi

/-- The orthogonal-ball radius for a type-(C) candidate. -/

```

CMVBallCase.typeBAreaAtRadius

Classification: main

Source: CMVBallCase.lean:326-334

Declaration SHA-256: 302ee62cf6614993146d2f898718e7b4c35270995d17efc0e3fbe06584054ba3

Mathematical statement. The radius-coordinate type-B area is $R^2(\pi - S(\beta_\lambda(R))) + S(\beta_\lambda(R) + \arccos \lambda) - (1 - \lambda)\pi$.

With $S(t) = t - \text{sintcost}$, the definition is

$$A_B(R) = R^2(\pi - S(\beta_\lambda(R))) + S(\beta_\lambda(R) + \arccos \lambda) - (1 - \lambda)\pi.$$

The final subtraction accounts for the density deficit on the entire unit disk contained in the candidate.

```

def typeBAreaAtRadius (lam R : ℝ) : ℝ :=
  R ^ 2 * (Real.pi - betaAtRadius lam R +
    Real.sin (betaAtRadius lam R) * Real.cos (betaAtRadius lam R)) +
  (betaAtRadius lam R + Real.arccos lam -
    Real.sin (betaAtRadius lam R + Real.arccos lam) *
    Real.cos (betaAtRadius lam R + Real.arccos lam)) -
  (1 - lam) * Real.pi

/-- Type-(C) weighted perimeter in its positive-radius chart. -/

```

CMVBallCase.typeBAreaAtRadius_agrees

Classification: main

Source: CMVBallCase.lean:352-357

Declaration SHA-256: c9cf34f1eb5dafbc32c5b2d81a1f3960d3e2a3fa29468cbef2fa5572ed5aa637

Mathematical statement. For $0 < \lambda < 1$ and $R \geq 0$, $A_B(R)$ equals the angular type-B area at $\beta_\lambda(R)$.

Substitution gives

$$A_B(R) = A_B(\beta_\lambda(R)),$$

with exactly the same density correction. Thus radius coordinates do not change the area constraint.

```

lemma typeBAreaAtRadius_agrees {lam R : ℝ}
  (hlam0 : 0 < lam) (hlam1 : lam < 1) (hR : 0 ≤ R) :
  typeBAreaAtRadius lam R = typeBArea lam (betaAtRadius lam R) := by
  rw [typeBAreaAtRadius, typeBArea, typeBAlpha,
    typeBRadius_betaAtRadius hlam0 hlam1 hR]

```

CMVBallCase.typeBAreaAtRadius_continuous

Classification: main

Source: CMVBallCase.lean:693-697

Declaration SHA-256: 654d93958fa1cc148933f3dbe93ac2d4925ccbef30b42f7a8afd01edc8ef532d

Mathematical statement. For every real λ , the total function $R \mapsto A_B(R)$ is continuous on the real line.

For fixed λ , the denominator $\sqrt{1 - \lambda^2}$ is a constant in the total real expression. Continuity of arctangent, trigonometric functions, and algebraic operations proves global continuity, including $R = 0$.

```
lemma typeBAreaAtRadius_continuous {lam : ℝ} :
  Continuous (typeBAreaAtRadius lam) := by
  unfold typeBAreaAtRadius betaAtRadius
  fun_prop
```

CMVBallCase.typeBAreaAtRadius_deriv_pos

Classification: main

Source: CMVBallCase.lean:663-669

Declaration SHA-256: 4fcb16a2eb66bac06f3aac56a18581ba3fae227c21397173be1ab0d712a98430

Mathematical statement. For $0 < \lambda < 1$ and $R > 0$, the derivative of A_B at R is positive.

The derivative $2RQ_B(R)$ is a product of positive factors. This establishes strict area growth away from the zero-radius endpoint.

```
lemma typeBAreaAtRadius_deriv_pos {lam R : ℝ}
  (hlam0 : 0 < lam) (hlam1 : lam < 1) (hR : 0 < R) :
  0 < deriv (typeBAreaAtRadius lam) R := by
  rw [(typeBAreaAtRadius_hasDerivAt hlam0 hlam1 hR).deriv]
  have hq := typeB_perimeter_factor_pos hlam0 hlam1 hR
  positivity
```

CMVBallCase.typeBAreaAtRadius_hasDerivAt

Classification: main

Source: CMVBallCase.lean:535-571

Declaration SHA-256: e5d44f049460bf59353073b4b5ae0475e767191b44bc8056fa589b30ce308ada

Mathematical statement. For $0 < \lambda < 1$ and $R > 0$, A_B has derivative $2RQ_B(R)$ at R .

After differentiating the two segment terms, $\sin(\beta_\lambda(R) + a) = R\sin\beta_\lambda(R)$ cancels their angular derivatives. Thus

$$A_B'(R) = 2RQ_B(R) = RP_B'(R).$$

```
lemma typeBAreaAtRadius_hasDerivAt {lam R : ℝ}
  (hlam0 : 0 < lam) (hlam1 : lam < 1) (hR : 0 < R) :
  HasDerivAt (typeBAreaAtRadius lam)
  (R * (2 * (Real.pi - betaAtRadius lam R +
    Real.sin (betaAtRadius lam R) * Real.cos (betaAtRadius lam R)))) R := by
  let db := -Real.sqrt (1 - lam ^ 2) /
    ((R - lam) ^ 2 + Real.sqrt (1 - lam ^ 2) ^ 2)
  have hb : HasDerivAt (betaAtRadius lam) db R :=
    betaAtRadius_hasDerivAt hlam0 hlam1
  have hsB := arcSegment_hasDerivAt hb
  have ha := hb.add_const (Real.arccos lam)
  have hsA := arcSegment_hasDerivAt ha
  have hq := (hasDerivAt_const R Real.pi).sub hsB
  have hr2 : HasDerivAt (fun x : ℝ => x ^ 2) (2 * R) R := by
    convert (hasDerivAt_id R).pow 2 using 1 <|>
    first | rfl | norm_num
  have hraw := (hr2.mul hq).add hsA |>.sub_const ((1 - lam) * Real.pi)
  have heq : typeBAreaAtRadius lam =f[nhds R]
    (fun x => x ^ 2 * (Real.pi - arcSegment (betaAtRadius lam x)) +
      arcSegment (betaAtRadius lam x + Real.arccos lam) -
      (1 - lam) * Real.pi) := by
    exact Filter.Eventually.of_forall fun x => by
      unfold typeBAreaAtRadius arcSegment
      ring
  have hfun := hraw.congr_of_eventuallyEq heq
  have hsin := sin_typeBAlpha_betaAtRadius hlam0 hlam1 hR.le
  have hcoef : R * (2 * (Real.pi - betaAtRadius lam R +
    Real.sin (betaAtRadius lam R) * Real.cos (betaAtRadius lam R))) =
    2 * R * (Real.pi - arcSegment (betaAtRadius lam R)) +
```

```

      R ^ 2 * (0 - 2 * Real.sin (betaAtRadius lam R) ^ 2 * db) +
      2 * Real.sin (betaAtRadius lam R + Real.arccos lam) ^ 2 * db := by
    rw [hsin]
    simp only [arcSegment]
    ring
    rw [hcoef]
    exact hfun

```

CMVBallCase.typeBAreaAtRadius_large_lower

Classification: main

Source: CMVBallCase.lean:784-809

Declaration SHA-256: 141341249a38db9c1c4d40b1c1642f4101a23d6266b432cbe39ad76a061df4b3

Mathematical statement. For $0 < \lambda < 1$ and $R \geq \lambda$, $A_B(R) > (\pi/2)R^2 - \pi$.

Here $\beta_\lambda(R) \leq \pi/2$, so $\pi - S(\beta_\lambda(R)) \geq \pi/2$; the other segment area is nonnegative. The density correction is strictly less than π , giving

$$A_B(R) > \frac{\pi}{2}R^2 - \pi.$$

The bound proves unbounded growth.

```

lemma typeBAreaAtRadius_large_lower {lam R : ℝ}
  (hlam0 : 0 < lam) (hlam1 : lam < 1) (hR : lam ≤ R) :
  R ^ 2 * (Real.pi / 2) - Real.pi < typeBAreaAtRadius lam R := by
  have hR0 : 0 < R := lt_of_lt_of_le hlam0 hR
  have hb := betaAtRadius_admissible hlam0 hlam1 hR0.le
  have hb2 := betaAtRadius_le_pi_div_two hlam0 hlam1 hR
  have hs := Real.sin_pos_of_pos_of_lt_pi hb.1
  (by linarith [hb.2, Real.pi_pos])
  have hc := Real.cos_nonneg_of_mem_Icc
  (by linarith [hb.1, Real.pi_pos], hb2)
  have hsegB : arcSegment (betaAtRadius lam R) ≤ Real.pi / 2 := by
    unfold arcSegment
    nlinarith
  have ha0 : 0 ≤ betaAtRadius lam R + Real.arccos lam :=
    add_nonneg hb.1.le (Real.arccos_nonneg lam)
  have hapi : betaAtRadius lam R + Real.arccos lam ≤ Real.pi := by
    linarith [hb.2]
  have hsegA : 0 ≤ arcSegment
    (betaAtRadius lam R + Real.arccos lam) :=
    arcSegment_nonneg ha0 hapi
  have hq : Real.pi / 2 ≤ Real.pi - arcSegment (betaAtRadius lam R) := by
    linarith
  have hp : 0 < Real.pi := Real.pi_pos
  unfold typeBAreaAtRadius arcSegment at *
  nlinarith [sq_nonneg R]

```

CMVBallCase.typeBAreaAtRadius_strictMonoOn

Classification: main

Source: CMVBallCase.lean:713-721

Declaration SHA-256: ab4db7494426ac1cdccec00e6e10d30da1a1b7a3f1a760b845607901df77bbd3

Mathematical statement. For $0 < \lambda < 1$, A_B is strictly increasing on $[0, \infty)$.

Positive derivative on $(0, \infty)$, together with continuity at zero, gives strict increase on the closed half-line by the mean value theorem. Thus even the retained endpoint introduces no area branch.

```

lemma typeBAreaAtRadius_strictMonoOn {lam : ℝ}
  (hlam0 : 0 < lam) (hlam1 : lam < 1) :
  StrictMonoOn (typeBAreaAtRadius lam) (Set.Ici 0) := by

```

```

apply strictMonoOn_of_deriv_pos (convex_Ici 0)
  typeBAreaAtRadius_continuous.continuousOn
intro R hR
rw [interior_Ici] at hR
exact typeBAreaAtRadius_deriv_pos hlam0 hlam1 hR

```

CMVBallCase.typeBAreaAtRadius_zero

Classification: main

Source: CMVBallCase.lean:749-758

Declaration SHA-256: 6d532317078066c3097d5d0410644fac9c192ee7d6af6804f7e2c0d6a360b53d

Mathematical statement. For $0 < \lambda < 1$, $A_B(0) = \lambda\pi$.

At radius zero, $\beta_\lambda(0) + a = \pi$. The radius term vanishes and the remaining area is

$$A_B(0) = S(\pi) - (1 - \lambda)\pi = \lambda\pi.$$

```

lemma typeBAreaAtRadius_zero {lam : ℝ}
  (hlam0 : 0 < lam) (hlam1 : lam < 1) :
  typeBAreaAtRadius lam 0 = lam * Real.pi := by
  rw [typeBAreaAtRadius_agrees hlam0 hlam1 (le_refl 0),
    betaAtRadius_zero hlam0 hlam1, typeBArea, typeBRadius_endpoint_zero,
    typeBAlpha]
  rw [show Real.pi - Real.arccos lam + Real.arccos lam = Real.pi by ring,
    Real.sin_pi]
  ring

```

CMVBallCase.typeBPerimeter

Classification: main

Source: CMVBallDefinitions.lean:18-21

Declaration SHA-256: 3f47b16460dd98ef1ba0cd3846453bf4afaacfb39faf78478473722bd9ce0388

Mathematical statement. The angular type-B weighted perimeter is $2((\pi - \beta)r + \lambda\alpha)$, with $\alpha = \beta + \arccos\lambda$ and $r = \sin\alpha/\sin\beta$.

The exterior arc has length $2r(\pi - \beta)$ and density one; the unit-circle arc has length 2α and density λ . Their weighted sum is

$$P_B(\beta) = 2((\pi - \beta)r + \lambda\alpha).$$

```

def typeBPerimeter (lam beta : ℝ) : ℝ :=
  2 * ((Real.pi - beta) * typeBRadius lam beta + lam * typeBAlpha lam beta)

/-- CMV's weighted area formula for a type-(B) candidate. -/

```

CMVBallCase.typeBPerimeterAtArea

Classification: main

Source: CMVBallCase.lean:1080-1084

Declaration SHA-256: 98f0853f0439181bac4c27175769bb69c6bb2e98d9167f9ffa0f7bf07436ebdd

Mathematical statement. The equal-area type-B profile is the real infimum of its all-candidate perimeter set.

The definition is

$$I_B(v) = \inf \mathcal{P}_B(v),$$

implemented by the total real infimum operation. On $0 < \lambda < 1, v > \lambda\pi$, the set is proved to be a nonempty singleton, so no empty-set convention affects the theorem.

```
def typeBPerimeterAtArea (lam v : ℝ) : ℝ :=
  sInf (typeBPerimeterValuesAtArea lam v)

/-- The equal-area type-(C) profile. On its geometric range the preceding
value set is proved to be a singleton, so this infimum is that unique value. -/
```

CMVBallCase.typeBPerimeterAtArea_eq

Classification: main

Source: CMVBallCase.lean:1124-1130

Declaration SHA-256: 57097d90269686a2eff890ae3d5ae1d3b35601393a261dbde597136f7ab0b41c

Mathematical statement. If β is the unique admissible type-B angle of area v , then $I_B(v) = P_B(\beta)$.

Under the admissibility, area, and angular uniqueness hypotheses, the value set is a singleton. Its real infimum is its sole element, proving

$$I_B(v) = P_B(\beta).$$

```
lemma typeBPerimeterAtArea_eq {lam v beta : ℝ}
  (hbeta : BAdmissible lam beta) (harea : typeBArea lam beta = v)
  (hunique : ∀ beta', BAdmissible lam beta' → typeBArea lam beta' = v → beta' = beta) :
  typeBPerimeterAtArea lam v = typeBPerimeter lam beta := by
  rw [typeBPerimeterAtArea, typeB_values_eq_singleton hbeta harea hunique]
  exact csInf_singleton _
```

CMVBallCase.typeBPerimeterAtArea_eq_radius

Classification: main

Source: CMVBallCase.lean:1165-1182

Declaration SHA-256: 28e93575ceb4de605bf1e8801ce418243463ddd89e855c02916af6db2f66c59d

Mathematical statement. For $0 < \lambda < 1, R > 0$, and $A_B(R) = v$, the all-candidate profile satisfies $I_B(v) = P_B(R)$.

Any admissible angle of area v has a nonnegative radius; strict area monotonicity forces that radius to be R , and angular injectivity forces the angle to be $\beta_\lambda(R)$. The infimum is therefore this genuine perimeter:

$$I_B(v) = P_B(R).$$

```
lemma typeBPerimeterAtArea_eq_radius {lam v R : ℝ}
  (hlam0 : 0 < lam) (hlam1 : lam < 1)
  (hR : 0 < R) (harea : typeBAreaAtRadius lam R = v) :
  typeBPerimeterAtArea lam v = typeBPerimeterAtRadius lam R := by
  rw [typeBPerimeterAtArea_eq (betaAtRadius_admissible hlam0 hlam1 hR.le)
    ((typeBAreaAtRadius_agrees hlam0 hlam1 hR.le).symm.trans harea)]
  · exact (typeBPerimeterAtRadius_agrees hlam0 hlam1 hR.le).symm
  · intro beta' hbeta' harea'
    apply typeBRadius_injective_on_admissible hlam0 hlam1 hbeta'
      (betaAtRadius_admissible hlam0 hlam1 hR.le)
    have hR' := typeBRadius_nonneg hlam0 hlam1 hbeta'
    have hREq : typeBRadius lam beta' = R := by
      apply (typeBAreaAtRadius_strictMonoOn hlam0 hlam1).injOn hR' hR.le
      rw [typeBAreaAtRadius_agrees hlam0 hlam1 hR',
        betaAtRadius_typeBRadius hlam0 hlam1 hbeta', harea', harea]
    rw [typeBRadius_betaAtRadius hlam0 hlam1 hR.le]
    exact hREq
```

CMVBallCase.typeBPerimeterAtArea_gt_commonEndpoint

Classification: main

Source: CMVBallCase.lean:2321-2333

Declaration SHA-256: 1bc2f925dd32fdc3436ba53a34596445576e91faa97e786b1962cb23ce1fbf2b

Mathematical statement. If $0 < \lambda < 1$ and $v > \lambda\pi$, then $I_B(v) > 2\lambda\pi$.

The type-B area v is realized at a positive radius R . Strict perimeter increase gives

$$I_B(v) = P_B(R) > P_B(0) = 2\lambda\pi,$$

making the profile perimeter a valid argument of the common-perimeter gap.

```
lemma typeBPerimeterAtArea_gt_commonEndpoint {lam v : ℝ}
  (hlam0 : 0 < lam) (hlam1 : lam < 1)
  (hv : lam * Real.pi < v) :
  2 * lam * Real.pi < typeBPerimeterAtArea lam v := by
  rcases exists_typeBRadius_at_area hlam0 hlam1 hv with ⟨R, hR, harea⟩
  change 0 < R at hR
  rw [typeBPerimeterAtArea_eq_radius hlam0 hlam1 hR harea,
    ← typeBPerimeterAtRadius_zero hlam0 hlam1]
  exact typeBPerimeterAtRadius_strictMonoOn hlam0 hlam1
    (show (0 : ℝ) ≤ 0 from le_refl) hR.le hR

/-- Weak C dominance at an area puts the common-perimeter area gap at the B
profile value at or below zero. -/
```

CMVBallCase.typeBPerimeterAtArea_strictMonoOn

Classification: main

Source: CMVBallCase.lean:1201-1219

Declaration SHA-256: c3d6e28ec8a214fad8521a957798e0d4a463a65ea8d06f253bc35c5c42925728

Mathematical statement. For $0 < \lambda < 1$, I_B is strictly increasing on $(\lambda\pi, \infty)$.

If $\lambda\pi < v < w$, the unique area radii satisfy $R_v < R_w$ by strict area increase. Strict perimeter increase and the profile evaluation identities give

$$I_B(v) < I_B(w).$$

```
lemma typeBPerimeterAtArea_strictMonoOn {lam : ℝ}
  (hlam0 : 0 < lam) (hlam1 : lam < 1) :
  StrictMonoOn (typeBPerimeterAtArea lam) (Set.Ioi (lam * Real.pi)) := by
  intro v hv w hw hvw
  rcases exists_typeBRadius_at_area hlam0 hlam1 hv with ⟨R, hR, hareaR⟩
  rcases exists_typeBRadius_at_area hlam0 hlam1 hw with ⟨S, hS, hareaS⟩
  change 0 < R at hR
  change 0 < S at hS
  have hRS : R < S := by
    by_contra hn
    have hle : S ≤ R := le_of_not_gt hn
    have ha := (typeBAreaAtRadius_strictMonoOn hlam0 hlam1).monotoneOn
      hS.le hR.le hle
    rw [hareaS, hareaR] at ha
    linarith
  rw [typeBPerimeterAtArea_eq_radius hlam0 hlam1 hR hareaR,
    typeBPerimeterAtArea_eq_radius hlam0 hlam1 hS hareaS]
  exact typeBPerimeterAtRadius_strictMonoOn hlam0 hlam1 hR.le hS.le hRS
```

CMVBallCase.typeBPerimeterAtRadius

Classification: main

Source: CMVBallCase.lean:321-325

Declaration SHA-256: c9c0993d1f5fbe4ba58debefa9858ea8f0b24ef4559a978342fbbc737aa8f2ff

Mathematical statement. The radius-coordinate type-B perimeter is $2((\pi - \beta_\lambda(R))R + \lambda(\beta_\lambda(R) + \arccos\lambda))$.

The definition is

$$P_B(R) = 2((\pi - \beta_\lambda(R))R + \lambda(\beta_\lambda(R) + \arccos\lambda)).$$

It weights the exterior arc by one and the unit-circle arc by λ ; equality with the angular perimeter is proved for $0 < \lambda < 1$, $R \geq 0$.

```
def typeBPerimeterAtRadius (lam R : ℝ) : ℝ :=
  2 * ((Real.pi - betaAtRadius lam R) * R +
    lam * (betaAtRadius lam R + Real.arccos lam))

/-- Type-(B) weighted area in the global radius chart. -/
```

CMVBallCase.typeBPerimeterAtRadius_agrees

Classification: main

Source: CMVBallCase.lean:345-351

Declaration SHA-256: fc3ebabe5ff5963057732d5c2e6cb24a284b9bf252ebb2ffea0fa55fa6ac0813

Mathematical statement. For $0 < \lambda < 1$ and $R \geq 0$, $P_B(R)$ equals the angular type-B perimeter at $\beta_\lambda(R)$.

Substituting the exact radius inverse gives

$$P_B(R) = P_B(\beta_\lambda(R)).$$

The equality includes the zero-radius endpoint.

```
lemma typeBPerimeterAtRadius_agrees {lam R : ℝ}
  (hlam0 : 0 < lam) (hlam1 : lam < 1) (hR : 0 ≤ R) :
  typeBPerimeterAtRadius lam R =
    typeBPerimeter lam (betaAtRadius lam R) := by
  rw [typeBPerimeterAtRadius, typeBPerimeter, typeBAlpha,
    typeBRadius_betaAtRadius hlam0 hlam1 hR]
```

CMVBallCase.typeBPerimeterAtRadius_continuous

Classification: main

Source: CMVBallCase.lean:698-702

Declaration SHA-256: bb5bba6b7941dc1416656ad77a30a9cf4928feec80267c75bbd92466dad64edc

Mathematical statement. For every real λ , the total function $R \mapsto P_B(R)$ is continuous on the real line.

The perimeter is formed from the continuous radius angle β_λ and polynomial operations in R . The result supplies continuity at the retained type-B endpoint.

```
lemma typeBPerimeterAtRadius_continuous {lam : ℝ} :
  Continuous (typeBPerimeterAtRadius lam) := by
  unfold typeBPerimeterAtRadius betaAtRadius
  fun_prop
```

CMVBallCase.typeBPerimeterAtRadius_deriv_pos

Classification: main

Source: CMVBallCase.lean:677-684

Declaration SHA-256: 10de2cba5f52d44c61da385e12b1e3a912b33e8b23dd2e310115984cf7543ad2

Mathematical statement. For $0 < \lambda < 1$, every positive R satisfies $P_B'(R) > 0$.

The derivative equals $2Q_B(R) > 0$. Its nonvanishing is also the hypothesis required for the local inverse function theorem.

```
lemma typeBPerimeterAtRadius_deriv_pos {lam R : ℝ}
  (hlam0 : 0 < lam) (hlam1 : lam < 1) :
  0 < R → 0 < deriv (typeBPerimeterAtRadius lam) R := by
  intro hR
  rw [(typeBPerimeterAtRadius_hasDerivAt hlam0 hlam1).deriv]
  have hq := typeB_perimeter_factor_pos hlam0 hlam1 hR
  positivity
```

CMVBallCase.typeBPerimeterAtRadius_hasDerivAt

Classification: main

Source: CMVBallCase.lean:441-464

Declaration SHA-256: a36eb5dc07528a5ef5ac5c46d69519b24cf266609e428cac791ae feae3d159fc

Mathematical statement. For $0 < \lambda < 1$ and every real R , P_B has derivative $2Q_B(R)$ at R .

Here $Q_B(R) = \pi - \beta_\lambda(R) + \sin\beta_\lambda(R)\cos\beta_\lambda(R)$. Differentiating the perimeter and substituting the chart derivative gives

$$P_B'(R) = 2Q_B(R).$$

```
lemma typeBPerimeterAtRadius_hasDerivAt {lam R : ℝ}
  (hlam0 : 0 < lam) (hlam1 : lam < 1) :
  HasDerivAt (typeBPerimeterAtRadius lam)
    (2 * (Real.pi - betaAtRadius lam R +
      Real.sin (betaAtRadius lam R) * Real.cos (betaAtRadius lam R))) R := by
  let db := -Real.sqrt (1 - lam ^ 2) /
    ((R - lam) ^ 2 + Real.sqrt (1 - lam ^ 2) ^ 2)
  have hb : HasDerivAt (betaAtRadius lam) db R :=
    betaAtRadius_hasDerivAt hlam0 hlam1
  have hraw := (((hasDerivAt_const R Real.pi).sub hb).mul
    (hasDerivAt_id R)).add
    ((hb.add_const (Real.arccos lam)).const_mul lam) |>.const_mul 2
  have hfun : HasDerivAt (typeBPerimeterAtRadius lam)
    (2 * ((0 - db) * R + (Real.pi - betaAtRadius lam R) * 1 +
      lam * db)) R := by
    apply hraw.congr_of_eventuallyEq
    exact Filter.Eventually.of_forall fun x => by
      simp [typeBPerimeterAtRadius, id]
    convert hfun using 1
  dsimp [db]
  rw [sin_betaAtRadius_mul_cos hlam0 hlam1]
  field_simp
  ring
```

CMVBallCase.typeBPerimeterAtRadius_hasStrictDerivAt

Classification: main

Source: CMVBallCase.lean:2072-2089

Declaration SHA-256: 89c1125258117d39f6582f2d8c1a2115de855d8212c426a1e171fed0a408f19b

Mathematical statement. For $0 < \lambda < 1$ and every real R , P_B has strict derivative $2Q_B(R)$ at R .

Strict differentiability means the two-point limit

$$\lim_{\substack{x, y \rightarrow R \\ x \neq y}} \frac{P_B(x) - P_B(y)}{x - y} = 2Q_B(R).$$

The ordinary derivative formula is continuous, so strict differentiability follows; at positive radii its nonzero value allows local inversion.

```
lemma typeBPerimeterAtRadius_hasStrictDerivAt {lam R : ℝ}
  (hlam0 : 0 < lam) (hlam1 : lam < 1) :
  HasStrictDerivAt (typeBPerimeterAtRadius lam)
    (2 * (Real.pi - betaAtRadius lam R +
      Real.sin (betaAtRadius lam R) * Real.cos (betaAtRadius lam R))) R := by
let d : ℝ → ℝ := fun x =>
  2 * (Real.pi - betaAtRadius lam x +
    Real.sin (betaAtRadius lam x) * Real.cos (betaAtRadius lam x))
have hstrict : HasStrictDerivAt (typeBPerimeterAtRadius lam) (d R) R := by
  apply hasStrictDerivAt_of_hasDerivAt_of_continuousAt
  · exact Filter.Eventually.of_forall fun x =>
    typeBPerimeterAtRadius_hasDerivAt (R := x) hlam0 hlam1
  · have hb : ContinuousAt (betaAtRadius lam) R :=
    (betaAtRadius_hasDerivAt hlam0 hlam1).continuousAt
    fun_prop
  exact hstrict

/-- The C perimeter has a strict derivative at every positive radius. -/
```

CMVBallCase.typeBPerimeterAtRadius_strictMonoOn

Classification: main

Source: CMVBallCase.lean:731-739

Declaration SHA-256: bcb0c4329c9e367c3237ca68300c9a926c8d4b286caa51d477c1d94e742ff2e8

Mathematical statement. For $0 < \lambda < 1$, P_B is strictly increasing on $[0, \infty)$.

Continuity and positivity of the derivative in the interior yield strict increase, including comparisons with $P_B(0) = 2\lambda\pi$. This makes the common-perimeter inverse unique.

```
lemma typeBPerimeterAtRadius_strictMonoOn {lam : ℝ}
  (hlam0 : 0 < lam) (hlam1 : lam < 1) :
  StrictMonoOn (typeBPerimeterAtRadius lam) (Set.Ici 0) := by
  apply strictMonoOn_of_deriv_pos (convex_Ici 0)
  typeBPerimeterAtRadius_continuous.continuousOn
  intro R hR
  rw [interior_Ici] at hR
  exact typeBPerimeterAtRadius_deriv_pos hlam0 hlam1 hR
```

CMVBallCase.typeBPerimeterAtRadius_zero

Classification: main

Source: CMVBallCase.lean:759-766

Declaration SHA-256: 4a8b3af699ffc34588703cbf0b3dfa0fe068d5c1e0ebc207f5e62ad1f86bcb22

Mathematical statement. For $0 < \lambda < 1$, $P_B(0) = 2\lambda\pi$.

Only the weighted unit-circle arc remains at the type-B endpoint. Its angle is 2π , hence its weighted length is

$$P_B(0) = 2\lambda\pi.$$

```
lemma typeBPerimeterAtRadius_zero {lam : ℝ}
  (hlam0 : 0 < lam) (hlam1 : lam < 1) :
  typeBPerimeterAtRadius lam 0 = 2 * lam * Real.pi := by
  rw [typeBPerimeterAtRadius_agrees hlam0 hlam1 (le_refl 0),
    betaAtRadius_zero hlam0 hlam1, typeBPerimeter, typeBRadius_endpoint_zero,
```

```
typeBAlpha]
ring
```

CMVBallCase.typeBPerimeterValuesAtArea

Classification: main

Source: CMVBallCase.lean:1068-1073

Declaration SHA-256: 7ffddd7593a4f7f71be09ace34b7c0443247b4c6e896e8cebb972d294f5834b0

Mathematical statement. This is the set of all type-B perimeter values from admissible angles of area exactly v .

Precisely,

$$\mathcal{P}_B(v) = \{p: \exists \beta, 0 < \beta \leq \pi - a, A_B(\beta) = v, P_B(\beta) = p\}.$$

No minimizing branch or perimeter ordering is built into this definition.

```
def typeBPerimeterValuesAtArea (lam v : ℝ) : Set ℝ :=
  { p | ∃ beta, BAdmissible lam beta ∧ typeBArea lam beta = v ∧
    typeBPerimeter lam beta = p }

/-- Every type-(C) perimeter arising from an admissible candidate of exactly
weighted area `v`. -/
```

CMVBallCase.typeBRadius

Classification: main

Source: CMVBallDefinitions.lean:14-17

Declaration SHA-256: 495dd878203ed375c56ea0dbe5c390a5dfde6e2a9e8eec96e771b8ebff636634

Mathematical statement. The type-B exterior-circle radius is $\sin(\beta + \arccos \lambda) / \sin \beta$.

The common chord gives $r \sin \beta = \sin \alpha$, so

$$r = \frac{\sin \alpha}{\sin \beta} = \frac{\sin(\beta + \arccos \lambda)}{\sin \beta}.$$

Admissibility for $0 < \lambda < 1$ ensures the denominator is positive and the radius nonnegative.

```
def typeBRadius (lam beta : ℝ) : ℝ :=
  Real.sin (typeBAlpha lam beta) / Real.sin beta

/-- CMV's weighted perimeter formula for a type-(B) candidate. -/
```

CMVBallCase.typeBRadiusAtPerimeter

Classification: main

Source: CMVBallCase.lean:1923-1930

Declaration SHA-256: 641ef25025d32c528f2e106a8f3ebc604806525be72db2a80847283bc6cc25ee

Mathematical statement. For an admissible density and $p > 2\lambda\pi$, $r_B(p)$ is the unique positive radius with $P_B(r_B(p)) = p$.

The source chooses the witness from the proved existence-and-uniqueness theorem, retaining its characterization

$$r_B(p) > 0, \quad P_B(r_B(p)) = p.$$

No inverse value is specified at or below the endpoint.

```
def typeBRadiusAtPerimeter {lam : ℝ} (hlam : AdmissibleDensity lam)
  (p : CommonPerimeter lam) :
  {R : ℝ // 0 < R ∧ typeBPerimeterAtRadius lam R = p.1} :=
```

```

    (existsUnique_typeBRadius_at_perimeter hlam.1 hlam.2 p.2).choose,
    (existsUnique_typeBRadius_at_perimeter hlam.1 hlam.2 p.2).choose_spec.1)

/-- The type-(C) common-perimeter inverse carries both positivity and its
defining equation in its codomain; there is no value outside the open domain. -/

```

CMVBallCase.typeBRadiusAtPerimeter_lt_typeCRadiusAtPerimeter

Classification: main

Source: CMVBallCase.lean:1975-1991

Declaration SHA-256: 37fe08774ef3026b52e3131542e0b473bbc2fb7ecab7c6ff79923246a9265841

Mathematical statement. For $0 < \lambda < 1$ and $p > 2\lambda\pi$, $r_B(p) < r_C(p)$.

If $r_C(p) \leq r_B(p)$, monotonicity and the equal-radius comparison would give

$$p = P_C(r_C(p)) \leq P_C(r_B(p)) < P_B(r_B(p)) = p.$$

This contradiction proves strict radius order.

```

theorem typeBRadiusAtPerimeter_lt_typeCRadiusAtPerimeter {lam : ℝ}
  (hlam : AdmissibleDensity lam) (p : CommonPerimeter lam) :
  (typeBRadiusAtPerimeter hlam p).1 < (typeCRadiusAtPerimeter hlam p).1 := by
  by_contra hnot
  have hle : (typeCRadiusAtPerimeter hlam p).1 ≤
    (typeBRadiusAtPerimeter hlam p).1 := le_of_not_gt hnot
  have hmono := (typeCPerimeterAtRadius_strictMonoOn hlam.1 hlam.2).monotoneOn
  have hperim_le : typeCPerimeterAtRadius lam (typeCRadiusAtPerimeter hlam p) ≤
    typeCPerimeterAtRadius lam (typeBRadiusAtPerimeter hlam p) :=
    hmono (typeCRadiusAtPerimeter_pos hlam p) (typeBRadiusAtPerimeter_pos hlam p) hle
  rw [typeCPerimeterAt_typeCRadiusAtPerimeter hlam p] at hperim_le
  have hgap := sameRadius_typeB_gt_typeC hlam.1 hlam.2
    (typeBRadiusAtPerimeter_pos hlam p)
  rw [typeBPerimeterAt_typeBRadiusAtPerimeter hlam p] at hgap
  linarith

/-- Both common-perimeter inverse radii strictly increase with the perimeter. -/

```

CMVBallCase.typeBRadius_betaAtRadius

Classification: main

Source: CMVBallCase.lean:200-211

Declaration SHA-256: e66b10b3d091fb706b8fab34231c42e11270bceb8fc1f4044acdab9ecdf216f5

Mathematical statement. For $0 < \lambda < 1$ and $R \geq 0$, the radius of $\beta_\lambda(R)$ is exactly R .

Substitute $\cot\beta_\lambda(R) = (R - \lambda)/s$ into the radius formula to obtain

$$\lambda + \operatorname{scot}\beta_\lambda(R) = R.$$

Positivity of s legitimizes cancellation and establishes surjectivity onto all nonnegative radii.

```

lemma typeBRadius_betaAtRadius {lam R : ℝ}
  (hlam0 : 0 < lam) (hlam1 : lam < 1) (hR : 0 ≤ R) :
  typeBRadius lam (betaAtRadius lam R) = R := by
  rw [typeBRadius_eq_add_sqrt_mul_cot hlam0 hlam1
    (betaAtRadius_admissible hlam0 hlam1 hR), cot_betaAtRadius hlam0 hlam1]
  have hsqrt : Real.sqrt (1 - lam ^ 2) ≠ 0 :=
    (sqrt_one_sub_sq_pos hlam0 hlam1).ne'
  field_simp [hsqrt] <|> ring

/-- The inverse chart validates the `R < λ` quadrant without a selected
fallback branch. -/

```

CMVBallCase.typeBRRadius_endpoint_zero

Classification: main

Source: CMVBallCase.lean:278-283

Declaration SHA-256: 22146b3bf286e99a897eee49e1b29bb2805a205fd3eb8650ff9bf77fc6595ab6

Mathematical statement. For every real λ , the type-B radius formula at $\beta = \pi - \arccos \lambda$ equals zero.

At this input $\alpha = \pi$, so the defining quotient evaluates to

$$\frac{\sin((\pi - \arccos \lambda) + \arccos \lambda)}{\sin(\pi - \arccos \lambda)} = 0.$$

The identity holds for the total real expression; its geometric interpretation is used only for $0 < \lambda < 1$.

```
lemma typeBRRadius_endpoint_zero {lam : ℝ} :
  typeBRRadius lam (Real.pi - Real.arccos lam) = 0 := by
  rw [typeBRRadius, typeBAlpha,
    show Real.pi - Real.arccos lam + Real.arccos lam = Real.pi by ring,
    Real.sin_pi, zero_div]
```

CMVBallCase.typeBRRadius_eq_add_sqrt_mul_cot

Classification: main

Source: CMVBallCase.lean:81-91

Declaration SHA-256: 3eb7e5731012db737e7b336d5d6d605bb445d1c2a4e8d3c0f094ba10820bb6da

Mathematical statement. For $0 < \lambda < 1$ and admissible β , the type-B radius equals $\lambda + \sqrt{1 - \lambda^2} \cot \beta$.

Expanding $\sin(\beta + \arccos \lambda)$ and dividing by the positive $\sin \beta$ gives

$$r = \lambda + \cot \beta.$$

This identity supplies a global inverse without a quadrant-dependent inverse tangent.

```
lemma typeBRRadius_eq_add_sqrt_mul_cot {lam beta : ℝ}
  (hlam0 : 0 < lam) (hlam1 : lam < 1) (hbeta : BAdmissible lam beta) :
  typeBRRadius lam beta =
    lam + Real.sqrt (1 - lam ^ 2) * Real.cot beta := by
  have hsin : Real.sin beta ≠ 0 := (typeB_sin_pos hlam0 hlam1 hbeta).ne'
  rw [typeBRRadius, typeBAlpha, Real.sin_add,
    Real.cos_arccos (by linarith) (by linarith), Real.sin_arccos,
    Real.cot_eq_cos_div_sin]
  field_simp [hsin] <=> ring

/-- Cotangent is strictly decreasing on the whole geometric angle interval. -/
```

CMVBallCase.typeBRRadius_injective_on_admissible

Classification: main

Source: CMVBallCase.lean:120-126

Declaration SHA-256: 0f30bd9be41461ee8fad9222c1b7b1550f99309779ba29cd8036dd728bb91efb

Mathematical statement. For $0 < \lambda < 1$, equal radii of admissible type-B angles imply equal angles.

Injectivity follows from strict decrease on $(0, \pi - \alpha]$. This is the uniqueness statement needed to recover every angular candidate from the global radius chart.

```
lemma typeBRRadius_injective_on_admissible {lam : ℝ}
  (hlam0 : 0 < lam) (hlam1 : lam < 1) :
  Set.InjOn (typeBRRadius lam) {beta | BAdmissible lam beta} :=
```

```
(typeBRadius_strictAntiOn hlam0 hlam1).injOn
```

```
/-- The branch-safe type-(B) inverse radius chart. The single arctangent
formula lands above  $\pi / 2$  exactly when  $R < \lambda$ . -/
```

CMVBallCase.typeBRadius_nonneg

Classification: main

Source: CMVBallCase.lean:232-240

Declaration SHA-256: 3582dff45831f2e7f5c8a52df6dd4d3c3ab5060e81b548fb6a841a2254794a31

Mathematical statement. For $0 < \lambda < 1$, every admissible type-B candidate has nonnegative radius.

In $r = \sin\alpha/\sin\beta$, the numerator is nonnegative and the denominator positive. The zero value is allowed at the retained angular endpoint.

```
lemma typeBRadius_nonneg {lam beta : ℝ} (hlam0 : 0 < lam) (hlam1 : lam < 1)
  (hbeta : BAdmissible lam beta) :
  0 ≤ typeBRadius lam beta := by
  rw [typeBRadius]
  exact div_nonneg (typeBAlpha_sin_nonneg hbeta)
    (typeB_sin_pos hlam0 hlam1 hbeta).le

/-- The complete nonnegative type-(B) radius range, including its zero-radius
endpoint. -/
```

CMVBallCase.typeBRadius_strictAntiOn

Classification: main

Source: CMVBallCase.lean:103-119

Declaration SHA-256: d474e75d38db799e62e6a1fd3b84d82959bcc51226ab32f06c71df9e248314af

Mathematical statement. For $0 < \lambda < 1$, the type-B radius is strictly decreasing on its whole admissible angular domain.

On $0 < \beta \leq \pi - a$, $r = \lambda + \operatorname{scot}\beta$ with $s > 0$. Strict decrease of cotangent gives strict decrease of radius, including comparisons with the zero-radius endpoint.

```
lemma typeBRadius_strictAntiOn {lam : ℝ} (hlam0 : 0 < lam) (hlam1 : lam < 1) :
  StrictAntiOn (typeBRadius lam) {beta | BAdmissible lam beta} := by
  intro beta hbeta beta' hbeta' hlt
  have hbeta_pi : beta < Real.pi := by
    have hacos : 0 < Real.arccos lam := Real.arccos_pos.2 hlam1
    linarith [hbeta.2]
  have hbeta'_pi : beta' < Real.pi := by
    have hacos : 0 < Real.arccos lam := Real.arccos_pos.2 hlam1
    linarith [hbeta'.2]
  have hcot : Real.cot beta' < Real.cot beta :=
    cot_strictAntiOn_Ioo_zero_pi (hbeta.1, hbeta_pi)
    (hbeta'.1, hbeta'_pi) hlt
  rw [typeBRadius_eq_add_sqrt_mul_cot hlam0 hlam1 hbeta,
    typeBRadius_eq_add_sqrt_mul_cot hlam0 hlam1 hbeta']
  nlinarith [sqrt_one_sub_sq_pos hlam0 hlam1]

/-- Equality of finite type-(B) radii forces equality of the source angle. -/
```

CMVBallCase.typeB_perimeter_factor_pos

Classification: main

Source: CMVBallCase.lean:623-643

Declaration SHA-256: ac194654106ebe6db799026b01d031f3446dcc24e3fc1b5dfd81512dcf10bc66

Mathematical statement. For $0 < \lambda < 1$ and $R > 0$, $Q_B(R) > 0$.

The factor has the form

$$Q_B(R) = S(\pi - \beta_\lambda(R)) > 0.$$

The complementary angle lies in $(0, \pi)$, so segment positivity proves the strict sign.

```
lemma typeB_perimeter_factor_pos {lam R : ℝ}
  (hlam0 : 0 < lam) (hlam1 : lam < 1) (hR : 0 < R) :
  0 < Real.pi - betaAtRadius lam R +
    Real.sin (betaAtRadius lam R) * Real.cos (betaAtRadius lam R) := by
  have hadm := betaAtRadius_admissible hlam0 hlam1 hR.le
  have hstrict : betaAtRadius lam R < Real.pi - Real.arccos lam := by
    by_contra hn
  have heq : betaAtRadius lam R = Real.pi - Real.arccos lam :=
    le_antisymm hadm.2 (le_of_not_gt hn)
  have hr := typeBRadius_betaAtRadius hlam0 hlam1 hR.le
  rw [heq, typeBRadius_endpoint_zero] at hr
  linarith
  have ht0 : 0 < Real.pi - betaAtRadius lam R := by
    have hacos : 0 < Real.arccos lam := Real.arccos_pos.2 hlam1
    linarith
  have htpi : Real.pi - betaAtRadius lam R < Real.pi := by
    linarith [hadm.1]
  have hseg := arcSegment_pos ht0 htpi
  rw [arcSegment, Real.sin_pi_sub, Real.cos_pi_sub] at hseg
  linarith
```

CMVBallCase.typeB_values_eq_singleton

Classification: main

Source: CMVBallCase.lean:1098-1110

Declaration SHA-256: 0aaa46dd0b8b451605797b9ef606b1e37cf80fd960f4abcc25d735f41934d431

Mathematical statement. If β is the unique admissible type-B angle of area v , its perimeter is the sole element of the value set.

Assume admissibility, $A_B(\beta) = v$, and that every admissible angle of that area equals β . Then

$$\mathcal{P}_B(v) = \{P_B(\beta)\}.$$

The lemma takes uniqueness as a hypothesis; the radius argument supplies it in the application.

```
lemma typeB_values_eq_singleton {lam v beta : ℝ}
  (hbeta : BAdmissible lam beta) (harea : typeBArea lam beta = v)
  (hunique : ∀ beta', BAdmissible lam beta' → typeBArea lam beta' = v → beta' = beta) :
  typeBPerimeterValuesAtArea lam v = {typeBPerimeter lam beta} := by
  ext p
  constructor
  · rintro (beta', hbeta', harea', rfl)
    simp [hunique beta' hbeta' harea']
  · intro hp
    have hp' : p = typeBPerimeter lam beta := by simpa using hp
    subst p
    exact typeBPerimeter_mem_values hbeta harea
```

CMVBallCase.typeCArea

Classification: main

Source: CMVBallDefinitions.lean:37-43

Declaration SHA-256: 1c838263bf547723f2086f0226358db55c1fd829ed4addfba436f74cda3734ab

Mathematical statement. The angular type-C weighted area is $(\pi - (1 - \lambda)(\beta - \sin\beta\cos\beta))/\tan^2\beta - (1 - \lambda)(\pi/2 - \beta - \sin\beta\cos\beta)$.

With $R = \cot\beta$, the formula is

$$\begin{aligned} A_C(\beta) &= R^2\pi - (1 - \lambda) \left[R^2 S(\beta) + S(\pi/2 - \beta) \right] \\ &= \frac{\pi - (1 - \lambda)(\beta - \sin\beta\cos\beta)}{\tan^2\beta} - (1 - \lambda)(\pi/2 - \beta - \sin\beta\cos\beta). \end{aligned}$$

The bracket is the overlap lens area, so the expression uses exactly the same density deficit as the geometric model.

```
def typeCArea (lam beta : ℝ) : ℝ :=
  (Real.pi - (1 - lam) * (beta - Real.sin beta * Real.cos beta)) /
    (Real.tan beta) ^ 2 -
    (1 - lam) *
      (Real.pi / 2 - beta - Real.sin beta * Real.cos beta)

end CMVBallCase
```

CMVBallCase.typeCAreaAtRadius

Classification: main

Source: CMVBallCase.lean:339-344

Declaration SHA-256: 348b519a27228df7e22a823dc6b7c50b1974e565424be0185003a97e07856f1d

Mathematical statement. The radius-coordinate type-C area is $R^2(\pi - (1 - \lambda)S(b(R))) - (1 - \lambda)S(\pi/2 - b(R))$.

The definition is

$$A_C(R) = R^2(\pi - (1 - \lambda)S(b(R))) - (1 - \lambda)S(\pi/2 - b(R)).$$

The overlap lens has area $R^2 S(b(R)) + S(\pi/2 - b(R))$; subtracting its density deficit from πR^2 yields the formula.

```
def typeCAreaAtRadius (lam R : ℝ) : ℝ :=
  R ^ 2 * (Real.pi - (1 - lam) *
    (bAtRadius R - Real.sin (bAtRadius R) * Real.cos (bAtRadius R))) -
    (1 - lam) * (Real.pi / 2 - bAtRadius R -
      Real.sin (bAtRadius R) * Real.cos (bAtRadius R))
```

CMVBallCase.typeCAreaAtRadius_agrees

Classification: main

Source: CMVBallCase.lean:367-377

Declaration SHA-256: ef9487e571bf7a8e9ecfdce2224335bbe39d0b9603b9c566eb6c213c2b58a4f8

Mathematical statement. For every real λ and $R > 0$, $A_C(R)$ equals the angular type-C area at $b(R)$.

The inverse-chart identity replaces $1/\tan^2 b(R)$ by R^2 , giving

$$A_C(R) = A_C(b(R)).$$

The equality is exact for all λ , although only $0 < \lambda < 1$ is used geometrically.

```

lemma typeCAreaAtRadius_agrees {lam R : ℝ} (hR : 0 < R) :
  typeCAreaAtRadius lam R = typeCArea lam (bAtRadius R) := by
  rw [typeCAreaAtRadius, typeCArea]
  have hradius := typeCRadius_bAtRadius hR
  rw [typeCRadius, one_div] at hradius
  have hden : (Real.tan (bAtRadius R) ^ 2)^-1 = R ^ 2 := by
    rw [← inv_pow, hradius]
  nth_rewrite 2 [div_eq_mul_inv]
  rw [hden]
  ring

```

CMVBallCase.typeCAreaAtRadius_continuousOn

Classification: main

Source: CMVBallCase.lean:703-707

Declaration SHA-256: c92a0518d3bc80f606df13b4b6f5c21e7bd9c2747515397578edebe4a4433736

Mathematical statement. For every real λ , A_C is continuous on $(0, \infty)$.

The derivative formula at each $R > 0$ implies continuity there. No claim about the value of the totalized expression at $R = 0$ is made.

```

lemma typeCAreaAtRadius_continuousOn {lam : ℝ} :
  ContinuousOn (typeCAreaAtRadius lam) (Set.Ioi 0) := by
  intro R hR
  exact (typeCAreaAtRadius_hasDerivAt hR).continuousAt.continuousWithinAt

```

CMVBallCase.typeCAreaAtRadius_deriv_pos

Classification: main

Source: CMVBallCase.lean:670-676

Declaration SHA-256: 5b0c9033c9a26a0f21e15311d7c9b43d1a45758e4340fc63a1e1040bf167f9d8

Mathematical statement. For $0 < \lambda < 1$ and $R > 0$, the derivative of A_C at R is positive.

The identity $A_C'(R) = 2RQ_C(R)$ and the factor positivity prove the result. It excludes multiple positive radii of the same area.

```

lemma typeCAreaAtRadius_deriv_pos {lam R : ℝ}
  (hlam0 : 0 < lam) (hlam1 : lam < 1) (hR : 0 < R) :
  0 < deriv (typeCAreaAtRadius lam) R := by
  rw [(typeCAreaAtRadius_hasDerivAt hR).deriv]
  have hq := typeC_perimeter_factor_pos hlam0 hlam1 hR
  positivity

```

CMVBallCase.typeCAreaAtRadius_hasDerivAt

Classification: main

Source: CMVBallCase.lean:572-609

Declaration SHA-256: b473b5e3212cdaca79ca31990e26c74cf5808e134b828057b642784ffa2747a8

Mathematical statement. For every real λ and $R > 0$, A_C has derivative $2RQ_C(R)$ at R .

The relation $\cos b(R) = R \sin b(R)$ cancels the derivatives of the two overlap segments. The remaining expression is

$$A_C'(R) = 2RQ_C(R) = RP_C'(R),$$

without a density restriction.

```

lemma typeCAreaAtRadius_hasDerivAt {lam R : ℝ} (hR : 0 < R) :
  HasDerivAt (typeCAreaAtRadius lam)
    (R * (2 * (Real.pi - (1 - lam) *
      (bAtRadius R - Real.sin (bAtRadius R) * Real.cos (bAtRadius R))))) R := by
  let db := -1 / (1 + R ^ 2)
  have hb : HasDerivAt bAtRadius db R := bAtRadius_hasDerivAt hR.ne'
  have hsB := arcSegment_hasDerivAt hb
  have hq := (hasDerivAt_const R Real.pi).sub
    (hsB.const_mul (1 - lam))
  have hr2 : HasDerivAt (fun x : ℝ => x ^ 2) (2 * R) R := by
    convert (hasDerivAt_id R).pow 2 using 1 <|>
    first | rfl | norm_num
  have hg := (hasDerivAt_const R (Real.pi / 2)).sub hb
  have hsG := arcSegment_hasDerivAt hg
  have hraw := (hr2.mul hq).sub (hsG.const_mul (1 - lam))
  have heq : typeCAreaAtRadius lam = [nhds R]
    (fun x => x ^ 2 * (Real.pi - (1 - lam) * arcSegment (bAtRadius x)) -
      (1 - lam) * arcSegment (Real.pi / 2 - bAtRadius x)) := by
    exact Filter.Eventually.of_forall fun x => by
      unfold typeCAreaAtRadius arcSegment
      simp only [Real.sin_pi_div_two_sub, Real.cos_pi_div_two_sub]
      ring
  have hfun := hraw.congr_of_eventuallyEq heq
  have hcos := cos_bAtRadius_eq_radius_mul_sin hR
  have hcoef : R * (2 * (Real.pi - (1 - lam) *
    (bAtRadius R - Real.sin (bAtRadius R) * Real.cos (bAtRadius R))))) =
    2 * R * (Real.pi - (1 - lam) * arcSegment (bAtRadius R)) +
    R ^ 2 * (0 - (1 - lam) *
      (2 * Real.sin (bAtRadius R) ^ 2 * db)) -
    (1 - lam) * (2 * Real.sin (Real.pi / 2 - bAtRadius R) ^ 2 *
      (0 - db)) := by
    rw [Real.sin_pi_div_two_sub]
    simp only [arcSegment]
    simp_rw [hcos]
    ring
  rw [hcoef]
  exact hfun

```

CMVBallCase.typeCAreaAtRadius_large_lower

Classification: main

Source: CMVBallCase.lean:833-865

Declaration SHA-256: b45b5794cbb45181f9541a2d6aaedfb79df716a6e045c02aa69ba8995a958442

Mathematical statement. For $0 < \lambda < 1$ and $R > 0$, $A_C(R) > \lambda \pi R^2 - \pi/2$.

Both overlap angles lie in $(0, \pi/2)$, with segment areas smaller than their angles. The coefficient of R^2 is greater than $\lambda\pi$, and the final subtracted term is less than $\pi/2$. Hence

$$A_C(R) > \lambda \pi R^2 - \frac{\pi}{2}.$$

```

lemma typeCAreaAtRadius_large_lower {lam R : ℝ}
  (hlam0 : 0 < lam) (hlam1 : lam < 1) (hR : 0 < R) :
  lam * Real.pi * R ^ 2 - Real.pi / 2 < typeCAreaAtRadius lam R := by
  rcases bAtRadius_admissible hR with (hb0, hb2)
  have hsegBpos : 0 < arcSegment (bAtRadius R) :=
    arcSegment_pos hb0 (by linarith [hb2, Real.pi_pos])
  have hsegBlt : arcSegment (bAtRadius R) < Real.pi / 2 :=
    (arcSegment_lt_self hb0 hb2).trans hb2
  have hg0 : 0 < Real.pi / 2 - bAtRadius R := by linarith
  have hg2 : Real.pi / 2 - bAtRadius R < Real.pi / 2 := by linarith
  have hsegGpos : 0 < arcSegment (Real.pi / 2 - bAtRadius R) :=
    arcSegment_pos hg0 (by linarith [hg2, Real.pi_pos])
  have hsegGlt : arcSegment (Real.pi / 2 - bAtRadius R) < Real.pi / 2 :=
    (arcSegment_lt_self hg0 hg2).trans hg2
  have hlam : 0 < 1 - lam := by linarith
  have hq : lam * Real.pi <

```

```

Real.pi - (1 - lam) * arcSegment (bAtRadius R) := by
  nlinarith [Real.pi_pos]
have hg : (1 - lam) * arcSegment (Real.pi / 2 - bAtRadius R) <
  Real.pi / 2 := by
  nlinarith [Real.pi_pos]
rw [show typeCAreaAtRadius lam R =
  R ^ 2 * (Real.pi - (1 - lam) * arcSegment (bAtRadius R)) -
  (1 - lam) * arcSegment (Real.pi / 2 - bAtRadius R) by
  unfold typeCAreaAtRadius arcSegment
  rw [Real.sin_pi_div_two_sub, Real.cos_pi_div_two_sub]
  ring]
have hp0 := mul_lt_mul_of_pos_left hq (sq_pos_of_pos hR)
have hp : lam * Real.pi * R ^ 2 <
  R ^ 2 * (Real.pi - (1 - lam) * arcSegment (bAtRadius R)) := by
  nlinarith
linarith

```

CMVBallCase.typeCAreaAtRadius_lt_full

Classification: main

Source: CMVBallCase.lean:810-832

Declaration SHA-256: a7ac56a00483dd17c810d49f7f3b39998e96e24cbcc05612f96aba66e39e49b6

Mathematical statement. For $0 < \lambda < 1$ and $R > 0$, $A_C(R) < \pi R^2$.

The weighted area subtracts $(1 - \lambda)$ times two strictly positive overlap-segment areas from the full disk area. Therefore

$$A_C(R) < \pi R^2.$$

At $R = \lambda$, this places $A_C(\lambda)$ below $\lambda\pi$.

```

lemma typeCAreaAtRadius_lt_full {lam R : ℝ}
  (hlam0 : 0 < lam) (hlam1 : lam < 1) (hR : 0 < R) :
  typeCAreaAtRadius lam R < Real.pi * R ^ 2 := by
  rcases bAtRadius_admissible hR with (hb0, hb2)
  have hsegB : 0 < arcSegment (bAtRadius R) :=
    arcSegment_pos hb0 (by linarith [hb2, Real.pi_pos])
  have hg0 : 0 < Real.pi / 2 - bAtRadius R := by linarith
  have hg2 : Real.pi / 2 - bAtRadius R < Real.pi / 2 := by linarith
  have hsegG : 0 < arcSegment (Real.pi / 2 - bAtRadius R) :=
    arcSegment_pos hg0 (by linarith [hg2, Real.pi_pos])
  have hlam : 0 < 1 - lam := by linarith
  rw [show typeCAreaAtRadius lam R =
    R ^ 2 * (Real.pi - (1 - lam) * arcSegment (bAtRadius R)) -
    (1 - lam) * arcSegment (Real.pi / 2 - bAtRadius R) by
    unfold typeCAreaAtRadius arcSegment
    rw [Real.sin_pi_div_two_sub, Real.cos_pi_div_two_sub]
    ring]
  have hp : 0 < R ^ 2 * ((1 - lam) * arcSegment (bAtRadius R)) :=
    mul_pos (sq_pos_of_pos hR) (mul_pos hlam hsegB)
  have hg : 0 < (1 - lam) * arcSegment (Real.pi / 2 - bAtRadius R) :=
    mul_pos hlam hsegG
  nlinarith

```

CMVBallCase.typeCAreaAtRadius_strictMonoOn

Classification: main

Source: CMVBallCase.lean:722-730

Declaration SHA-256: bbc5de3a1bc43d4360f2a79a6c274beccf88985cbfcb7b993a4fe30a532f86a2

Mathematical statement. For $0 < \lambda < 1$, A_C is strictly increasing on $(0, \infty)$.

The positive derivative and continuity imply $A_C(R) < A_C(T)$ whenever $0 < R < T$. The result is restricted to the genuine positive-radius chart.

```

lemma typeCAreaAtRadius_strictMonoOn {lam : ℝ}
  (hlam0 : 0 < lam) (hlam1 : lam < 1) :
  StrictMonoOn (typeCAreaAtRadius lam) (Set.Ioi 0) := by
  apply strictMonoOn_of_deriv_pos (convex_Ioi 0)
  typeCAreaAtRadius_continuousOn
  intro R hR
  rw [interior_Ioi] at hR
  exact typeCAreaAtRadius_deriv_pos hlam0 hlam1 hR

```

CMVBallCase.typeCPerimeter

Classification: main

Source: CMVBallDefinitions.lean:33-36

Declaration SHA-256: dae34722c1182356b6a588c3b1c13561ad7e927882e5ef9397cd8e01e6fe4454

Mathematical statement. The angular type-C weighted perimeter is $2(\pi - (1 - \lambda)\beta)/\tan\beta$.

With $R = 1/\tan\beta$, subtract the density deficit $(1 - \lambda)2R\beta$ of the inner arc from the full circumference $2\pi R$. The source formula is

$$P_C(\beta) = \frac{2(\pi - (1 - \lambda)\beta)}{\tan\beta}.$$

```

def typeCPerimeter (lam beta : ℝ) : ℝ :=
  2 * (Real.pi - (1 - lam) * beta) / Real.tan beta

/-- CMV's weighted area formula for a type-(C) candidate. -/

```

CMVBallCase.typeCPerimeterAtArea

Classification: main

Source: CMVBallCase.lean:1085-1087

Declaration SHA-256: 2e3dfe31281e09c4e4e68ab71c76cfa596f35e20fe535735ebbe9fcbb7c59fc8

Mathematical statement. The equal-area type-C profile is the real infimum of its all-candidate perimeter set.

The definition is

$$I_C(v) = \inf \mathcal{P}_C(v).$$

The proof identifies this infimum with an actual candidate perimeter throughout the theorem's area domain, rather than relying on a totalized default.

```

def typeCPerimeterAtArea (lam v : ℝ) : ℝ :=
  sInf (typeCPerimeterValuesAtArea lam v)

```

CMVBallCase.typeCPerimeterAtArea_eq

Classification: main

Source: CMVBallCase.lean:1131-1137

Declaration SHA-256: 910202180ff5ab8baae5e253008da92ce732b62fac764c99dfe1e8b67dffc4a3

Mathematical statement. If b is the unique admissible type-C angle of area v , then $I_C(v) = P_C(b)$.

The specified candidate and uniqueness hypotheses identify $\mathcal{P}_C(v)$ with its perimeter singleton. Taking the infimum gives

$$I_C(v) = P_C(b),$$

without a branch choice.

```

lemma typeCPerimeterAtArea_eq {lam v beta : ℝ}
  (hbeta : CAdmissible beta) (harea : typeCArea lam beta = v)
  (hunique : ∀ beta', CAdmissible beta' → typeCArea lam beta' = v → beta' = beta) :
  typeCPerimeterAtArea lam v = typeCPerimeter lam beta := by
  rw [typeCPerimeterAtArea, typeC_values_eq_singleton hbeta harea hunique]
  exact csInf_singleton _

```

CMVBallCase.typeCPerimeterAtArea_eq_radius

Classification: main

Source: CMVBallCase.lean:1183-1200

Declaration SHA-256: 67e8363dd82abecb62d39098aa381615d4943a0c48b8f9f5ecc2be957cbd9e7a

Mathematical statement. For $0 < \lambda < 1$, $R > 0$, and $A_C(R) = v$, the all-candidate profile satisfies $I_C(v) = P_C(R)$.

Every admissible type-C angle has positive radius. Strict area monotonicity and the inverse chart make the candidate of area v unique, so its perimeter is the infimum over all candidates:

$$I_C(v) = P_C(R).$$

```

lemma typeCPerimeterAtArea_eq_radius {lam v R : ℝ}
  (hlam0 : 0 < lam) (hlam1 : lam < 1)
  (hR : 0 < R) (harea : typeCAreaAtRadius lam R = v) :
  typeCPerimeterAtArea lam v = typeCPerimeterAtRadius lam R := by
  rw [typeCPerimeterAtArea_eq (bAtRadius_admissible hR)
    ((typeCAreaAtRadius_agrees hR).symm.trans harea)]
  · exact (typeCPerimeterAtRadius_agrees hR).symm
  · intro beta' hbeta' harea'
    apply typeCRadius_injective_on_admissible hbeta'
      (bAtRadius_admissible hR)
    have hR' := typeCRadius_pos hbeta'
    have hREq : typeCRadius beta' = R := by
      apply (typeCAreaAtRadius_strictMonoOn hlam0 hlam1).injOn hR' hR
      rw [typeCAreaAtRadius_agrees hR', bAtRadius_typeCRadius hbeta',
        harea', harea]
    rw [typeCRadius_bAtRadius hR]
    exact hREq

```

CMVBallCase.typeCPerimeterAtArea_lt_of_commonPerimeterAreaGap_neg

Classification: main

Source: CMVBallCase.lean:2378-2422

Declaration SHA-256: 6ed9af03cd6961ea327cfb3f5237fc3e86432b34c2cd4ca2282daeb7016ccc0a

Mathematical statement. If $0 < \lambda < 1$, $v > \lambda\pi$, and $G(I_B(v)) < 0$, then $I_C(v) < I_B(v)$.

At $p = I_B(v)$, negativity says

$$A_C(r_C(p)) > v.$$

The unique type-C radius of area v is strictly smaller, so its strictly increasing perimeter gives

$$I_C(v) < p = I_B(v).$$

```

lemma typeCPerimeterAtArea_lt_of_commonPerimeterAreaGap_neg {lam v : ℝ}
  (hlam : AdmissibleDensity lam) (hv : lam * Real.pi < v)
  (hgap : commonPerimeterAreaGap hlam
    {typeBPerimeterAtArea lam v,
      typeBPerimeterAtArea_gt_commonEndpoint hlam.1 hlam.2 hv} < 0) :
  typeCPerimeterAtArea lam v < typeBPerimeterAtArea lam v := by
  rcases exists_typeBRadius_at_area hlam.1 hlam.2 hv with
    ⟨RB, hRB, hareaB⟩
  rcases exists_typeCRadius_at_area hlam.1 hlam.2 hv with
    ⟨SC, hSC, hareaC⟩

```

```

change 0 < RB at hRB
change 0 < SC at hSC
let p : CommonPerimeter lam :=
  ⟨typeBPerimeterAtArea lam v,
    typeBPerimeterAtArea_gt_commonEndpoint hlam.1 hlam.2 hv⟩
have hprofileB :
  typeBPerimeterAtArea lam v = typeBPerimeterAtRadius lam RB :=
  typeBPerimeterAtArea_eq_radius hlam.1 hlam.2 hRB hareaB
have hprofileC :
  typeCPerimeterAtArea lam v = typeCPerimeterAtRadius lam SC :=
  typeCPerimeterAtArea_eq_radius hlam.1 hlam.2 hSC hareaC
have heqB : (typeBRadiusAtPerimeter hlam p).1 = RB :=
  typeBRadiusAtPerimeter_unique hlam p hRB hprofileB.symm
have harea_lt :
  v < typeCAreaAtRadius lam (typeCRadiusAtPerimeter hlam p) := by
  change commonPerimeterAreaGap hlam p < 0 at hgap
  unfold commonPerimeterAreaGap at hgap
  rw [heqB, hareaB] at hgap
  linarith
have hSC_lt : SC < (typeCRadiusAtPerimeter hlam p).1 := by
  by_contra hnot
  have hle : (typeCRadiusAtPerimeter hlam p).1 ≤ SC := le_of_not_gt hnot
  have harea_le :=
    (typeCAreaAtRadius_strictMonoOn hlam.1 hlam.2).monotoneOn
    (typeCRadiusAtPerimeter_pos hlam p) hSC hle
  rw [hareaC] at harea_le
  exact (not_lt_of_ge harea_le) harea_lt
have hperim_lt := typeCPerimeterAtRadius_strictMonoOn hlam.1 hlam.2
  hSC (typeCRadiusAtPerimeter_pos hlam p) hSC_lt
rw [← hprofileC, typeCPerimeterAt_typeCRadiusAtPerimeter hlam p] at hperim_lt
exact hperim_lt

/-- Once the orthogonal-ball profile is no worse than the equal-area type-(B)
profile, the concrete common-perimeter gap forces strict dominance at every
larger weighted area. -/

```

CMVBallCase.typeCPerimeterAtRadius

Classification: main

Source: CMVBallCase.lean:335-338

Declaration SHA-256: 5bfe6bbb6f40601afee63cea3bd361fb5c2ec4e0cae513f610cfd4255fe4ae2f

Mathematical statement. The radius-coordinate type-C perimeter is $2R(\pi - (1 - \lambda)b(R))$.

The definition is

$$P_C(R) = 2R(\pi - (1 - \lambda)b(R)).$$

Starting with circumference $2\pi R$, the expression subtracts $(1 - \lambda)$ times the inner arc length $2Rb(R)$.

```

def typeCPerimeterAtRadius (lam R : ℝ) : ℝ :=
  2 * R * (Real.pi - (1 - lam) * bAtRadius R)

/-- Type-(C) weighted area in its positive-radius chart. -/

```

CMVBallCase.typeCPerimeterAtRadius_agrees

Classification: main

Source: CMVBallCase.lean:358-366

Declaration SHA-256: 07f1596dce2184a5c103c4a465e9a7f508c9fbe84d173c9f12c79dc46e54a1a2

Mathematical statement. For every real λ and $R > 0$, $P_C(R)$ equals the angular type-C perimeter at $b(R)$.

Using $1/\tan b(R) = R$ gives

$$P_C(R) = P_C(b(R)).$$

The algebraic equality does not require an admissible density.

```
lemma typeCPerimeterAtRadius_agrees {lam R : ℝ} (hR : 0 < R) :
  typeCPerimeterAtRadius lam R =
    typeCPerimeter lam (bAtRadius R) := by
  rw [typeCPerimeterAtRadius, typeCPerimeter, div_eq_mul_inv]
  have hradius := typeCRadius_bAtRadius hR
  rw [typeCRadius, one_div] at hradius
  rw [hradius]
  ring
```

CMVBallCase.typeCPerimeterAtRadius_continuousOn

Classification: main

Source: CMVBallCase.lean:708-712

Declaration SHA-256: 8ba982df91d14a4376354beb85fd26540d8e9d0d83b34015727e5932cb96b536

Mathematical statement. For every real λ , P_C is continuous on $(0, \infty)$.

Differentiability at positive radii gives continuity on the geometric radius domain. This is enough for intermediate-value arguments on compact positive-radius intervals.

```
lemma typeCPerimeterAtRadius_continuousOn {lam : ℝ} :
  ContinuousOn (typeCPerimeterAtRadius lam) (Set.Ioi 0) := by
  intro R hR
  exact (typeCPerimeterAtRadius_hasDerivAt hR).continuousAt.continuousWithinAt
```

CMVBallCase.typeCPerimeterAtRadius_deriv_pos

Classification: main

Source: CMVBallCase.lean:685-692

Declaration SHA-256: 0a0f4d788d01de74c363719e0d086cb74d4be34b638c450f00b59a31d9f27b99

Mathematical statement. For $0 < \lambda < 1$, every positive R satisfies $P_C'(R) > 0$.

The derivative equals $2Q_C(R) > 0$. This proves perimeter monotonicity and permits inversion at each genuine common perimeter.

```
lemma typeCPerimeterAtRadius_deriv_pos {lam R : ℝ}
  (hlam0 : 0 < lam) (hlam1 : lam < 1) :
  0 < R → 0 < deriv (typeCPerimeterAtRadius lam) R := by
  intro hR
  rw [(typeCPerimeterAtRadius_hasDerivAt hR).deriv]
  have hq := typeC_perimeter_factor_pos hlam0 hlam1 hR
  positivity
```

CMVBallCase.typeCPerimeterAtRadius_gt_pi_mul_radius

Classification: main

Source: CMVBallCase.lean:1841-1852

Declaration SHA-256: 096d139a89edea080509b072e5206625c2521a1d0213be4b9a082a2bd82369f3

Mathematical statement. If $0 < \lambda < 1$ and $R > 0$, then $P_C(R) > \pi R$.

Since $(1 - \lambda)b(R) < \pi/2$, the factor $\pi - (1 - \lambda)b(R)$ is greater than $\pi/2$. Multiplication by $2R > 0$ gives

$$P_C(R) > \pi R,$$

proving unbounded perimeter growth.

```
lemma typeCPerimeterAtRadius_gt_pi_mul_radius {lam R : ℝ}
  (hlam0 : 0 < lam) (hlam1 : lam < 1) (hR : 0 < R) :
  Real.pi * R < typeCPerimeterAtRadius lam R := by
  rw [typeCPerimeterAtRadius]
  have hb := bAtRadius_admissible hR
  have hfactor : Real.pi / 2 < Real.pi - (1 - lam) * bAtRadius R := by
    have hweight : (1 - lam) * bAtRadius R < Real.pi / 2 := by
      nlinarith [hb.2, hlam0, hlam1, Real.pi_pos]
    linarith
  nlinarith

/-- A type-(B) radius exists uniquely at every genuine common perimeter. -/
```

CMVBallCase.typeCPerimeterAtRadius_hasDerivAt

Classification: main

Source: CMVBallCase.lean:465-487

Declaration SHA-256: f5d2c7b26d784940465bfecd68ce738beed977fd9837beb85c6df684f2f89412

Mathematical statement. For every real λ and $R > 0$, P_C has derivative $2Q_C(R)$ at R .

The derivative identity is

$$P_C'(R) = 2Q_C(R) = 2\left[\pi - (1 - \lambda)(b(R) - \sin b(R)\cos b(R))\right].$$

It is a derivative formula for arbitrary density; positivity is asserted separately under $0 < \lambda < 1$.

```
lemma typeCPerimeterAtRadius_hasDerivAt {lam R : ℝ} (hR : 0 < R) :
  HasDerivAt (typeCPerimeterAtRadius lam)
    (2 * (Real.pi - (1 - lam) *
      (bAtRadius R - Real.sin (bAtRadius R) * Real.cos (bAtRadius R)))) R := by
  let db := -1 / (1 + R ^ 2)
  have hb : HasDerivAt bAtRadius db R := bAtRadius_hasDerivAt hR.ne'
  have hraw := ((hasDerivAt_id R).mul
    ((hasDerivAt_const R Real.pi).sub
      (hb.const_mul (1 - lam))))).const_mul 2
  have hfun : HasDerivAt (typeCPerimeterAtRadius lam)
    (2 * (1 * (Real.pi - (1 - lam) * bAtRadius R) +
      R * (0 - (1 - lam) * db))) R := by
    apply hraw.congr_of_eventuallyEq
    exact Filter.Eventually.of_forall fun x => by
      simp only [typeCPerimeterAtRadius, id_eq, Pi.mul_apply, Pi.sub_apply]
      ring
  convert hfun using 1
  dsimp [db]
  rw [sin_bAtRadius_mul_cos hR.ne']
  field_simp
  ring

/-- Twice the circular-segment area with unit radius. -/
```

CMVBallCase.typeCPerimeterAtRadius_hasStrictDerivAt

Classification: main

Source: CMVBallCase.lean:2090-2107

Declaration SHA-256: 3e2b1b6740838724efc3c00deba8f2d16532a3855f0a5cf96b2667b1a8855fae

Mathematical statement. For $0 < \lambda < 1$ and $R > 0$, P_C has strict derivative $2Q_C(R)$ at R .

The derivative formula is continuous in a neighborhood of every positive radius. Therefore the two-point derivative limit holds:

$$\lim_{\substack{x, y \rightarrow R \\ x \neq y}} \frac{P_C(x) - P_C(y)}{x - y} = 2Q_C(R).$$

This provides the strict differentiability required by the inverse function theorem.

```
lemma typeCPerimeterAtRadius_hasStrictDerivAt {lam R : ℝ}
  (hlam0 : 0 < lam) (hlam1 : lam < 1) (hR : 0 < R) :
  HasStrictDerivAt (typeCPerimeterAtRadius lam)
    (2 * (Real.pi - (1 - lam) *
      (bAtRadius R - Real.sin (bAtRadius R) * Real.cos (bAtRadius R)))) R := by
let d : ℝ → ℝ := fun x =>
  2 * (Real.pi - (1 - lam) *
    (bAtRadius x - Real.sin (bAtRadius x) * Real.cos (bAtRadius x)))
have hstrict : HasStrictDerivAt (typeCPerimeterAtRadius lam) (d R) R := by
  apply hasStrictDerivAt_of_hasDerivAt_of_continuousAt
  · filter_upwards [eventually_gt_nhds hR] with x hx
    exact typeCPerimeterAtRadius_hasDerivAt hx
  · have hb : ContinuousAt bAtRadius R :=
      (bAtRadius_hasDerivAt hR.ne').continuousAt
    fun_prop
  exact hstrict

/-- The concrete B-minus-C area gap at a genuine common perimeter. -/
```

CMVBallCase.typeCPerimeterAtRadius_strictMonoOn

Classification: main

Source: CMVBallCase.lean:740-748

Declaration SHA-256: ebda1658a94a124439fef17a703088753a165f7a75d443b97607309192ac577f

Mathematical statement. For $0 < \lambda < 1$, P_C is strictly increasing on $(0, \infty)$.

The mean value theorem applied between any two positive radii gives strict increase. This transports perimeter inequalities to radius inequalities and conversely.

```
lemma typeCPerimeterAtRadius_strictMonoOn {lam : ℝ}
  (hlam0 : 0 < lam) (hlam1 : lam < 1) :
  StrictMonoOn (typeCPerimeterAtRadius lam) (Set.Ioi 0) := by
  apply strictMonoOn_of_deriv_pos (convex_Ioi 0)
  typeCPerimeterAtRadius_continuousOn
  intro R hR
  rw [interior_Ioi] at hR
  exact typeCPerimeterAtRadius_deriv_pos hlam0 hlam1 hR
```

CMVBallCase.typeCPerimeterValuesAtArea

Classification: main

Source: CMVBallCase.lean:1074-1079

Declaration SHA-256: cb485477b22802c53c6af448371d844cdb35a1ad16956ed8dcc86d1d34b9f786

Mathematical statement. This is the set of all type-C perimeter values from admissible angles of area exactly v .

Precisely,

$$\mathcal{P}_C(v) = \{p : \exists b, 0 < b < \pi/2, A_C(b) = v, P_C(b) = p\}.$$

Every angle in the complete finite domain is quantified.

```
def typeCPerimeterValuesAtArea (lam v : ℝ) : Set ℝ :=
  { p | ∃ beta, CAdmissible beta ∧ typeCArea lam beta = v ∧
    typeCPerimeter lam beta = p }
```

```
/-- The equal-area type-(B) profile. On its geometric range the preceding
value set is proved to be a singleton, so this infimum is that unique value. -/
```

CMVBallCase.typeCRadius

Classification: main

Source: CMVBallDefinitions.lean:30-32

Declaration SHA-256: a418ec9c9727a88b5945a9e22b827624dc661a72d7ca8532d18c95ff79188dd8

Mathematical statement. The angular type-C radius is $1/\tan\beta$.

Orthogonality gives

$$R = \frac{1}{\tan\beta} = \cot\beta \quad (0 < \beta < \pi/2).$$

The total real formula is not interpreted as a finite geometric radius at either excluded endpoint.

```
def typeCRadius (beta : ℝ) : ℝ := 1 / Real.tan beta

/-- CMV's weighted perimeter formula for a type-(C) candidate. -/
```

CMVBallCase.typeCRadiusAtPerimeter

Classification: main

Source: CMVBallCase.lean:1931-1936

Declaration SHA-256: a58a803701a0a2d5cf3a9913ddb7c5b645d75bedddb7e49e42f7c5d55ebc49a7

Mathematical statement. For an admissible density and $p > 2\lambda\pi$, $r_C(p)$ is the unique positive radius with $P_C(r_C(p)) = p$.

The definition selects the uniquely characterized positive solution and carries its defining equation:

$$r_C(p) > 0, \quad P_C(r_C(p)) = p.$$

The use of choice here does not choose among competing geometric branches, since uniqueness has already been proved.

```
def typeCRadiusAtPerimeter {lam : ℝ} (hlam : AdmissibleDensity lam)
  (p : CommonPerimeter lam) :
  {R : ℝ // 0 < R ∧ typeCPerimeterAtRadius lam R = p.1} :=
  ⟨(existsUnique_typeCRadius_at_perimeter hlam.1 hlam.2 p.2).choose,
    (existsUnique_typeCRadius_at_perimeter hlam.1 hlam.2 p.2).choose_spec.1⟩
```

CMVBallCase.typeCRadius_bAtRadius

Classification: main

Source: CMVBallCase.lean:290-295

Declaration SHA-256: 6e7442e12b01ef5ceace4300d1d77efe39044855122de67c6d01f583bea0c6a8

Mathematical statement. For $R > 0$, the type-C radius of $b(R)$ equals R .

The identity $\tan(\arctan(1/R)) = 1/R$, followed by reciprocal cancellation, proves

$$\cot b(R) = R.$$

Positivity excludes division at zero.

```
lemma typeCRadius_bAtRadius {R : ℝ} (hR : 0 < R) :
  typeCRadius (bAtRadius R) = R := by
  rw [typeCRadius, bAtRadius, Real.tan_arctan]
  field_simp [hR.ne']
```

```
-- The C inverse is a genuine inverse on its complete finite angular domain. -/
```

CMVBallCase.typeCRadius_injective_on_admissible

Classification: main

Source: CMVBallCase.lean:48-52

Declaration SHA-256: c8f101f2d5546c0e443cf5cbb59e42ff32eecef077bdf0c84c0da4896e2b3865

Mathematical statement. Two admissible type-C angles with equal radii are equal.

The restriction $\beta \mapsto \cot\beta$ to $(0, \pi/2)$ is injective by strict decrease. Thus no second angular branch can have the same positive radius.

```
lemma typeCRadius_injective_on_admissible :
  Set.InjOn typeCRadius {beta | CAdmissible beta} := by
  change Set.InjOn typeCRadius (Set.Ioo 0 (Real.pi / 2))
  exact typeCRadius_strictAntiOn.injOn
/-- Positivity of the Snell-law square-root coefficient. -/
```

CMVBallCase.typeCRadius_pos

Classification: main

Source: CMVBallCase.lean:28-35

Declaration SHA-256: 1a7173c1755709ae7faf95a9a6109b9e8aa56a4449eb1eeb91c7af81f092911c

Mathematical statement. Every admissible type-C angle has positive radius.

For $0 < \beta < \pi/2$, $\tan\beta > 0$, hence $1/\tan\beta > 0$. This places every angular candidate in the positive radius chart.

```
lemma typeCRadius_pos {beta : ℝ} (hbeta : CAdmissible beta) :
  0 < typeCRadius beta := by
  rw [typeCRadius]
  exact one_div_pos.mpr
    (Real.tan_pos_of_pos_of_lt_pi_div_two hbeta.1 hbeta.2)

/-- The type-(C) angle has no hidden radius branch: its radius is strictly
decreasing throughout the complete admissible open interval. -/
```

CMVBallCase.typeCRadius_strictAntiOn

Classification: main

Source: CMVBallCase.lean:36-47

Declaration SHA-256: 5aff3dac04279f6fbf34d1285134552fc7dd876dc06d4b5558e36e5ba717c3c1

Mathematical statement. The type-C radius is strictly decreasing on $(0, \pi/2)$.

If $0 < \beta < \beta' < \pi/2$, then

$$0 < \frac{1}{\tan\beta'} < \frac{1}{\tan\beta}.$$

Strict increase of tangent and positivity justify taking reciprocals.

```
lemma typeCRadius_strictAntiOn :
  StrictAntiOn typeCRadius (Set.Ioo 0 (Real.pi / 2)) := by
  intro beta hbeta beta' hbeta' hlt
  rw [typeCRadius]
  exact one_div_lt_one_div_of_lt
    (Real.tan_pos_of_pos_of_lt_pi_div_two hbeta.1 hbeta.2)
    (Real.strictMonoOn_tan
```

```

    (by linarith [hbeta.1, Real.pi_pos], hbeta.2)
    (by linarith [hbeta'.1, Real.pi_pos], hbeta'.2) hlt)

/-- Equality of admissible type-(C) radii forces equality of their angle
parameters; this is the branch-uniqueness fact used by radius parametrizations. -/

```

CMVBallCase.typeC_dominance_persists

Classification: main

Source: CMVBallCase.lean:2423-2448

Declaration SHA-256: f5ada2cd0133c47ab6c9f8734b2bc2a998e82cc392fe14d449f57858a255b00e

Mathematical statement. For all real λ, v_0, v with $0 < \lambda < 1$ and $\lambda\pi < v_0 < v$, $I_C(v_0) \leq I_B(v_0)$ implies $I_C(v) < I_B(v)$.

Set $p_0 = I_B(v_0)$ and $p = I_B(v)$. Strict profile increase and initial dominance give

$$p_0 < p, \quad G(p) < G(p_0) \leq 0.$$

The final bridge yields

$$I_C(v) < I_B(v).$$

This is the exact primary root and has no hypothesis asserting geometric classification or prior strict dominance.

```

theorem typeC_dominance_persists :
  ∀ {lam v₀ v : ℝ},
    0 < lam → lam < 1 → lam * Real.pi < v₀ → v₀ < v →
      typeCPerimeterAtArea lam v₀ ≤ typeBPerimeterAtArea lam v₀ →
      typeCPerimeterAtArea lam v < typeBPerimeterAtArea lam v := by
  intro lam v₀ v hlam0 hlam1 hv₀ hv hinitial
  let hlam : AdmissibleDensity lam := ⟨hlam0, hlam1⟩
  have hv' : lam * Real.pi < v := hv₀.trans hv
  let p₀ : CommonPerimeter lam :=
    ⟨typeBPerimeterAtArea lam v₀,
     typeBPerimeterAtArea_gt_commonEndpoint hlam0 hlam1 hv₀⟩
  let p : CommonPerimeter lam :=
    ⟨typeBPerimeterAtArea lam v,
     typeBPerimeterAtArea_gt_commonEndpoint hlam0 hlam1 hv'⟩
  have hp : p₀ < p := by
    exact typeBPerimeterAtArea_strictMonoOn hlam0 hlam1 hv₀ hv' hv
  have hgap₀ : commonPerimeterAreaGap hlam p₀ ≤ 0 := by
    exact commonPerimeterAreaGap_at_typeBProfile_nonpos hlam hv₀ hinitial
  have hgap_lt : commonPerimeterAreaGap hlam p <
    commonPerimeterAreaGap hlam p₀ :=
    commonPerimeterAreaGap_strictAnti hlam hp
  have hgap : commonPerimeterAreaGap hlam p < 0 :=
    lt_of_lt_of_le hgap_lt hgap₀
  exact typeCPerimeterAtArea_lt_of_commonPerimeterAreaGap_neg hlam hv' hgap
end CMVBallCase

```

CMVBallCase.typeC_perimeter_factor_pos

Classification: main

Source: CMVBallCase.lean:644-662

Declaration SHA-256: c725d934df891e67e5699ac456ab0db215e125103ab9c675b3e231a6c66930e3

Mathematical statement. For $0 < \lambda < 1$ and $R > 0$, $Q_C(R) > 0$.

Since $0 < S(b(R)) < b(R) < \pi/2$ and $0 < 1 - \lambda < 1$, the factor satisfies

$$Q_C(R) = \pi - (1 - \lambda)S(b(R)) > 0.$$

```

lemma typeC_perimeter_factor_pos {lam R : ℝ}
  (hlam0 : 0 < lam) (hlam1 : lam < 1) (hR : 0 < R) :
  0 < Real.pi - (1 - lam) *
    (bAtRadius R - Real.sin (bAtRadius R) * Real.cos (bAtRadius R)) := by
have hb := bAtRadius_admissible hR
have hs : 0 < Real.sin (bAtRadius R) :=
  Real.sin_pos_of_pos_of_lt_pi hb.1 (by linarith [hb.2, Real.pi_pos])
have hc : 0 < Real.cos (bAtRadius R) :=
  Real.cos_pos_of_mem_Ioo (by linarith [hb.1, Real.pi_pos], hb.2)
have hsegl : arcSegment (bAtRadius R) < bAtRadius R := by
  unfold arcSegment
  nlinarith
have hseglpos : 0 < arcSegment (bAtRadius R) :=
  arcSegment_pos hb.1 (by linarith [hb.2, Real.pi_pos])
have hm : (1 - lam) * arcSegment (bAtRadius R) <
  arcSegment (bAtRadius R) := by nlinarith
dsimp [arcSegment] at hsegl hm
linarith [hb.2, Real.pi_pos]

```

CMVBallCase.typeC_values_eq_singleton

Classification: main

Source: CMVBallCase.lean:1111-1123

Declaration SHA-256: afcbdbc6bce67fe83c6357fe4f5677ffc4fb903b985aa4822c1eef450d8ed5ce

Mathematical statement. If b is the unique admissible type-C angle of area v , its perimeter is the sole element of the value set.

Given admissibility, the area equation, and uniqueness among all admissible angles of that area,

$$\mathcal{P}_C(v) = \{P_C(b)\}.$$

This is a conditional set identity, not an independent existence theorem.

```

lemma typeC_values_eq_singleton {lam v beta : ℝ}
  (hbeta : CAdmissible beta) (harea : typeCArea lam beta = v)
  (hunique : ∀ beta', CAdmissible beta' → typeCArea lam beta' = v → beta' = beta) :
  typeCPerimeterValuesAtArea lam v = {typeCPerimeter lam beta} := by
ext p
constructor
· rintro (beta', hbeta', harea', rfl)
  simp [hunique beta' hbeta' harea']
· intro hp
  have hp' : p = typeCPerimeter lam beta := by simp using hp
  subst p
  exact typeCPerimeter_mem_values hbeta harea

```

CMVBallCase.arcSegment_nonneg

Classification: supplement

Source: CMVBallCase.lean:776-783

Declaration SHA-256: 984106fd2f39d9277591bc5e64a6ef86b070468729c98368039d0fdaf7f7e0f5

Mathematical statement. $S(x) \geq 0$ for $0 \leq x \leq \pi$.

Interior positivity combines with $S(0) = 0$ and $S(\pi) = \pi$. This closed-interval version allows the type-B area estimate to include its segment endpoint.

```

lemma arcSegment_nonneg {x : ℝ} (hx0 : 0 ≤ x) (hxpi : x ≤ Real.pi) :
  0 ≤ arcSegment x := by
  rcases eq_or_lt_of_le hx0 with rfl | hx0'
  · simp [arcSegment]
  rcases eq_or_lt_of_le hxpi with rfl | hxpi'

```

```
· simp [arcSegment, Real.sin_pi, Real.pi_pos.le]
exact (arcSegment_pos hx0' hxpi').le
```

CMVBallCase.arctan_neg_lam_div_sqrt_eq_arccos_sub_pi_div_two

Classification: supplement

Source: CMVBallCase.lean:163-182

Declaration SHA-256: ae6498b57a7901ab66208434683619d4fcd6fd2d5539929922be1071e5b55472

Mathematical statement. For $0 < \lambda < 1$, $\arctan(-\lambda/\sqrt{1-\lambda^2}) = \arccos\lambda - \pi/2$.

Both sides of

$$\arctan \frac{-\lambda}{\sqrt{1-\lambda^2}} = \arccos\lambda - \frac{\pi}{2}$$

lie in the principal arctangent interval and have tangent $-\lambda/s$. The identity identifies the inverse chart's value at radius zero without a branch ambiguity.

```
lemma arctan_neg_lam_div_sqrt_eq_arccos_sub_pi_div_two {lam : ℝ}
  (hlam0 : 0 < lam) (hlam1 : lam < 1) :
  Real.arctan (-lam / Real.sqrt (1 - lam ^ 2)) =
  Real.arccos lam - Real.pi / 2 := by
  have hsqrt : 0 < Real.sqrt (1 - lam ^ 2) := sqrt_one_sub_sq_pos hlam0 hlam1
  have hangle_lower : -(Real.pi / 2) < Real.arccos lam - Real.pi / 2 := by
    have hacos : 0 < Real.arccos lam := Real.arccos_pos.2 hlam1
    linarith
  have hangle_upper : Real.arccos lam - Real.pi / 2 < Real.pi / 2 := by
    have hacos : Real.arccos lam < Real.pi / 2 :=
      Real.arccos_lt_pi_div_two.2 hlam0
    linarith
  have htan : Real.tan (Real.arccos lam - Real.pi / 2) =
    -lam / Real.sqrt (1 - lam ^ 2) := by
    rw [show Real.arccos lam - Real.pi / 2 =
      -(Real.pi / 2 - Real.arccos lam) by ring, Real.tan_neg,
      Real.tan_pi_div_two_sub, Real.tan_arccos, inv_div] <|>
    field_simp [hsqrt.ne'] <|> ring
  rw [← htan, Real.arctan_tan hangle_lower hangle_upper]
```

CMVBallCase.bAtRadius_hasDerivAt

Classification: supplement

Source: CMVBallCase.lean:393-402

Declaration SHA-256: 1653427eb62cf98e802e2428e9d6de180448223da78362c5f886faa270cd5aad

Mathematical statement. At each nonzero real R , b has derivative $-1/(1 + R^2)$.

The chain rule for $\arctan(1/R)$ gives

$$b'(R) = -\frac{1}{1 + R^2}.$$

The hypothesis is $R \neq 0$, not merely positive radius, because the total expression is also differentiable at negative radii.

```
lemma bAtRadius_hasDerivAt {R : ℝ} (hR : R ≠ 0) :
  HasDerivAt bAtRadius (-1 / (1 + R ^ 2)) R := by
  have hi : HasDerivAt (fun x : ℝ => 1 / x) (-1 / R ^ 2) R := by
    convert (hasDerivAt_const R 1).div (hasDerivAt_id R) hR using 1 <|>
    first | rfl | simp only [id_eq, zero_mul, one_mul, zero_sub]
  have ha := hi.arctan
  unfold bAtRadius
  convert ha using 1 <|>
  first | rfl | (field_simp [hR] <|> ring)
```

CMVBallCase.betaAtRadius_density_hasDerivAt

Classification: supplement

Source: CMVBallCase.lean:1437-1454

Declaration SHA-256: 81dbbf3b42fcfb5c1dad2e201bd766325c3f6f4bbc9066d7c22e79ab478c30f

Mathematical statement. For $0 < \lambda < 1$ and every real R , the density derivative of $\beta_\lambda(R)$ is $(1 - \lambda R)/(\sqrt{1 - \lambda^2} D(\lambda, R))$.

Differentiate the quotient $(R - c)/\sqrt{1 - c^2}$ inside the arctangent, holding R fixed. Simplification using $s^2 = 1 - \lambda^2$ yields

$$\left. \frac{d}{dc} \beta_c(R) \right|_{c=\lambda} = \frac{1 - \lambda R}{\sqrt{1 - \lambda^2} D(\lambda, R)}.$$

```
lemma betaAtRadius_density_hasDerivAt {lam R : ℝ}
  (hlam0 : 0 < lam) (hlam1 : lam < 1) :
  HasDerivAt (fun c => betaAtRadius c R)
    ((1 - lam * R) /
      (Real.sqrt (1 - lam ^ 2) * fixedRadiusD lam R)) lam := by
  have hs := sqrt_one_sub_sq_pos hlam0 hlam1
  have hs2 := sq_sqrt_one_sub_sq hlam0 hlam1
  have hd := fixedRadiusD_pos (R := R) hlam0 hlam1
  have hq := ((hasDerivAt_const lam R).sub (hasDerivAt_id lam)).div
    (sqrt_one_sub_sq_hasDerivAt hlam0 hlam1) hs.ne'
  have hb := (hasDerivAt_const lam (Real.pi / 2)).sub hq.arctan
  convert hb using 1 <;> try rfl
  simp only [Pi.sub_apply, Pi.div_apply, id_eq]
  field_simp [hs.ne', hd.ne']
  unfold fixedRadiusD
  rw [hs2]
  ring
```

CMVBallCase.betaAtRadius_hasDerivAt

Classification: supplement

Source: CMVBallCase.lean:378-392

Declaration SHA-256: 7bfaeb583b2974229111c194bad221441e7cfff9af27728eb440c5e3349f59cc9

Mathematical statement. For $0 < \lambda < 1$ and every real R , β_λ has derivative $-s/((R - \lambda)^2 + s^2)$ at R .

Differentiate $\pi/2 - \arctan((R - \lambda)/s)$, holding λ fixed, to obtain

$$\beta'_\lambda(R) = -\frac{s}{(R - \lambda)^2 + s^2}.$$

Since $s > 0$, the denominator never vanishes.

```
lemma betaAtRadius_hasDerivAt {lam R : ℝ}
  (hlam0 : 0 < lam) (hlam1 : lam < 1) :
  HasDerivAt (betaAtRadius lam)
    (-Real.sqrt (1 - lam ^ 2) /
      ((R - lam) ^ 2 + Real.sqrt (1 - lam ^ 2) ^ 2)) R := by
  have hs : Real.sqrt (1 - lam ^ 2) ≠ 0 :=
    (sqrt_one_sub_sq_pos hlam0 hlam1).ne'
  have hlin : HasDerivAt (fun x : ℝ => x - lam) 1 R := by
    simpa only [id_eq] using (hasDerivAt_id R).sub_const lam
  have hq := hlin.div_const (Real.sqrt (1 - lam ^ 2))
  have ha := hq.arctan
  have hb := (hasDerivAt_const R (Real.pi / 2)).sub ha
  convert hb using 1 <;>
  first | rfl | (field_simp [hs] <;> ring)
```

CMVBallCase.betaAtRadius_pos

Classification: supplement

Source: CMVBallCase.lean:133-138

Declaration SHA-256: 9bec3a5f14c974a03f3298e5c5d48fc6f60c6514fa386f2e0d0afaa8a38cf980

Mathematical statement. For $0 < \lambda < 1$ and every real R , $\beta_\lambda(R) > 0$.

The real arctangent is strictly below $\pi/2$, so its difference from $\pi/2$ is positive. No nonnegativity assumption on R is needed for this bound.

```
lemma betaAtRadius_pos {lam R : ℝ} (hlam0 : 0 < lam) (hlam1 : lam < 1) :  
  0 < betaAtRadius lam R := by  
  unfold betaAtRadius  
  linarith [Real.arctan_lt_pi_div_two  
    ((R - lam) / Real.sqrt (1 - lam ^ 2))]
```

CMVBallCase.commonPerimeterAreaGapAbove

Classification: supplement

Source: CMVBallCase.lean:2235-2238

Declaration SHA-256: e2b63b71cf44b62491a0ebd0b8526e6a8332a7c063ef2fe57ee5f88f8d32ce4f

Mathematical statement. For an admissible density and $p > 2\lambda\pi$, the real function $G_p(q)$ is $G(\max(q, p))$.

The definition is

$$G_p(q) = G(\max(q, p)).$$

It is a constant extension below p and agrees with G for $q \geq p$. Only its restriction to that upper interval is used; it does not assign inverse radii to inadmissible perimeters.

```
def commonPerimeterAreaGapAbove {lam : ℝ} (hlam : AdmissibleDensity lam)  
  (p : CommonPerimeter lam) (q : ℝ) : ℝ :=  
  commonPerimeterAreaGap hlam (commonPerimeterClamp p q)
```

CMVBallCase.commonPerimeterAreaGapAbove_hasDerivWithinAt

Classification: supplement

Source: CMVBallCase.lean:2239-2266

Declaration SHA-256: 2420c176e6ebf43330a3d88b7b4b75bdaec4c565775a067f362c214f4599e5c2

Mathematical statement. For $0 < \lambda < 1$, $p > 2\lambda\pi$, and $q \geq p$, G_p has derivative within $[p, \infty)$ equal to $r_B(q) - r_C(q)$.

Local agreement with a differentiable representative of G proves the relative derivative formula

$$\lim_{\substack{x \rightarrow q, x \geq p \\ x \neq q}} \frac{G_p(x) - G_p(q)}{x - q} = r_B(q) - r_C(q).$$

At $q = p$ this is a one-sided derivative, not a claim of ordinary differentiability across the clamping point.

```
lemma commonPerimeterAreaGapAbove_hasDerivWithinAt {lam : ℝ}  
  (hlam : AdmissibleDensity lam) (p : CommonPerimeter lam)  
  {q : ℝ} (hq : p.1 ≤ q) :  
  HasDerivWithinAt (commonPerimeterAreaGapAbove hlam p)  
    ((typeBRadiusAtPerimeter hlam ⟨q, p.2.trans_le hq⟩).1 -  
     (typeCRadiusAtPerimeter hlam ⟨q, p.2.trans_le hq⟩).1)  
    (Set.Ici p.1) q := by  
  let qs : CommonPerimeter lam := ⟨q, p.2.trans_le hq⟩  
  rcases commonPerimeterAreaGap_local_hasDerivAt hlam qs with
```

```

    (gap, hgap, hagree)
  have hagree' :
    ∀f x in nhdsWithin q (Set.Ici p.1),
      ∀ hx : 2 * lam * Real.pi < x,
        gap x = commonPerimeterAreaGap hlam (x, hx) :=
    mem_nhdsWithin_of_mem_nhds hagree
  have heq : commonPerimeterAreaGapAbove hlam p =f[nhdsWithin q (Set.Ici p.1)]
    gap := by
      filter_upwards [hagree', self_mem_nhdsWithin] with x hxagree hxmem
      have hxdomain : 2 * lam * Real.pi < x := p.2.trans_le hxmem
      rw [commonPerimeterAreaGapAbove,
        commonPerimeterClamp_eq p hxmem]
      exact (hxagree hxdomain).symm
  have hwithin :=
    hgap.hasDerivWithinAt.congr_of_eventuallyEq_of_mem heq hq
  simpa [qs] using hwithin

/-- The concrete common-perimeter area gap is strictly decreasing throughout
the genuine open perimeter domain. -/

```

CMVBallCase.commonPerimeterClamp

Classification: supplement

Source: CMVBallCase.lean:2223-2226

Declaration SHA-256: 4b4d169c7aed59d327b47ade1e0c9f86cfd6aa971b1e181dd947fb1ce217487c

Mathematical statement. For $p > 2\lambda\pi$ and real q , the clamped common perimeter is $\max(q, p)$.

The clamped value satisfies

$$\max(q, p) \geq p > 2\lambda\pi.$$

This transports a function on the constrained perimeter domain to an ordinary real argument for interval calculus.

```

def commonPerimeterClamp {lam : ℝ} (p : CommonPerimeter lam) (q : ℝ) :
  CommonPerimeter lam :=
  ⟨max q p.1, p.2.trans_le (le_max_right q p.1)⟩

```

CMVBallCase.commonPerimeterClamp_eq

Classification: supplement

Source: CMVBallCase.lean:2227-2234

Declaration SHA-256: e3584ce3c3e58723599d9275ac8d07529840c51ccaf5b4e38a9fb3ce88756176

Mathematical statement. If $p > 2\lambda\pi$ and $q \geq p$, clamping q at p leaves q unchanged.

Under $p \leq q$,

$$\max(q, p) = q.$$

This also identifies the corresponding constrained-domain values, so clamping does not alter the gap anywhere on the interval used by the mean value theorem.

```

lemma commonPerimeterClamp_eq {lam : ℝ} (p : CommonPerimeter lam)
  {q : ℝ} (hq : p.1 ≤ q) :
  commonPerimeterClamp p q = ⟨q, p.2.trans_le hq⟩ := by
  apply Subtype.ext
  exact max_eq_left hq

/-- Real-valued transport of the genuine gap above a fixed genuine perimeter.
Only its restriction to that upper interval is used. -/

```

CMVBallCase.cos_bAtRadius_eq_radius_mul_sin

Classification: supplement

Source: CMVBallCase.lean:524-534

Declaration SHA-256: 809c57b3c3abce1f1b8d70a94abd2bbd7102bca4c59fe3ebbe305dbaa59362d5

Mathematical statement. For $R > 0$, $\cos b(R) = R \sin b(R)$.

The inverse equation $\cot b(R) = R$ and positive sine give

$$\cos b(R) = R \sin b(R).$$

Its square identifies the two segment contributions in the derivative of the overlap area.

```
lemma cos_bAtRadius_eq_radius_mul_sin {R : ℝ} (hR : 0 < R) :  
  Real.cos (bAtRadius R) = R * Real.sin (bAtRadius R) := by  
  have hadm := bAtRadius_admissible hR  
  have hs : Real.sin (bAtRadius R) ≠ 0 :=  
    (Real.sin_pos_of_pos_of_lt_pi hadm.1 (by  
      linarith [hadm.2, Real.pi_pos])).ne'  
  have hr := typeCRadius_bAtRadius hR  
  rw [typeCRadius, one_div, Real.tan_inv_eq_cot,  
    Real.cot_eq_cos_div_sin] at hr  
  exact (div_eq_iff hs).mp hr
```

CMVBallCase.cot_betaAtRadius

Classification: supplement

Source: CMVBallCase.lean:157-162

Declaration SHA-256: b6035ec4d3398844c66860e34566e7cf25d89bc622f0a05527f8328bc5c348c9

Mathematical statement. For $0 < \lambda < 1$ and all real R , $\cot \beta_\lambda(R) = (R - \lambda) / \sqrt{1 - \lambda^2}$.

The complementary-angle identity turns the cotangent into the tangent of an arctangent, giving

$$\cot \beta_\lambda(R) = \frac{R - \lambda}{\sqrt{1 - \lambda^2}}.$$

This exact identity is valid on the entire real radius expression, before imposing the geometric radius domain.

```
lemma cot_betaAtRadius {lam R : ℝ} (hlam0 : 0 < lam) (hlam1 : lam < 1) :  
  Real.cot (betaAtRadius lam R) =  
    (R - lam) / Real.sqrt (1 - lam ^ 2) := by  
  rw [betaAtRadius, ← Real.tan_inv_eq_cot, ← Real.tan_pi_div_two_sub]  
  convert Real.tan_arctan ((R - lam) / Real.sqrt (1 - lam ^ 2)) using 1 <|> ring
```

CMVBallCase.cot_strictAntiOn_Ioo_zero_pi

Classification: supplement

Source: CMVBallCase.lean:92-102

Declaration SHA-256: ace6f8c9441fb437f575beea814ae69da33538450d13d0301b36483ea0bd1558

Mathematical statement. Cotangent is strictly decreasing on $(0, \pi)$.

For $0 < x < y < \pi$, $\cot y < \cot x$. The proof uses $\cot x = \tan(\pi/2 - x)$ and strict increase of tangent on $(-\pi/2, \pi/2)$.

```
lemma cot_strictAntiOn_Ioo_zero_pi :  
  StrictAntiOn Real.cot (Set.Ioo 0 Real.pi) := by  
  intro x hx y hy hxy  
  have htan : Real.tan (Real.pi / 2 - y) < Real.tan (Real.pi / 2 - x) :=  
    Real.strictMonoOn_tan  
    (by linarith [hy.2, Real.pi_pos], by linarith [hy.1])
```

```

    (by linarith [hx.2, Real.pi_pos], by linarith [hx.1])
    (by linarith)
    simp only [Real.tan_pi_div_two_sub, Real.tan_inv_eq_cot] using htan

/-- Type-(B) radius is strictly decreasing; no hidden angular branch remains. -/

```

CMVBallCase.fixedRadiusD_density_hasDerivAt

Classification: supplement

Source: CMVBallCase.lean:1455-1461

Declaration SHA-256: 8c046ed0856b99b8d7465ed1688893c4d60d5bfff0186da511b6237c36d6dc073

Mathematical statement. For all real λ and R , $c \mapsto D(c, R)$ has derivative $-2R$ at λ .

The expanded expression and its derivative are

$$D(c, R) = R^2 - 2Rc + 1, \quad \frac{d}{dc}D(c, R) = -2R.$$

Thus its density derivative is constant, even though the unexpanded expression contains two quadratic terms.

```

lemma fixedRadiusD_density_hasDerivAt (lam R : ℝ) :
  HasDerivAt (fun c => fixedRadiusD c R) (-2 * R) lam := by
  unfold fixedRadiusD
  convert (((hasDerivAt_const lam R).sub (hasDerivAt_id lam)).pow 2).add
    ((hasDerivAt_const lam 1).sub ((hasDerivAt_id lam).pow 2)) using 1 <.>
  first | rfl | (simp only [Pi.sub_apply, id_eq]; ring)

```

CMVBallCase.sin_bAtRadius_mul_cos

Classification: supplement

Source: CMVBallCase.lean:424-440

Declaration SHA-256: 10fbbcdf677fa47ab4e65fa4fcec664bfcee9cb8b5e893c8c90bb4f234de0fd3

Mathematical statement. For every nonzero real R , $\sin b(R)\cos b(R) = R/(1 + R^2)$.

The sine and cosine formulas for $\arctan(1/R)$ have a common square-root denominator. Multiplying and simplifying gives

$$\sin b(R)\cos b(R) = \frac{R}{1 + R^2},$$

for either sign of nonzero R .

```

lemma sin_bAtRadius_mul_cos {R : ℝ} (hR : R ≠ 0) :
  Real.sin (bAtRadius R) * Real.cos (bAtRadius R) =
    R / (1 + R ^ 2) := by
  let q := Real.sqrt (1 + (1 / R) ^ 2)
  have hq : q ^ 2 = 1 + (1 / R) ^ 2 :=
    Real.sq_sqrt (by positivity)
  have hq0 : q ≠ 0 := by positivity
  rw [bAtRadius, Real.sin_arctan, Real.cos_arctan]
  change ((1 / R) / q) * (1 / q) = _
  calc
    ((1 / R) / q) * (1 / q) = (1 / R) / q ^ 2 := by
      field_simp [hq0]
    _ = R / (1 + R ^ 2) := by
      rw [hq]
      field_simp [hR]
      ring

```

CMVBallCase.sin_betaAtRadius_mul_cos

Classification: supplement

Source: CMVBallCase.lean:403-423

Declaration SHA-256: e81216d3d0ee48ad5f9aec6192ac30e08e48ebe31d1c00ea3a7ac17be472a58d

Mathematical statement. For $0 < \lambda < 1$ and all real R , $\sin\beta_\lambda(R)\cos\beta_\lambda(R) = (R - \lambda)s/((R - \lambda)^2 + s^2)$.

Complementary-angle and arctangent identities give

$$\sin\beta_\lambda(R)\cos\beta_\lambda(R) = \frac{(R - \lambda)s}{(R - \lambda)^2 + s^2}.$$

This rational expression supplies the cancellation in the type-B perimeter derivative.

```
lemma sin_betaAtRadius_mul_cos {lam R : ℝ}
  (hlam0 : 0 < lam) (hlam1 : lam < 1) :
  Real.sin (betaAtRadius lam R) * Real.cos (betaAtRadius lam R) =
    (R - lam) * Real.sqrt (1 - lam ^ 2) /
      ((R - lam) ^ 2 + Real.sqrt (1 - lam ^ 2) ^ 2) := by
  have hs : 0 < Real.sqrt (1 - lam ^ 2) :=
    sqrt_one_sub_sq_pos hlam0 hlam1
  let x := (R - lam) / Real.sqrt (1 - lam ^ 2)
  have hq : Real.sqrt (1 + x ^ 2) ^ 2 = 1 + x ^ 2 :=
    Real.sq_sqrt (by positivity)
  have hq0 : Real.sqrt (1 + x ^ 2) ≠ 0 := by positivity
  rw [betaAtRadius, Real.sin_pi_div_two_sub, Real.cos_pi_div_two_sub,
    Real.cos_arctan, Real.sin_arctan]
  change (1 / Real.sqrt (1 + x ^ 2)) *
    (x / Real.sqrt (1 + x ^ 2)) = _
  field_simp [hs.ne', hq0]
  rw [hq]
  dsimp [x]
  field_simp [hs.ne']
  ring
```

CMVBallCase.sin_typeBAlpha_betaAtRadius

Classification: supplement

Source: CMVBallCase.lean:513-523

Declaration SHA-256: 6ddcb65dd46c70fb4136025d8ae05d542b93e1cc2d2cac116509821e0248fdb0

Mathematical statement. For $0 < \lambda < 1$ and $R \geq 0$, $\sin(\beta_\lambda(R) + \arccos\lambda) = R\sin\beta_\lambda(R)$.

Multiply the defining radius equation by its nonzero denominator to obtain

$$\sin(\beta_\lambda(R) + a) = R\sin\beta_\lambda(R).$$

This cancels the angular terms in the area derivative.

```
lemma sin_typeBAlpha_betaAtRadius {lam R : ℝ}
  (hlam0 : 0 < lam) (hlam1 : lam < 1) (hR : 0 ≤ R) :
  Real.sin (betaAtRadius lam R + Real.arccos lam) =
    R * Real.sin (betaAtRadius lam R) := by
  have hadm := betaAtRadius_admissible hlam0 hlam1 hR
  have hs : Real.sin (betaAtRadius lam R) ≠ 0 :=
    (typeB_sin_pos hlam0 hlam1 hadm).ne'
  have hr := typeBRadius_betaAtRadius hlam0 hlam1 hR
  rw [typeBRadius, typeBAlpha] at hr
  exact (div_eq_iff hs).mp hr
```

CMVBallCase.sq_sqrt_one_sub_sq

Classification: supplement

Source: CMVBallCase.lean:58-62

Declaration SHA-256: 398e78a14ca43373831b820093b0241c39626cd64fae57944b26894a61e0287e

Mathematical statement. For $0 < \lambda < 1$, the square of $\sqrt{1 - \lambda^2}$ is $1 - \lambda^2$.

Nonnegativity of the radicand gives the identity

$$s^2 = 1 - \lambda^2.$$

It is used to simplify rational derivative formulas without changing their domains.

```
lemma sq_sqrt_one_sub_sq {lam : ℝ} (hlam0 : 0 < lam) (hlam1 : lam < 1) :  
  Real.sqrt (1 - lam ^ 2) ^ 2 = 1 - lam ^ 2 :=  
  Real.sq_sqrt (by nlinarith)  
  
/-- The type-(B) parameter has positive sine on its complete finite domain. -/
```

CMVBallCase.sqrt_one_sub_sq_hasDerivAt

Classification: supplement

Source: CMVBallCase.lean:1428-1436

Declaration SHA-256: 1f9fdbe32f5f9bb2811002baeba3f0bd3fa1478086f09a6412852133afef60cf

Mathematical statement. For $0 < \lambda < 1$, $c \mapsto \sqrt{1 - c^2}$ has derivative $-\lambda/\sqrt{1 - \lambda^2}$ at λ .

The radicand is strictly positive at λ , so differentiating the square root gives

$$\left. \frac{d}{dc} \sqrt{1 - c^2} \right|_{c=\lambda} = -\frac{\lambda}{\sqrt{1 - \lambda^2}}.$$

This is the elementary chain-rule input for density derivatives of the angle and gap.

```
lemma sqrt_one_sub_sq_hasDerivAt {lam : ℝ}  
  (hlam0 : 0 < lam) (hlam1 : lam < 1) :  
  HasDerivAt (fun c : ℝ => Real.sqrt (1 - c ^ 2))  
    (-lam / Real.sqrt (1 - lam ^ 2)) lam := by  
  have h := ((hasDerivAt_const lam 1).sub ((hasDerivAt_id lam).pow 2)).sqrt  
    (ne_of_gt (show 0 < 1 - lam ^ 2 by nlinarith))  
  convert h using 1 <=> first | rfl |  
    (simp only [Pi.sub_apply, Pi.pow_apply, id_eq]; ring)
```

CMVBallCase.sqrt_one_sub_sq_pos

Classification: supplement

Source: CMVBallCase.lean:53-57

Declaration SHA-256: 0bd6d03a5431ddb7e353eee713defc6c88eb6f034d3e70eb881f76b26abc6488

Mathematical statement. If $0 < \lambda < 1$, then $\sqrt{1 - \lambda^2} > 0$.

The density hypotheses give $1 - \lambda^2 > 0$. Taking its nonnegative square root gives a strictly positive coefficient, allowing division in the type-B chart.

```
lemma sqrt_one_sub_sq_pos {lam : ℝ} (hlam0 : 0 < lam) (hlam1 : lam < 1) :  
  0 < Real.sqrt (1 - lam ^ 2) := by  
  apply Real.sqrt_pos.2  
  nlinarith
```

CMVBallCase.typeBAlpha_sin_nonneg

Classification: supplement

Source: CMVBallCase.lean:72-80

Declaration SHA-256: 84b0b6514b1ce040ec07e3234a2e8730ab3f09c5e1a14930d078033a2d70a73f

Mathematical statement. For every admissible type-B angle β , $\sin(\beta + \arccos\lambda)$ is nonnegative, without a separate density assumption.

No separate density assumption is required: admissibility and $\arccos\lambda \geq 0$ put $\alpha = \beta + \arccos\lambda$ in $[0, \pi]$. Its sine is nonnegative, including the endpoint $\alpha = \pi$.

```
lemma typeBAlpha_sin_nonneg {lam beta : ℝ}
  (hbeta : BAdmissible lam beta) :
  0 ≤ Real.sin (typeBAlpha lam beta) := by
  rw [typeBAlpha]
  apply Real.sin_nonneg_of_nonneg_of_le_pi
  · exact add_nonneg hbeta.1.le (Real.arccos_nonneg lam)
  · linarith [hbeta.2]

/-- Snell's law puts the type-(B) radius on a cotangent chart. -/
```

CMVBallCase.typeBPerimeterAt_typeBRadiusAtPerimeter

Classification: supplement

Source: CMVBallCase.lean:1947-1951

Declaration SHA-256: 8c6523ab6857fea0ddfb73478b78aea70dd1ba779a838fdf73e197a8305abde2

Mathematical statement. For $0 < \lambda < 1$ and $p > 2\lambda\pi$, $P_B(r_B(p)) = p$.

The defining perimeter equation of the inverse is

$$P_B(r_B(p)) = p.$$

It is extracted from the stored characterization and is an exact equality at a genuine common perimeter.

```
lemma typeBPerimeterAt_typeBRadiusAtPerimeter {lam : ℝ}
  (hlam : AdmissibleDensity lam) (p : CommonPerimeter lam) :
  typeBPerimeterAtRadius lam (typeBRadiusAtPerimeter hlam p) = p.1 :=
  (typeBRadiusAtPerimeter hlam p).2.2
```

CMVBallCase.typeBPerimeter_mem_values

Classification: supplement

Source: CMVBallCase.lean:1088-1092

Declaration SHA-256: ab1cda17a1dd925edbcd6476abecb0e519a6fb9430ca150e790c745fb1bfa2ae

Mathematical statement. An admissible type-B angle of area v contributes its perimeter to the type-B value set at v .

If β is admissible and $A_B(\beta) = v$, then $P_B(\beta) \in \mathcal{P}_B(v)$. This is the direct existential witness, with no density assumption.

```
lemma typeBPerimeter_mem_values {lam v beta : ℝ}
  (hbeta : BAdmissible lam beta) (harea : typeBArea lam beta = v) :
  typeBPerimeter lam beta ∈ typeBPerimeterValuesAtArea lam v :=
  ⟨beta, hbeta, harea, rfl⟩
```

CMVBallCase.typeBRadiusAtPerimeter_pos

Classification: supplement

Source: CMVBallCase.lean:1937-1941

Declaration SHA-256: 35f84c501e62b1b673fd6a9b793e1ea9f012ba0b179ed229e723b724fe0b5365

Mathematical statement. For $0 < \lambda < 1$ and $p > 2\lambda\pi$, $r_B(p) > 0$.

This extracts the positivity already contained in the inverse radius definition. It permits applying derivative and monotonicity results on the positive radius chart.

```
lemma typeBRadiusAtPerimeter_pos {lam : ℝ} (hlam : AdmissibleDensity lam)
  (p : CommonPerimeter lam) :
  0 < (typeBRadiusAtPerimeter hlam p).1 :=
  (typeBRadiusAtPerimeter hlam p).2.1
```

CMVBallCase.typeBRadiusAtPerimeter_unique

Classification: supplement

Source: CMVBallCase.lean:1958-1965

Declaration SHA-256: 45ce1f082ef5383bd43764a4e1518cd8d4004f8b048ca23100637518a3e785ee

Mathematical statement. If $0 < \lambda < 1$, $p > 2\lambda\pi$, $R > 0$, and $P_B(R) = p$, then $r_B(p) = R$.

Both radii solve the same positive-radius perimeter equation. Uniqueness gives

$$r_B(p) = R,$$

identifying the chosen global inverse with any local inverse or explicit positive candidate satisfying that equation.

```
lemma typeBRadiusAtPerimeter_unique {lam R : ℝ} (hlam : AdmissibleDensity lam)
  (p : CommonPerimeter lam) (hR : 0 < R)
  (hperim : typeBPerimeterAtRadius lam R = p.1) :
  (typeBRadiusAtPerimeter hlam p).1 = R :=
  (existsUnique_typeBRadius_at_perimeter hlam.1 hlam.2 p.2).unique
  (typeBRadiusAtPerimeter hlam p).2 ⟨hR, hperim⟩

/-- Right uniqueness of the C inverse on its positive radius chart. -/
```

CMVBallCase.typeB_sin_pos

Classification: supplement

Source: CMVBallCase.lean:63-71

Declaration SHA-256: 778d4c7a3220c8e895cdffc64fd5909fe596a22442647d2c fb4c145e12e3651e

Mathematical statement. For $0 < \lambda < 1$ and an admissible type-B angle β , $\sin\beta > 0$.

Since $0 < \arccos\lambda$, admissibility implies $0 < \beta < \pi$. Hence $\sin\beta > 0$, so the defining radius quotient has a nonzero denominator.

```
lemma typeB_sin_pos {lam beta : ℝ} (hlam0 : 0 < lam) (hlam1 : lam < 1)
  (hbeta : BAdmissible lam beta) :
  0 < Real.sin beta := by
  apply Real.sin_pos_of_pos_of_lt_pi hbeta.1
  have hacos : 0 < Real.arccos lam := Real.arccos_pos.2 hlam1
  linarith [hbeta.2]

/-- The exterior arc's source sine is nonnegative, including the
zero-radius endpoint where its angle is `π`. -/
```

CMVBallCase.typeCPerimeterAt_typeCRadiusAtPerimeter

Classification: supplement

Source: CMVBallCase.lean:1952-1957

Declaration SHA-256: 97f0a09e977f148c1547234ee78701dc58ab72d702573d3d019190ba9b227ce1

Mathematical statement. For $0 < \lambda < 1$ and $p > 2\lambda\pi$, $P_C(r_C(p)) = p$.

The existence theorem used to define r_C supplies the equation

$$P_C(r_C(p)) = p.$$

It identifies the actual perimeter of the radius selected by the inverse.

```
lemma typeCPerimeterAt_typeCRadiusAtPerimeter {lam : ℝ}
  (hlam : AdmissibleDensity lam) (p : CommonPerimeter lam) :
  typeCPerimeterAtRadius lam (typeCRadiusAtPerimeter hlam p) = p.1 :=
  (typeCRadiusAtPerimeter hlam p).2.2

/-- Right uniqueness of the B inverse on its positive radius chart. -/
```

CMVBallCase.typeCPerimeter_mem_values

Classification: supplement

Source: CMVBallCase.lean:1093-1097

Declaration SHA-256: ad514705a9dc71bb1959ff8334149fb5759b7a8f9d6bb4f82c2b59f07695ab1c

Mathematical statement. An admissible type-C angle of area v contributes its perimeter to the type-C value set at v .

If $0 < b < \pi/2$ and $A_C(b) = v$, the angle b witnesses $P_C(b) \in \mathcal{P}_C(v)$. No uniqueness hypothesis is used here.

```
lemma typeCPerimeter_mem_values {lam v beta : ℝ}
  (hbeta : CAdmissible beta) (harea : typeCArea lam beta = v) :
  typeCPerimeter lam beta ∈ typeCPerimeterValuesAtArea lam v :=
  ⟨beta, hbeta, harea, rfl⟩
```

CMVBallCase.typeCRadiusAtPerimeter_pos

Classification: supplement

Source: CMVBallCase.lean:1942-1946

Declaration SHA-256: 6d7f4a31d01f4556bf0e8a9719b79bf61ccb4965c124289a537df2d839c9fd39

Mathematical statement. For $0 < \lambda < 1$ and $p > 2\lambda\pi$, $r_C(p) > 0$.

Positivity is part of the defining characterization of the chosen inverse. The lemma exposes it for subsequent uses of the positive-radius analytic formulas.

```
lemma typeCRadiusAtPerimeter_pos {lam : ℝ} (hlam : AdmissibleDensity lam)
  (p : CommonPerimeter lam) :
  0 < (typeCRadiusAtPerimeter hlam p).1 :=
  (typeCRadiusAtPerimeter hlam p).2.1
```

CMVBallCase.typeCRadiusAtPerimeter_unique

Classification: supplement

Source: CMVBallCase.lean:1966-1974

Declaration SHA-256: 6d0bba8a25476b26089b665ae685627f40cb11720cbb717de9b249a9d0e5a11c

Mathematical statement. If $0 < \lambda < 1$, $p > 2\lambda\pi$, $R > 0$, and $P_C(R) = p$, then $r_C(p) = R$.

Uniqueness of the positive solution at perimeter p gives

$$r_C(p) = R.$$

It is used to identify the local inverse supplied by calculus with the genuine common-perimeter inverse.

```
lemma typeCRadiusAtPerimeter_unique {lam R : ℝ} (hlam : AdmissibleDensity lam)
  (p : CommonPerimeter lam) (hR : 0 < R)
  (hperim : typeCPerimeterAtRadius lam R = p.1) :
  (typeCRadiusAtPerimeter hlam p).1 = R :=
  (existsUnique_typeCRadius_at_perimeter hlam.1 hlam.2 p.2).unique
  (typeCRadiusAtPerimeter hlam p).2 ⟨hR, hperim⟩

/-- At common perimeter, the global same-radius inequality forces the B
radius strictly below the C radius. -/
```

Generated certificate families

External references

- **CMV** — Antonio Cañete; Michele Miranda Jr.; Davide Vittone, *Some isoperimetric problems in planes with density*, arXiv:0906.1256v1 [math.DG], 8 June 2009; Proposition 3.22, pp. 32–33; Theorem 3.23 and following remark, pp. 34–36; candidate formulas and Figure 19, pp. 34–35. <https://arxiv.org/abs/0906.1256v1>